RN

GUÍA INTERACTIVA

React Native con Expo

Apps para iOS y Android con React: componentes, estilos, listas, navegación con Expo Router, datos, librerías, y cómo publicar en App Store y Google Play.

Versión PDF · 1 de noviembre de 2026
21 capítulos · Los ejemplos interactivos están en la app (npm run dev)

Índice

Empezá acá

1. Cómo usar esta guía
2. Cómo funciona React Native

Parte 1 · Fundamentos

3. Crear un proyecto con Expo
4. Los componentes básicos
5. Estilos y layout
6. Listas
7. Formularios y teclado
8. Toques, gestos y animaciones

Parte 2 · Una app de verdad

9. Navegación con Expo Router
10. Datos del servidor
11. Estado y almacenamiento
12. Login y sesión
13. Cámara, notificaciones y más
14. Las librerías más usadas

Parte 3 · Calidad

15. Buenas prácticas y rendimiento
16. Tests y depuración

Parte 4 · Publicar

17. Builds con EAS
18. Subir a App Store y Google Play

19. Lo que piden las tiendas
20. Actualizaciones OTA y CI/CD

Parte 5 · Repaso

21. Chuleta y preguntas de entrevista

Empezá acá

Qué es React Native, cómo funciona por dentro y cómo usar esta guía y su app de ejemplo.

Capítulo 1

Cómo usar esta guía

React Native en un párrafo, las demos en el celular dibujado y la app de ejemplo para tu teléfono.

Esta guía es para hacer **apps de iOS y Android con React**. Si sabés React para la web, ya sabés la mitad: componentes, props, estado, hooks y TypeScript funcionan igual. Lo que cambia son los componentes con los que dibujás la pantalla, cómo se estiliza, cómo se navega, las funciones del teléfono (cámara, notificaciones, ubicación) y, sobre todo, **cómo se publica**: en la web hacés deploy; en el celular pasás por la revisión de Apple y de Google.

React Native en un párrafo

En lugar de `<div>` y `<span>`, escribís `<View>` y `<Text>`. React Native traduce cada uno a la vista

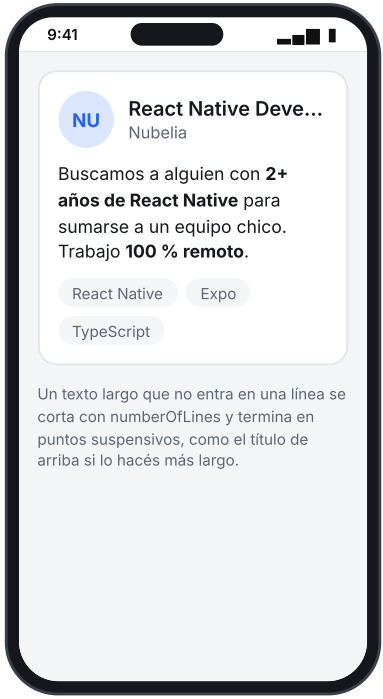
nativa del sistema: un `UIView` en iOS, un `android.view.View` en Android. No es una página web metida en una app: los botones, las listas y el teclado son los del sistema, y por eso se sienten nativos. Tu JavaScript corre en un motor propio (**Hermes**) dentro de la app.

Expo es el framework que se usa para hacer apps con React Native, y el que recomienda el equipo de React Native para empezar un proyecto. Te da el proyecto armado, la navegación (Expo Router), decenas de librerías para usar el hardware del teléfono, y servicios para compilar en la nube y subir a las tiendas (EAS). Lo vemos en cómo funciona.

Las demos corren adentro de un celular

Cada demo es un componente de React Native de verdad. En la guía se ve dentro de un celular dibujado, y debajo está su código:

DEMO Tu primera pantalla



app-expo/src/demos/componentes/PrimeraPantalla.tsx

```
import { ScrollView, StyleSheet, Text, View } from
  'react-native'
import { ofertas } from '../datos'
import { colores, espacio } from '../tema'

// View es la caja (como un div), Text es el único lugar
  donde puede ir texto,
// y ScrollView hace que el contenido se pueda
  desplazar.
```

```
export default function PrimeraPantalla() {
  const oferta = ofertas[0]

  return (
    <ScrollView style={styles.pantalla}
      contentContainerStyle={styles.contenido}>
      <View style={styles.tarjeta}>
        <View style={styles.encabezado}>
          {/* Un avatar con iniciales: un View redondo
            con un Text centrado */}
          <View style={styles.avatar}>
            <Text style={styles.iniciales}>
              {oferta.empresa.slice(0, 2).toUpperCase()}
              </Text>
          </View>
          <View style={styles.flex}>
            <Text style={styles.titulo} numberOfLines=
              {1}>
              {oferta.titulo}
            </Text>
            <Text style={styles.suave}>{oferta.empresa}
              </Text>
          </View>
        </View>

        {/* Los Text se pueden anidar: el de adentro
          hereda y suma estilos */}
        <Text style={styles.parrafo}>
          Buscamos a alguien con <Text style=
            {styles.negrita}>2+ años de React
            Native</Text> para
          sumarse a un equipo chico. Trabajo <Text
            style={styles.negrita}>100 % remoto</Text>.
        </Text>

        <View style={styles.chips}>
          {oferta.stack.map((tecnologia) => (
            <View key={tecnologia} style={styles.chip}>
              <Text style={styles.chipTexto}>
                {tecnologia}</Text>
            </View>
          ))}
```

```
        </View>
      </View>

      <Text style={styles.nota}>
        Un texto largo que no entra en una línea se
          corta con numberOfLines y termina en puntos
        suspensivos, como el título de arriba si lo
          hacés más largo.
      </Text>
    </ScrollView>
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.fondo },
  contenido: { padding: espacio.l, gap: espacio.l },
  tarjeta: {
    backgroundColor: colores.tarjeta,
    borderRadius: 16,
    padding: espacio.l,
    gap: espacio.m,
    borderWidth: StyleSheet.hairlineWidth,
    borderColor: colores.borde,
  },
  encabezado: { flexDirection: 'row', alignItems:
    'center', gap: espacio.m },
  avatar: {
    width: 48,
    height: 48,
    borderRadius: 24,
    backgroundColor: colores.primarioSuave,
    alignItems: 'center',
    justifyContent: 'center',
  },
  iniciales: { color: colores.primario, fontWeight:
    '700', fontSize: 16 },
  flex: { flex: 1 },
  titulo: { fontSize: 17, fontWeight: '600', color:
    colores.texto },
  suave: { color: colores.textoSuave, marginTop: 2 },
```

```
  parrafo: { fontSize: 15, lineHeight: 22, color:
    colores.texto },
  negrita: { fontWeight: '700' },
  chips: { flexDirection: 'row', flexWrap: 'wrap', gap:
    espacio.s },
  chip: {
    backgroundColor: colores.fondo,
    borderRadius: 999,
    paddingVertical: espacio.xs,
    paddingHorizontal: espacio.m,
  },
  chipTexto: { fontSize: 13, color: colores.textoSuave
    },
  nota: { color: colores.textoSuave, fontSize: 13,
    lineHeight: 19 },
})
```

En el navegador, esas demos usan **react-native-web**, una librería que dibuja `View`, `Text`, `FlatList`, etc. con elementos HTML. Es la misma que usa Expo cuando corrés tu app en la web. Se parece mucho a lo que ves en el teléfono, pero no es idéntico: las fuentes, las sombras, el teclado y los gestos se sienten distintos. Para eso está la app.

La app de ejemplo: en tu teléfono

Todo el código de esta guía vive en la carpeta `app-expo/` del proyecto: una app completa de búsqueda de empleo hecha con Expo, con login, lista de ofertas con búsqueda y scroll infinito, detalle, ofertas guardadas, foto de perfil y una pestaña **Demos** con las mismas demos de la guía, corriendo nativas.

Para abrirla en tu celular:

1. Instalá **Expo Go** desde la App Store o Google Play.
2. En la computadora:

```
cd app-expo
npm install
npx expo start
```

3. Escaneá el código QR que aparece en la terminal: con la cámara en iPhone, o desde Expo Go en Android. La computadora y el teléfono tienen que estar en la misma red Wi-Fi (si no, `npx expo start --tunnel`).
4. Entrá con cualquier email y la contraseña `expo1234`.

Cada vez que guardás un archivo, la app se actualiza sola en el teléfono (**Fast Refresh**), sin perder el estado.

> 💡 **Sin teléfono a mano**
>
> Con `npx expo start` también podés apretar `w` para abrirla en el navegador, `i` para el simulador de iOS (necesita una Mac con Xcode) o `a` para el emulador de Android (necesita Android Studio).

Los snippets que no son demos (gestos con Reanimated, SQLite, notificaciones, cámara...) también son archivos del proyecto, en `app-`

`expo/src/ejemplos/`, y pasan el chequeo de tipos de TypeScript. El código que leés compila.

> ℹ️ **Versiones**
>
> Escrita en noviembre de 2026 con **Expo SDK 58**, que trae **React Native 0.88** y **React 19.3**. Expo saca tres SDK por año y cada uno trae una versión nueva de React Native. Expo Go solo abre proyectos del último SDK: si un día no abre la app, actualizá el proyecto con `npx expo install expo@latest --fix`.

Cómo está armada

1. **Fundamentos**: crear el proyecto, los componentes básicos, estilos, listas, formularios y animaciones.
2. **Una app de verdad**: navegación con Expo Router, datos del servidor con TanStack Query, estado y almacenamiento, login, funciones del teléfono y una tabla de las librerías más usadas.
3. **Calidad**: buenas prácticas, rendimiento, accesibilidad, tests y depuración.
4. **Publicar**: builds con EAS, cómo subir a App Store y Google Play, **todo lo que piden las tiendas**, y actualizaciones sin pasar por la tienda.
5. **Repaso**: chuleta y preguntas de entrevista.

Si venís de la guía de React, todo lo de hooks y React 19 vale acá: `useState`, `useEffect`, `use`, `useOptimistic`, el React Compiler (que en Expo viene activado).

Capítulo 2

Cómo funciona React Native

Componentes nativos, la Nueva Arquitectura, Hermes, y por qué Expo.

No hace falta saber cómo funciona por dentro para hacer una app, pero te lo van a preguntar en entrevistas, y ayuda a entender por qué algunas cosas son lentas, por qué algunas librerías no andan en Expo Go y qué significa "Nueva Arquitectura".

Tres partes: JavaScript, nativo y el puente entre los dos

Una app de React Native es una app nativa normal (un proyecto de Xcode y uno de Android Studio) que trae adentro:

- **Hermes**, un motor de JavaScript hecho para React Native. Compila tu código a *bytecode* al hacer el build, así la app arranca rápido y usa poca memoria. Desde el SDK 56, Expo usa **Hermes v1**.

- **Tu bundle de JavaScript**: todos tus componentes, empaquetados por **Metro** (el "Vite" de React Native).
- **El renderer (Fabric)**: cuando React calcula qué cambió, Fabric crea o actualiza las vistas nativas. Tu `<Text>` termina siendo un `UITextView` / `TextView` de verdad.



La Nueva Arquitectura

Hasta 2024, JavaScript y lo nativo se hablaban por un **bridge**: mensajes JSON asincrónicos, en tandas. Era lento para cosas como medir un elemento o seguir el dedo en un gesto. La **Nueva Arquitectura** lo reemplazó por **JSI** (JavaScript Interface): JavaScript llama funciones de C++ directamente, de forma sincrónica si hace falta, sin serializar nada.

Pieza	Qué hace
JSI	JavaScript tiene referencias a objetos de C++ y llama sus métodos directo.

Pieza	Qué hace
Fabric	El renderer nuevo: soporta las features concurrentes de React 18/19 (`useTransition`, Suspense) y medir el layout sincrónicamente.
TurboModules	Los módulos nativos (cámara, almacenamiento...) se cargan recién cuando los usás.
Codegen	Genera el código que une los tipos de TypeScript con el lado nativo.

Desde el **SDK 55** la arquitectura vieja ya no existe: todas las librerías que uses tienen que ser compatibles con la nueva. Las populares lo son hace rato.

> 🎤 **En una entrevista**
>
> "¿Qué es la Nueva Arquitectura?" Respuesta corta: reemplaza el bridge asincrónico (mensajes JSON) por JSI, que permite llamar código nativo directo desde JavaScript; Fabric es el nuevo renderer y TurboModules los módulos nativos con carga diferida. El resultado: menos latencia, layout sincrónico, y soporte para las features concurrentes de React.

Hilos: por qué una animación se traba

React Native usa principalmente dos hilos:

- **El hilo de JavaScript**: corre React, tus componentes, tus `useEffect`, tus `fetch`.
- **El hilo principal (UI)**: dibuja la pantalla y recibe los toques.

Si el hilo de JS está ocupado (un render enorme, un `JSON.parse` de 5 MB, un loop pesado), todo lo que dependa de él se congela: un botón tarda en responder, una animación hecha con `setState` salta. Por eso las animaciones y gestos serios corren en el hilo de UI con **Reanimated** (lo vemos en interacción) y las listas largas virtualizan las filas.

La meta es **60 cuadros por segundo** (120 en pantallas ProMotion): unos 16 ms por cuadro.

Por qué Expo

Podés crear una app con React Native "pelado" (`npx @react-native-community/cli init`), pero tendrías que resolver vos la navegación, los íconos, el splash, los permisos, las builds, las firmas... Expo es el framework que trae todo eso, y la documentación oficial de React Native recomienda empezar con un framework.

Pieza de Expo	Qué resuelve
Expo SDK	Más de 50 librerías (`expo-camera`, `expo-image`, `expo-notifications`...) que funcionan igual en iOS, Android y web.
Expo Router	Navegación con archivos, como en Next.js.

Pieza de Expo	Qué resuelve
Prebuild (CNG)	Las carpetas `ios/` y `android/` se **generan** a partir de `app.json`. No las editás a mano ni las subís a git.
Config plugins	Cada librería sabe qué cambiar en esos proyectos nativos (permisos, claves).
Expo Go	Una app de las tiendas para probar tu proyecto sin compilar nada.
Development builds	Tu propia "Expo Go", con tus librerías nativas.
EAS	Servicios en la nube: Build, Submit, Update, Workflows.

> ℹ️ **CNG: Continuous Native Generation**
>
> Es la idea más importante de Expo. Las carpetas nativas son un resultado, como `dist/` en Vite: `npx expo prebuild` las genera desde `app.json` y los config plugins. Por eso el `.gitignore` del proyecto tiene `/ios` y `/android`. Si necesitás tocar algo nativo, lo hacés con un config plugin, y el cambio sobrevive a cada actualización del SDK.

Expo Go o development build

Expo Go es una app que ya trae compiladas todas las librerías del Expo SDK. Por eso podés escanear un QR y ver tu app sin instalar Xcode: tu JavaScript se carga

adentro de Expo Go. Es ideal para aprender y para esta guía.

El límite: **solo tiene las librerías nativas que trae**. Si instalás una que no está (por ejemplo `react-native-mmkv`, Stripe o Sentry con código nativo), necesitás un **development build**: una versión de Expo Go hecha con tus dependencias. Se crea con `eas build --profile development` o localmente con `npx expo run:ios` / `npx expo run:android`.

	Expo Go	Development build
Instalar	Desde la tienda	La compilás vos (EAS o local)
Librerías nativas	Solo las del Expo SDK	Cualquiera
Ícono, nombre, splash, permisos propios	No (son los de Expo Go)	Sí
Notificaciones push en Android	No	Sí
Para	Aprender, prototipos	Cualquier proyecto real

En la práctica: arrancás con Expo Go, y el día que agregás una librería nativa que no está, pasás a un development build. Tu código no cambia.

Parte 1 · Fundamentos

Crear el proyecto y los bloques de toda pantalla: componentes, estilos, listas, formularios y animaciones.

Capítulo 3

Crear un proyecto con Expo

create-expo-app, la estructura, Expo Go o development build, y los comandos del día a día.

Crear el proyecto

```
npx create-expo-app@latest mi-app
cd mi-app
npx expo start
```

La plantilla por defecto ya trae **TypeScript**, **Expo Router** con pestañas, el **React Compiler** activado y modo oscuro. Si querés arrancar de cero: `npx create-`

`expo-app@latest mi-app --template blank-typescript`.

Necesitás **Node 22.13 o más nuevo** (el SDK 58 lo exige). No hace falta instalar Xcode ni Android Studio para empezar: con Expo Go en el teléfono alcanza.

La estructura

Así quedó la app de esta guía (`app-expo/`), que salió de esa plantilla:

```
app-expo/
├── app.json              ← configuración de la app: nombre, ícono, permisos, plugins
├── eas.json              ← perfiles de build para EAS (lo vemos al publicar)
├── package.json          ← "main": "expo-router/entry"
├── tsconfig.json         ← alias @/ → src/
├── assets/               ← íconos, splash, imágenes, fuentes
└── src/
    ├── app/              ← las PANTALLAS: cada archivo es una ruta (Expo Router)
    │   ├── _layout.tsx   ← el layout raíz: providers, Stack, splash
    │   ├── login.tsx     ← /login
    │   ├── (tabs)/       ← un grupo con pestañas (no suma nada a la URL)
    │   └── oferta/[id].tsx  ← /oferta/5
    ├── components/       ← componentes reutilizables
    ├── hooks/  stores/  api/  lib/
    └── demos/            ← las demos de esta guía
```

La regla de Expo Router: **en `src/app/` solo van pantallas y layouts**. Cualquier otro archivo que pongas ahí se convierte en una ruta. Los componentes, hooks y utilidades van afuera.

app.json: la configuración de la app

Todo lo que en un proyecto nativo estaría repartido entre Xcode y Android Studio (nombre, ícono, identificador, permisos, splash) va en `app.json`. Expo genera los proyectos nativos a partir de este archivo.

app-expo/app.json

```
{
  "expo": {
    "name": "Búsqueda RN",
    "slug": "guia-react-native",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/images/icon.png",
    "scheme": "busquedarn",
    "userInterfaceStyle": "light",
    "ios": {
      "icon": "./assets/expo.icon",
      "bundleIdentifier": "com.ejemplo.busquedarn",
      "supportsTablet": false,
      "config": {
        "usesNonExemptEncryption": false
      },
      "usesAppleSignIn": true
    },
    "android": {
      "package": "com.ejemplo.busquedarn",
```

```
    "adaptiveIcon": {
      "backgroundColor": "#E6F4FE",
      "foregroundImage": "./assets/images/android-
        icon-foreground.png",
      "backgroundImage": "./assets/images/android-
        icon-background.png",
      "monochromeImage": "./assets/images/android-
        icon-monochrome.png"
    },
    "predictiveBackGestureEnabled": false
  },
  "web": {
    "output": "single",
    "favicon": "./assets/images/favicon.png"
  },
  "plugins": [
    "expo-router",
    [
      "expo-splash-screen",
      {
        "backgroundColor": "#2563eb",
        "image": "./assets/images/splash-icon.png",
        "imageWidth": 76
      }
    ],
    "expo-secure-store",
    [
      "expo-image-picker",
      {
        "photosPermission": false,
        "cameraPermission": "Usamos la cámara para que
          saques tu foto de perfil y para leer códigos
          QR de ofertas.",
        "microphonePermission": false
      }
    ],
    "expo-sqlite",
    [
      "expo-camera",
      {
```

```
          "cameraPermission": "Usamos la cámara para que
            saques tu foto de perfil y para leer códigos
            QR de ofertas.",
          "microphonePermission": false,
          "recordAudioAndroid": false
        }
      ],
      [
        "expo-location",
        {
          "locationWhenInUsePermission": "Usamos tu
            ubicación para mostrarte ofertas de tu
            ciudad.",
          "locationAlwaysAndWhenInUsePermission": false,
          "locationAlwaysPermission": false,
          "motionUsagePermission": false,
          "locationFullAccuracyPermission": false
        }
      ],
      [
        "expo-local-authentication",
        {
          "faceIDPermission": "Usamos Face ID para
            proteger tus postulaciones."
        }
      ],
      [
        "expo-notifications",
        {
          "color": "#2563eb"
        }
      ],
      "expo-apple-authentication"
    ],
    "experiments": {
      "typedRoutes": true,
      "reactCompiler": true
    }
  }
}
```

Lo más importante:

Clave	Para qué
`name`	El nombre que se ve debajo del ícono.
`slug`	El nombre del proyecto en tu cuenta de Expo.
`version`	La versión visible para los usuarios (`1.0.0`).
`scheme`	Para los deep links: `busquedarn://oferta/5` abre la app.
`ios.bundleIdentifier` / `android.package`	El identificador único en las tiendas. **No se puede cambiar** después de publicar.
`plugins`	Config plugins: cada librería nativa agrega lo suyo (permisos, textos, claves).
`experiments.typedRoutes`	Tipos para las rutas: `router.push('/ofertaa')` no compila.
`experiments.reactCompiler`	El React Compiler: memoiza solo, sin `useMemo` ni `memo`.

Cuando necesitás lógica (variables de entorno, distintas variantes), usás `app.config.ts`. Recibe lo que haya en `app.json` y devuelve la configuración final:

app-expo/src/ejemplos/app.config.ejemplo.ts

```ts
import type { ConfigContext, ExpoConfig } from
  'expo/config'

// app.config.ts reemplaza (o extiende) a app.json
  cuando necesitás lógica:
// por ejemplo, instalar "dev", "preview" y "producción"
  a la vez en el mismo celular.
// Este archivo es un ejemplo; en un proyecto real va en
  la raíz y se llama app.config.ts.
const VARIANTE = process.env.APP_VARIANT ?? 'production'
  // lo define cada perfil de eas.json

const sufijo = VARIANTE === 'production' ? '' :
  `.${VARIANTE}`
const nombre = VARIANTE === 'production' ? 'Búsqueda RN'
  : `Búsqueda RN (${VARIANTE})`

export default ({ config }: ConfigContext): ExpoConfig
  => ({
  ...config, // lo que venga de app.json
  name: nombre,
  slug: 'guia-react-native',
  ios: { ...config.ios, bundleIdentifier:
    `com.ejemplo.busquedarn${sufijo}` },
  android: { ...config.android, package:
    `com.ejemplo.busquedarn${sufijo}` },
  extra: {
    ...config.extra,
    variante: VARIANTE,
  },
})
```

Los comandos del día a día

```
npx expo start            # servidor de desarrollo
  (Metro) + QR para Expo Go
npx expo start --clear    # lo mismo, borrando la caché
  de Metro
npx expo install expo-camera   # instala la versión
  compatible con tu SDK
npx expo install --fix    # corrige versiones
  incompatibles
npx expo-doctor           # revisa el proyecto:
  versiones, configuración, problemas comunes
npx expo prebuild         # genera ios/ y android/ desde
  app.json (casi nunca hace falta)
npx expo run:ios          # compila e instala en el
  simulador (necesita Xcode)
npx expo run:android      # compila e instala en el
  emulador (necesita Android Studio)
npx expo export --platform web   # build estático para
  la web
```

Con el servidor andando, en la terminal: `r` recarga, `m` abre el menú de desarrollo en el teléfono, `j` abre las DevTools, `i` / `a` / `w` abren iOS, Android o web. En el teléfono, sacudilo para abrir el menú.

> **⚠️ npx expo install, no npm install**
>
> Cada SDK de Expo funciona con versiones puntuales de las librerías nativas (react-native-reanimated, react-native-screens, etc.). `npx expo install` elige la compatible con tu SDK; `npm install` instala la última, que puede no funcionar. Para librerías de JavaScript puro (zod, date-fns, zustand) da igual, pero acostumbrate a usar siempre `npx expo install`.

Código distinto por plataforma

Tres formas, de la más chica a la más grande:

```
import { Platform } from 'react-native'

// 1. Una condición
const esIos = Platform.OS === 'ios' // 'ios' | 'android'
  | 'web'

// 2. Un valor por plataforma
const sombra = Platform.select({
  ios: { shadowOpacity: 0.2, shadowRadius: 8 },
  android: { elevation: 4 },
  default: {},
})
```

3. **Un archivo por plataforma**: si existen `Mapa.ios.tsx` y `Mapa.android.tsx`, `import Mapa from './Mapa'` elige el correcto al compilar. La app de la guía lo usa

para las pestañas: `(tabs)/_layout.tsx` usa las pestañas nativas, y `(tabs)/_layout.web.tsx` las reemplaza solo en la web:

app-expo/src/app/(tabs)/_layout.web.tsx

```
import { Tabs } from 'expo-router/js-tabs'
import { colores } from '@/demos/tema'

// En la web no hay barra de pestañas nativa: este
  archivo .web.tsx reemplaza a _layout.tsx
// solo ahí, con las pestañas hechas en JavaScript (las
  de React Navigation).
export default function TabsLayoutWeb() {
  return (
    <Tabs screenOptions={{ headerShown: false,
      tabBarActiveTintColor: colores.primario }}>
      <Tabs.Screen name="index" options={{ title:
        'Ofertas' }} />
      <Tabs.Screen name="guardadas" options={{ title:
        'Guardadas' }} />
      <Tabs.Screen name="demos" options={{ title:
        'Demos' }} />
      <Tabs.Screen name="perfil" options={{ title:
        'Perfil' }} />
    </Tabs>
  )
}
```

Variables de entorno

Expo carga solo los archivos `.env`. Las variables que empiezan con `EXPO_PUBLIC_` quedan disponibles en tu código:

```
# .env
EXPO_PUBLIC_API_URL=https://api.ejemplo.com
```

```
const url = process.env.EXPO_PUBLIC_API_URL
```

> ⚠️ **Todo lo que va en la app es público**
>
> Esas variables se copian dentro del bundle de JavaScript, y cualquiera puede descomprimir una app y leerlo. En una app **no hay secretos**: ni claves de Stripe secretas, ni la contraseña de la base de datos, ni la API key de OpenAI. Eso va en tu backend, y la app le pide a tu backend. Para las builds en la nube existen las variables de entorno de EAS.

Actualizar de SDK

Cada SDK nuevo (3 por año) trae una versión nueva de React Native. Para actualizar:

```
npx expo install expo@^59 --fix   # sube expo y ajusta
  todas las librerías
npx expo-doctor
```

Antes, leé el *changelog* del SDK en el blog de Expo: ahí están los cambios que rompen cosas. Conviene no

quedarse más de un SDK atrás, porque las tiendas suben sus requisitos todos los años (lo vemos en requisitos) y un SDK viejo no los cumple.

Capítulo 4

Los componentes básicos

View, Text, Image, ScrollView, Pressable, TextInput, Modal y compañía, con su equivalente web.

En React Native no hay HTML. Hay componentes que React Native traduce a vistas nativas. La buena noticia: son pocos, y casi todos tienen un equivalente web obvio.

En la web	En React Native	De dónde
`<div>`	`View`	react-native
`<p>`, `<span>`, `<h1>`	`Text`	react-native
`<img>`	`Image`	**expo-image** (mejor que el de react-native)
`<button>`	`Pressable`	react-native
`<input>`, `<textarea>`	`TextInput`	react-native
un `<div>` con `overflow: auto`	`ScrollView`	react-native

En la web	En React Native	De dónde
`<ul>` con muchos items	`FlatList`, `SectionList`, `FlashList`	react-native, @shopify/flash-list
`<dialog>`	`Modal` (o una ruta modal)	react-native, expo-router
`<a href>`	`Link`	expo-router
`<input type="checkbox">`	`Switch`	react-native
un spinner	`ActivityIndicator`	react-native

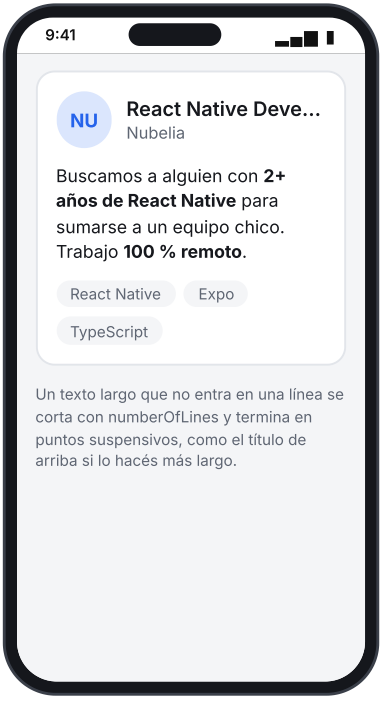
Las reglas que cambian

1. **El texto solo puede ir dentro de `<Text>`.** Un string suelto dentro de un `View` tira el error *"Text strings must be rendered within a `<Text>` component"*. Ojo con `{cantidad && <Text>…</Text>}`: si `cantidad` es `0`, React intenta renderizar el `0` suelto y la app explota. Usá `{cantidad > 0 && …}`.
2. **No hay CSS ni `className`**: los estilos son objetos de JavaScript en la prop `style` (lo vemos en estilos).
3. **No hay `onClick`**: se usa `onPress`. Y no hay hover (salvo en la web).
4. **Los estilos no se heredan**, salvo entre `Text` anidados. Un `color` en un `View` no le cambia el color al texto de adentro.
5. **Todo es flexbox, en columna**: `display: flex` y `flexDirection: 'column'` por defecto.

View y Text

`View` es la caja: agrupa, da layout y fondo. `Text` muestra texto, y es el único que lo hace. Mirá la demo de la bienvenida de nuevo, ahora con el código en mente:

DEMO View, Text y ScrollView



app-expo/src/demos/componentes/PrimeraPantalla.tsx

```
import { ScrollView, StyleSheet, Text, View } from
  'react-native'
import { ofertas } from '../datos'
import { colores, espacio } from '../tema'

// View es la caja (como un div), Text es el único lugar
  donde puede ir texto,
// y ScrollView hace que el contenido se pueda
  desplazar.
export default function PrimeraPantalla() {
  const oferta = ofertas[0]

  return (
    <ScrollView style={styles.pantalla}
      contentContainerStyle={styles.contenido}>
      <View style={styles.tarjeta}>
        <View style={styles.encabezado}>
          {/* Un avatar con iniciales: un View redondo
            con un Text centrado */}
          <View style={styles.avatar}>
            <Text style={styles.iniciales}>
              {oferta.empresa.slice(0, 2).toUpperCase()}
              </Text>
          </View>
          <View style={styles.flex}>
            <Text style={styles.titulo} numberOfLines=
              {1}>
              {oferta.titulo}
            </Text>
            <Text style={styles.suave}>{oferta.empresa}
              </Text>
          </View>
        </View>

        {/* Los Text se pueden anidar: el de adentro
          hereda y suma estilos */}
```

```
        <Text style={styles.parrafo}>
          Buscamos a alguien con <Text style=
            {styles.negrita}>2+ años de React
            Native</Text> para
          sumarse a un equipo chico. Trabajo <Text
            style={styles.negrita}>100 % remoto</Text>.
        </Text>

        <View style={styles.chips}>
          {oferta.stack.map((tecnologia) => (
            <View key={tecnologia} style={styles.chip}>
              <Text style={styles.chipTexto}>
                {tecnologia}</Text>
            </View>
          ))}
        </View>
      </View>

      <Text style={styles.nota}>
        Un texto largo que no entra en una línea se
          corta con numberOfLines y termina en puntos
        suspensivos, como el título de arriba si lo
          hacés más largo.
      </Text>
    </ScrollView>
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.fondo },
  contenido: { padding: espacio.l, gap: espacio.l },
  tarjeta: {
    backgroundColor: colores.tarjeta,
    borderRadius: 16,
    padding: espacio.l,
    gap: espacio.m,
    borderWidth: StyleSheet.hairlineWidth,
    borderColor: colores.borde,
  },
  encabezado: { flexDirection: 'row', alignItems:
    'center', gap: espacio.m },
```

```
  avatar: {
    width: 48,
    height: 48,
    borderRadius: 24,
    backgroundColor: colores.primarioSuave,
    alignItems: 'center',
    justifyContent: 'center',
  },
  iniciales: { color: colores.primario, fontWeight:
    '700', fontSize: 16 },
  flex: { flex: 1 },
  titulo: { fontSize: 17, fontWeight: '600', color:
    colores.texto },
  suave: { color: colores.textoSuave, marginTop: 2 },
  parrafo: { fontSize: 15, lineHeight: 22, color:
    colores.texto },
  negrita: { fontWeight: '700' },
  chips: { flexDirection: 'row', flexWrap: 'wrap', gap:
    espacio.s },
  chip: {
    backgroundColor: colores.fondo,
    borderRadius: 999,
    paddingVertical: espacio.xs,
    paddingHorizontal: espacio.m,
  },
  chipTexto: { fontSize: 13, color: colores.textoSuave
    },
  nota: { color: colores.textoSuave, fontSize: 13,
    lineHeight: 19 },
})
```

Lo que conviene saber de `Text`:

- **`numberOfLines`** corta el texto y agrega "...". Es la forma de evitar que un título largo rompa la tarjeta.
- **Anidar `Text`** suma estilos: el de adentro hereda el tamaño y color del de afuera. Así se hace una palabra en negrita en medio de un párrafo.

- `selectable` deja copiar el texto (por defecto no se puede).
- `onPress` funciona en `Text`: útil para un link dentro de un párrafo.
- El tamaño respeta la **accesibilidad del sistema**: si el usuario agrandó la letra, tu `fontSize: 15` se ve más grande. Diseñá para eso (o limitá con `maxFontSizeMultiplier`).

ScrollView

En el celular, nada scrollea solo: si el contenido no entra, se corta. `ScrollView` lo hace desplazable.

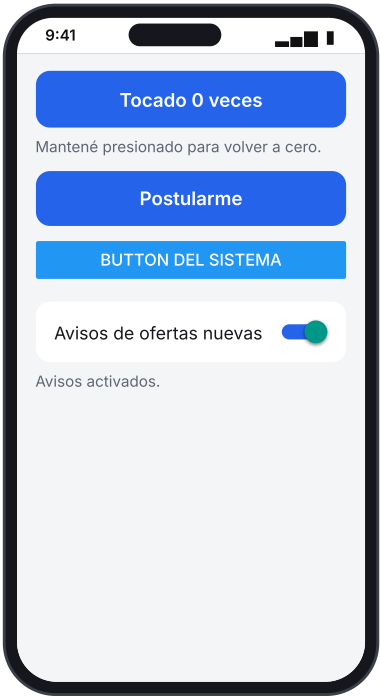
```
<ScrollView
  style={{ flex: 1 }}                        // el tamaño
    del ScrollView (la "ventana")
  contentContainerStyle={{ padding: 16, gap: 12 }}  //
    el estilo del contenido de adentro
  keyboardShouldPersistTaps="handled"        // tocar un
    botón con el teclado abierto funciona al primer
    toque
  refreshControl={<RefreshControl refreshing={cargando}
    onRefresh={recargar} />}
>
  {/* ... */}
</ScrollView>
```

`ScrollView` renderiza **todos** sus hijos de una vez. Para 10 tarjetas está perfecto; para 500 filas, usá una lista, que renderiza solo lo que se ve. La diferencia entre `style` y `contentContainerStyle` confunde siempre: el padding y el `gap` van en `contentContainerStyle`.

Pressable: todo lo que se toca

`Pressable` es la base de botones, tarjetas tocables y filas de una lista. Recibe una **función de estilo** que sabe si está presionado, para dar feedback visual:

DEMO Pressable, Button y Switch



app-expo/src/demos/componentes/Botones.tsx

```
import { useState } from 'react'
```

```
import { ActivityIndicator, Button, Pressable,
  StyleSheet, Switch, Text, View } from 'react-native'
import { colores, espacio } from '../tema'

// Pressable es la base de cualquier cosa que se toca:
  recibe una función de estilo
// que sabe si está presionado, y eventos como onPress y
  onLongPress.
export default function Botones() {
  const [toques, setToques] = useState(0)
  const [enviando, setEnviando] = useState(false)
  const [avisos, setAvisos] = useState(true)

  function enviar() {
    setEnviando(true)
    setTimeout(() => setEnviando(false), 1500)
  }

  return (
    <View style={styles.pantalla}>
      <Pressable
        onPress={() => setToques((n) => n + 1)}
        onLongPress={() => setToques(0)}
        hitSlop={8}
        style={({ pressed }) => [styles.boton, pressed
          && styles.botonPresionado]}
      >
        <Text style={styles.botonTexto}>Tocado {toques}
          veces</Text>
      </Pressable>
      <Text style={styles.ayuda}>Mantené presionado para
        volver a cero.</Text>

      <Pressable
        onPress={enviar}
        disabled={enviando}
        style={({ pressed }) => [styles.boton, (pressed
          || enviando) && styles.botonPresionado]}
      >
        {enviando ? <ActivityIndicator color="#fff" /> :
          <Text style=
```

```
          {styles.botonTexto}>Postularme</Text>}
      </Pressable>

      {/* Button es el botón del sistema: se ve distinto
        en iOS y Android, y casi no se puede estilizar
        */}
      <Button title="Button del sistema" onPress={() =>
        setToques((n) => n + 1)} />

      <View style={styles.fila}>
        <Text style={styles.texto}>Avisos de ofertas
          nuevas</Text>
        <Switch
          value={avisos}
          onValueChange={setAvisos}
          trackColor={{ true: colores.primario, false:
            colores.borde }}
        />
      </View>
      <Text style={styles.ayuda}>Avisos {avisos ?
        'activados' : 'desactivados'}.</Text>
    </View>
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.fondo,
    padding: espacio.l, gap: espacio.m },
  boton: {
    backgroundColor: colores.primario,
    paddingVertical: 14,
    borderRadius: 12,
    alignItems: 'center',
    minHeight: 48,
    justifyContent: 'center',
  },
  botonPresionado: { opacity: 0.7, transform: [{ scale:
    0.98 }] },
  botonTexto: { color: '#fff', fontWeight: '600',
    fontSize: 16 },
  ayuda: { color: colores.textoSuave, fontSize: 13,
    marginTop: -4 },
```

```
  fila: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    backgroundColor: colores.tarjeta,
    padding: espacio.l,
    borderRadius: 12,
    marginTop: espacio.s,
  },
  texto: { fontSize: 15, color: colores.texto },
})
```

- `onPress`, `onLongPress`, `onPressIn` / `onPressOut`.
- `hitSlop` agranda el área tocable sin cambiar el tamaño visible. Apple recomienda que todo lo tocable mida **al menos 44×44 puntos** (Google dice 48).
- `disabled` desactiva el toque (y avisa a los lectores de pantalla).
- `Button` es el botón del sistema: casi no se puede estilizar. En apps reales se arma uno propio con `Pressable`.

> ### ℹ️ TouchableOpacity y compañía
>
> Vas a ver `TouchableOpacity`, `TouchableHighlight` y `TouchableWithoutFeedback` en código viejo y en tutoriales. Siguen funcionando, pero `Pressable` los reemplaza a todos. En una entrevista, decí `Pressable`.

Image: mejor con expo-image

El `Image` de react-native funciona, pero **expo-image** es lo que se usa: cachea en disco, muestra un placeholder borroso mientras carga, hace transiciones y soporta más formatos (WebP, AVIF, SVG, GIF animado).

```
import { Image } from 'expo-image'

<Image
  source={{ uri: oferta.logoUrl }}
  placeholder={{ blurhash:
    'L6PZfSi_.AyE_3t7t7R**0o#DgR4' }}  // se ve mientras
    carga
  contentFit="cover"          // como object-fit en CSS
  transition={200}            // fundido al aparecer
  style={{ width: 48, height: 48, borderRadius: 24 }}
  accessibilityLabel={`Logo de ${oferta.empresa}`}
/>

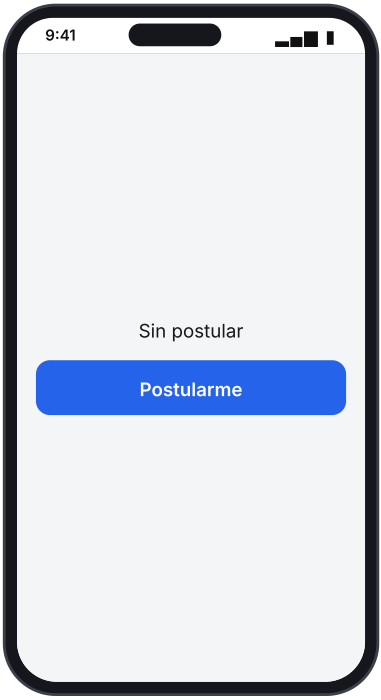
// Una imagen local: require() para que Metro la incluya
  en el bundle
<Image source={require('@/assets/images/icon.png')}
  style={{ width: 80, height: 80 }} />
```

Las imágenes remotas **necesitan un tamaño** (`width` y `height`, o `flex` / `aspectRatio`): el celular no sabe cuánto mide una imagen hasta descargarla, y no va a mover el layout cuando llegue. En la pestaña Perfil de la app se usa con la foto que elegís de la galería.

Modal

Un `Modal` se dibuja por encima de todo. La demo usa uno **transparente** con fondo oscuro, para un diálogo de confirmación:

DEMO Modal



`app-expo/src/demos/componentes/Confirmar.tsx`

```
import { useState } from 'react'
```

```
import { Modal, Pressable, StyleSheet, Text, View } from
  'react-native'
import { colores, espacio } from '../tema'

// Modal muestra contenido por encima de la pantalla.
  Con transparent se ve lo de atrás
// y el fondo oscuro lo pone uno. onRequestClose es
  obligatorio para el botón "atrás" de Android.
export default function Confirmar() {
  const [visible, setVisible] = useState(false)
  const [estado, setEstado] = useState('Sin postular')

  return (
    <View style={styles.pantalla}>
      <Text style={styles.estado}>{estado}</Text>
      <Pressable style={styles.boton} onPress={() =>
        setVisible(true)}>
        <Text style=
          {styles.botonTexto}>Postularme</Text>
      </Pressable>

      <Modal
        visible={visible}
        transparent
        animationType="fade"
        onRequestClose={() => setVisible(false)}
      >
        {/* Tocar el fondo cierra el modal */}
        <Pressable style={styles.fondo} onPress={() =>
          setVisible(false)}>
          {/* Este Pressable vacío evita que un toque en
            la tarjeta llegue al fondo */}
          <Pressable style={styles.tarjeta} onPress={()
            => {}}>
            <Text style={styles.titulo}>¿Te postulás a
              Nubelia?</Text>
            <Text style={styles.texto}>Le vamos a mandar
              tu perfil y tu CV.</Text>
            <View style={styles.acciones}>
              <Pressable style={[styles.accion,
                styles.secundaria]} onPress={() =>
                setVisible(false)}>
```

```
                <Text style=
                  {styles.secundariaTexto}>Cancelar</Tex
                  t>
              </Pressable>
              <Pressable
                style={styles.accion}
                onPress={() => {
                  setEstado('Postulación enviada ✓')
                  setVisible(false)
                }}
              >
                <Text style=
                  {styles.botonTexto}>Enviar</Text>
              </Pressable>
            </View>
          </Pressable>
        </Pressable>
      </Modal>
    </View>
  )
}

const styles = StyleSheet.create({
  pantalla: {
    flex: 1,
    backgroundColor: colores.fondo,
    padding: espacio.l,
    justifyContent: 'center',
    gap: espacio.l,
  },
  estado: { textAlign: 'center', fontSize: 16, color:
    colores.texto },
  boton: { backgroundColor: colores.primario, padding:
    14, borderRadius: 12, alignItems: 'center' },
  botonTexto: { color: '#fff', fontWeight: '600',
    fontSize: 16 },
  fondo: {
    flex: 1,
    backgroundColor: 'rgba(0,0,0,0.45)',
    justifyContent: 'center',
    padding: espacio.xl,
```

```
  },
  tarjeta: { backgroundColor: colores.tarjeta,
    borderRadius: 16, padding: espacio.xl, gap:
    espacio.s },
  titulo: { fontSize: 18, fontWeight: '700', color:
    colores.texto },
  texto: { fontSize: 15, color: colores.textoSuave },
  acciones: { flexDirection: 'row', gap: espacio.s,
    marginTop: espacio.m },
  accion: {
    flex: 1,
    backgroundColor: colores.primario,
    padding: 12,
    borderRadius: 10,
    alignItems: 'center',
  },
  secundaria: { backgroundColor: colores.fondo },
  secundariaTexto: { color: colores.texto, fontWeight:
    '600', fontSize: 16 },
})
```

- `visible` lo muestra u oculta (lo controlás con estado).
- `transparent` deja ver lo de atrás; sin eso, el modal ocupa toda la pantalla con fondo blanco.
- `animationType`: `'none'`, `'fade'` o `'slide'`.
- `onRequestClose` es obligatorio en la práctica: se llama cuando el usuario aprieta "atrás" en Android o desliza para cerrar en iOS.

Para pantallas completas que se abren desde abajo (un formulario, un filtro), en apps con Expo Router se usa una **ruta modal** en lugar del componente `Modal`, porque así tiene URL, botón de cerrar nativo y gesto de deslizar. Lo vemos en navegación.

Safe area: la isla, el notch y la barra de gestos

Los teléfonos tienen partes de la pantalla tapadas: la Dynamic Island o el notch arriba, la barra de gestos abajo. Y desde Android 15 todas las apps se dibujan **de borde a borde** (detrás de la barra de estado y de navegación); en las versiones actuales de Expo es obligatorio. Tu contenido tiene que correrse para no quedar tapado:

```
import { SafeAreaView, useSafeAreaInsets } from 'react-
  native-safe-area-context'

// Opción 1: un contenedor que agrega el padding justo
<SafeAreaView edges={['top']} style={{ flex: 1 }}>
  {/* ... */}
</SafeAreaView>

// Opción 2: los valores, para usarlos donde quieras
const insets = useSafeAreaInsets()
<View style={{ paddingBottom: insets.bottom + 16 }}>…
  </View>
```

Usá el de **react-native-safe-area-context**, no el `SafeAreaView` de react-native (que solo funcionaba en iOS y está deprecado). Si la pantalla tiene un header de Expo Router, el header ya se ocupa de la parte de arriba. La pantalla de ofertas de la app usa `edges={['top']}` porque no tiene header.

StatusBar

La barra de arriba (hora, batería) tiene íconos claros u oscuros. Con `expo-status-bar` lo elegís según tu fondo:

```
import { StatusBar } from 'expo-status-bar'

<StatusBar style="dark" />   // íconos oscuros, para fondo claro. "light" para fondo oscuro, "auto" sigue al tema
```

Componentes que vas a ver en código viejo

Componente	Qué pasó
`TouchableOpacity`, `TouchableHighlight`	Reemplazados por `Pressable`.
`SafeAreaView` de react-native	Deprecado: usá el de react-native-safe-area-context.
`ImageBackground`	Deprecado en el SDK 58: una `Image` con `StyleSheet.absoluteFill` detrás del contenido.
`AsyncStorage` de react-native	Se sacó hace años: ahora es `@react-native-async-storage/async-storage`.
`Picker`, `DatePickerIOS`, `ProgressViewIOS`	Se sacaron del core: hay librerías de la comunidad, o `@expo/ui`.

💡 Componentes 100 % nativos: Expo UI

`@expo/ui` (estable desde el SDK 56) te da componentes de **SwiftUI** en iOS y de **Jetpack Compose** en Android: menús contextuales, pickers, sliders, date pickers, con el look exacto del sistema. Útil cuando querés que un control se vea idéntico al de las apps de Apple o Google.

Capítulo 5

Estilos y layout

StyleSheet, flexbox con otros valores por defecto, tamaños, plataformas, modo oscuro y librerías de estilos.

En React Native no hay CSS: los estilos son **objetos de JavaScript** con nombres en camelCase. Se parecen mucho a CSS, pero no hay cascada, ni selectores, ni media queries, ni unidades.

StyleSheet y la prop style

```
import { StyleSheet, Text, View } from 'react-native'

function Chip({ texto, activo }: { texto: string;
  activo: boolean }) {
  return (
    // Un array combina estilos: el último gana. false,
      null y undefined se ignoran.
    <View style={[styles.chip, activo &&
      styles.activo]}>
      <Text style={[styles.texto, activo && { color:
        '#fff' }]}>{texto}</Text>
    </View>
  )
}
```

```
const styles = StyleSheet.create({
  chip: { paddingVertical: 4, paddingHorizontal: 12,
    borderRadius: 999, backgroundColor: '#eef2ff' },
  activo: { backgroundColor: '#2563eb' },
  texto: { fontSize: 13, color: '#2563eb', fontWeight:
    '600' },
})
```

- **Sin unidades**: `padding: 16` son 16 *puntos* (píxeles independientes de la densidad). En un iPhone con pantalla 3x son 48 píxeles físicos. También se aceptan porcentajes como string: `width: '50%'`.
- `StyleSheet.create` no es obligatorio (un objeto suelto funciona igual), pero valida los nombres en desarrollo, da autocompletado con TypeScript y deja los estilos fuera del render.
- **No hay herencia**: solo un `Text` adentro de otro `Text` hereda la fuente y el color.
- **No hay shorthand de varios valores**: `padding: '8px 16px'` no existe. Se usa `paddingVertical: 8, paddingHorizontal: 16`. Lo mismo con `margin` y `border`.
- `StyleSheet.hairlineWidth` es la línea más fina que la pantalla puede dibujar: ideal para separadores.

Flexbox, con otros valores por defecto

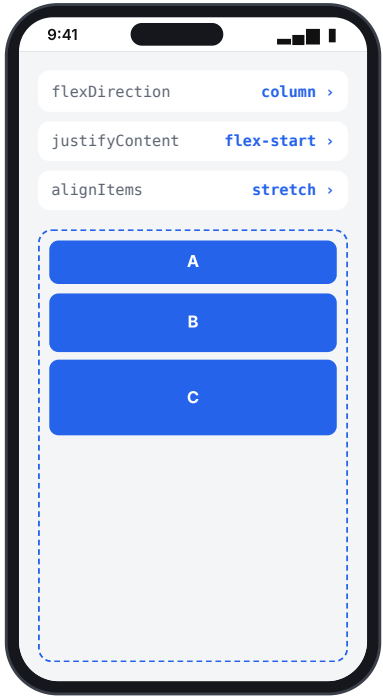
Todo `View` es un contenedor flex. Funciona como en CSS, con tres diferencias que te van a confundir la

primera semana:

	En CSS	En React Native
`flexDirection`	`row`	**`column`**
`alignContent`	`stretch`	`flex-start`
`flexShrink`	`1`	`0` (los hijos no se achican solos)
`flex: 1`	`1 1 0%`	"ocupá todo el espacio que sobra"

Tocá los botones para ver cómo cambia la distribución:

DEMO Flexbox



app-expo/src/demos/estilos/Flexbox.tsx

```
import { useState } from 'react'
import { Pressable, StyleSheet, Text, View, type
  ViewStyle } from 'react-native'
import { colores, espacio } from '../tema'

// En React Native todo View es flex, y la dirección por
  defecto es columna (en la web es fila).
const opciones = {
  flexDirection: ['column', 'row'],
  justifyContent: ['flex-start', 'center', 'flex-end',
    'space-between', 'space-around'],
```

```
  alignItems: ['stretch', 'flex-start', 'center', 'flex-
    end'],
} as const

type Propiedad = keyof typeof opciones

export default function Flexbox() {
  const [estilo, setEstilo] = useState<ViewStyle>({
    flexDirection: 'column',
    justifyContent: 'flex-start',
    alignItems: 'stretch',
  })

  // Cada toque pasa al siguiente valor de esa propiedad
  function siguiente(propiedad: Propiedad) {
    const valores: readonly string[] =
      opciones[propiedad]
    const actual =
      valores.indexOf(String(estilo[propiedad]))
    setEstilo({ ...estilo, [propiedad]: valores[(actual
      + 1) % valores.length] })
  }

  return (
    <View style={styles.pantalla}>
      {(Object.keys(opciones) as
        Propiedad[]).map((propiedad) => (
        <Pressable key={propiedad} style=
          {styles.control} onPress={() =>
          siguiente(propiedad)}>
          <Text style={styles.propiedad}>{propiedad}
            </Text>
          <Text style={styles.valor}>
            {String(estilo[propiedad])} ›</Text>
        </Pressable>
      ))}

      <View style={[styles.contenedor, estilo]}>
        {['A', 'B', 'C'].map((letra, i) => (
          <View key={letra} style={[styles.caja, {
            minHeight: 36 + i * 14, minWidth: 36 + i *
            14 }]}>
```

```
            <Text style={styles.letra}>{letra}</Text>
          </View>
        ))}
      </View>
    </View>
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.fondo,
    padding: espacio.l, gap: espacio.s },
  control: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    backgroundColor: colores.tarjeta,
    paddingVertical: 10,
    paddingHorizontal: espacio.m,
    borderRadius: 10,
  },
  propiedad: { fontFamily: 'monospace', fontSize: 13,
    color: colores.textoSuave },
  valor: { fontFamily: 'monospace', fontSize: 13, color:
    colores.primario, fontWeight: '600' },
  contenedor: {
    flex: 1,
    marginTop: espacio.s,
    borderWidth: 2,
    borderStyle: 'dashed',
    borderColor: colores.primario,
    borderRadius: 12,
    padding: espacio.s,
    gap: espacio.s,
  },
  caja: {
    backgroundColor: colores.primario,
    borderRadius: 8,
    alignItems: 'center',
    justifyContent: 'center',
  },
  letra: { color: '#fff', fontWeight: '700' },
```

```
})
```

El patrón más común: una pantalla con `flex: 1` (ocupa todo el alto), una fila con `flexDirection: 'row'`, y un hijo con `flex: 1` que empuja al resto contra el borde. `gap`, `rowGap` y `columnGap` funcionan y te ahorran márgenes.

> ⚠️ **El texto que se sale de la pantalla**
>
> Como `flexShrink` es `0`, un `Text` largo dentro de una fila se sale por la derecha en lugar de cortarse. La solución: `flex: 1` (o `flexShrink: 1`) en el contenedor del texto, y `numberOfLines` si querés cortarlo. Es el bug de layout más común en React Native.

Posición

`position: 'absolute'` posiciona respecto del **padre** (todos los elementos son `relative` por defecto). No existe `position: fixed`: para algo que queda fijo, ponelo afuera del `ScrollView`.
`StyleSheet.absoluteFill` es un atajo para `position: 'absolute'` con `top`, `left`, `right` y `bottom` en `0`.

Tamaños de pantalla

```
import { useWindowDimensions } from 'react-native'
```

```
function Grilla() {
  const { width } = useWindowDimensions() // se
    actualiza al rotar o en una tablet con ventanas
  const columnas = width > 700 ? 3 : 1
  // ...
}
```

Usá el hook `useWindowDimensions`, no `Dimensions.get('window')`: el hook se vuelve a renderizar cuando cambia el tamaño. Para algo proporcional, `aspectRatio: 16 / 9` con un ancho fijo calcula el alto.

Sombras y efectos

Desde la Nueva Arquitectura, `boxShadow` funciona igual que en CSS, en iOS y Android:

```
const tarjeta = {
  boxShadow: '0 2px 8px rgba(0, 0, 0, 0.12)',   // igual
    que en CSS
  // en código viejo:
    shadowColor/shadowOffset/shadowOpacity/shadowRadius
    (iOS) + elevation (Android)
}
```

También funcionan `filter` (`'blur(4px)'`, `'grayscale(1)'`), `transform`, `opacity`, `mixBlendMode` y `outline`. Para degradados se usa `expo-linear-gradient`, y para el efecto vidrio de iOS 26, `expo-glass-effect`.

Fuentes

Por defecto se usa la fuente del sistema (San Francisco en iOS, Roboto en Android), que es lo que recomiendan Apple y Google. Para una fuente propia, `expo-font` la mete en la app al compilar:

```
// app.json
"plugins": [
  ["expo-font", { "fonts": ["./assets/fonts/Inter-
    Regular.ttf", "./assets/fonts/Inter-Bold.ttf"] }]
]
```

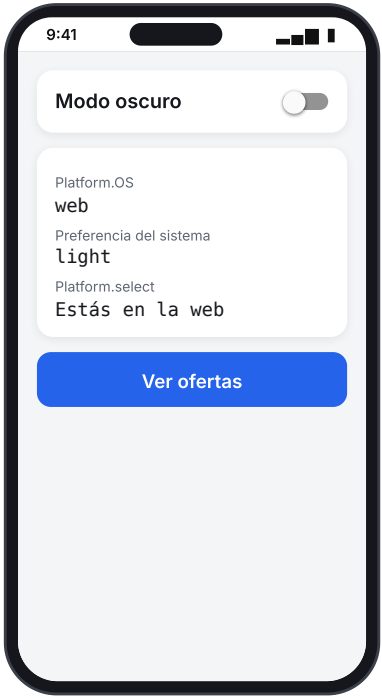
```
<Text style={{ fontFamily: 'Inter-Bold' }}>Hola</Text>
```

En Android, `fontWeight: 'bold'` no elige el archivo "Bold" de una fuente propia: cada grosor es una `fontFamily` distinta. Para las fuentes de Google existe `@expo-google-fonts/inter` (y una por cada fuente).

Modo oscuro

`useColorScheme()` devuelve la preferencia del sistema: `'light'`, `'dark'` o `null`. Con eso elegís un tema:

DEMO Tema y Platform



app-expo/src/demos/estilos/Tema.tsx

```
import { useState } from 'react'
import { Platform, Pressable, StyleSheet, Switch, Text,
  View, useColorScheme } from 'react-native'

// Un tema con una versión clara y una oscura. En una
  app real el modo sale de
// useColorScheme() (la preferencia del sistema) y se
  puede pisar desde la configuración.
const temas = {
  claro: { fondo: '#f4f5f7', tarjeta: '#ffffff', texto:
    '#16181d', suave: '#5f6673', primario: '#2563eb' },
```

```
  oscuro: { fondo: '#0d0f12', tarjeta: '#1b1e24', texto:
    '#f1f3f6', suave: '#9aa2b1', primario: '#6b9bff' },
}

export default function Tema() {
  const sistema = useColorScheme() // 'light' | 'dark' |
    null
  const [oscuro, setOscuro] = useState(sistema ===
    'dark')
  const t = oscuro ? temas.oscuro : temas.claro

  return (
    <View style={[styles.pantalla, { backgroundColor:
      t.fondo }]}>
      <View style={[styles.tarjeta, { backgroundColor:
        t.tarjeta }]}>
        <Text style={[styles.titulo, { color: t.texto
          }]}>Modo oscuro</Text>
        <Switch value={oscuro} onValueChange={setOscuro}
          />
      </View>

      <View style={[styles.tarjeta, styles.columna, {
        backgroundColor: t.tarjeta }]}>
        <Text style={[styles.etiqueta, { color: t.suave
          }]}>Platform.OS</Text>
        <Text style={[styles.valor, { color: t.texto
          }]}>{Platform.OS}</Text>
        <Text style={[styles.etiqueta, { color: t.suave
          }]}>Preferencia del sistema</Text>
        <Text style={[styles.valor, { color: t.texto
          }]}>{sistema ?? 'sin preferencia'}</Text>
        <Text style={[styles.etiqueta, { color: t.suave
          }]}>Platform.select</Text>
        <Text style={[styles.valor, { color: t.texto
          }]}>
          {Platform.select({ ios: 'Estás en un iPhone',
            android: 'Estás en Android', default: 'Estás
            en la web' })}
        </Text>
      </View>
```

```
      <Pressable style={[styles.boton, {
        backgroundColor: t.primario }]}>
        <Text style={styles.botonTexto}>Ver
          ofertas</Text>
      </Pressable>
    </View>
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, padding: 16, gap: 12 },
  tarjeta: {
    borderRadius: 14,
    padding: 16,
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    // Desde React Native 0.76, boxShadow funciona igual
      en iOS, Android y web
    boxShadow: '0 2px 8px rgba(0,0,0,0.08)',
  },
  columna: { flexDirection: 'column', alignItems: 'flex-
    start', gap: 2 },
  titulo: { fontSize: 17, fontWeight: '600' },
  etiqueta: { fontSize: 12, marginTop: 8 },
  valor: {
    fontSize: 16,
    fontWeight: '500',
    // Una fuente monoespaciada tiene otro nombre en
      cada plataforma
    fontFamily: Platform.select({ ios: 'Menlo', default:
      'monospace' }),
  },
  boton: { padding: 14, borderRadius: 12, alignItems:
    'center' },
  botonTexto: { color: '#fff', fontWeight: '600',
    fontSize: 16 },
})
```

Para que el hook responda, `app.json` tiene que decir `"userInterfaceStyle": "automatic"` (la app de la guía usa `"light"` para mantenerla simple). Con Expo Router, envolvé la app en `ThemeProvider` con `DarkTheme` o `DefaultTheme` (los exporta `expo-router`), así los headers y las pestañas también cambian.

La forma prolija de organizarlo: un objeto de colores por tema y un hook propio que devuelve el actual.

```
const colores = {
  light: { fondo: '#fff', texto: '#111', primario:
    '#2563eb' },
  dark: { fondo: '#000', texto: '#f5f5f5', primario:
    '#6b9bff' },
}

export function useColores() {
  return colores[useColorScheme() ?? 'light']
}
```

Librerías de estilos

`StyleSheet` alcanza para cualquier app, pero en los equipos vas a ver alguna de estas:

Librería	Qué es	Cuándo
NativeWind	Tailwind en React Native: `className="flex-1 p-4 bg-white"`.	Si el equipo viene de Tailwind en la web. La más popular.

Librería	Qué es	Cuándo
Uniwind	Tailwind 4, de los creadores de Unistyles. Más rápida que NativeWind.	Proyectos nuevos con Tailwind.
Unistyles 3	`StyleSheet.create` con temas, breakpoints y variantes, sin re-renders.	Si querés seguir escribiendo StyleSheet pero con temas.
Tamagui	Kit de componentes + compilador que optimiza estilos. Comparte código con la web.	Apps universales (web + nativo) con diseño propio.
React Native Paper	Componentes de Material Design listos.	Apps internas o con look de Android.
HeroUI Native, **gluestack-ui**	Kits de componentes accesibles, basados en Tailwind.	Arrancar rápido con componentes armados.

🎤 En una entrevista

"¿Cómo manejás estilos en React Native?" Una buena respuesta: StyleSheet con un tema centralizado (colores, espacios, tipografía) y un hook para el modo oscuro; y si el equipo usa Tailwind, NativeWind o Uniwind. Mencioná que no hay cascada, que flexbox va en columna por defecto, y que los tamaños son puntos, no píxeles.

Capítulo 6

Listas

FlatList, SectionList, FlashList y Legend List: renderizar miles de filas sin trabarse.

Una lista de ofertas, de mensajes, de contactos: casi toda app tiene una. Y es donde más se nota si una app de React Native está bien hecha.

Por qué no un map adentro de un ScrollView

```
// Funciona con 20 items. Con 2.000, la pantalla tarda
  segundos en aparecer.
<ScrollView>
  {ofertas.map((o) => <TarjetaOferta key={o.id} oferta=
    {o} />)}
</ScrollView>
```

`ScrollView` crea **todas** las vistas nativas de entrada, aunque no se vean. Las listas **virtualizan**: crean solo las filas que están en pantalla (y un poco más arriba y abajo), y las van creando y destruyendo mientras scrolleás.

FlatList

Es la lista que viene con React Native. Recibe los datos y una función que dibuja cada fila:

DEMO **FlatList con búsqueda y paginación**



`app-expo/src/demos/listas/ListaOfertas.tsx`

```
import { useState } from 'react'
import { FlatList, Pressable, StyleSheet, Text,
  TextInput, View } from 'react-native'
```

```tsx
import { nombreModalidad, ofertas, type Oferta } from
  '../datos'
import { colores, espacio } from '../tema'

const POR_PAGINA = 5

// FlatList solo renderiza lo que está en pantalla (y un
  poco más). Para listas largas
// siempre FlatList (o FlashList), nunca un .map()
  adentro de un ScrollView.
export default function ListaOfertas() {
  const [busqueda, setBusqueda] = useState('')
  const [cantidad, setCantidad] = useState(POR_PAGINA)

  const filtradas = ofertas.filter((o) =>
    `${o.titulo}
      ${o.empresa}`.toLowerCase().includes(busqueda.trim
      ().toLowerCase()),
  )
  const visibles = filtradas.slice(0, cantidad)

  return (
    <View style={styles.pantalla}>
      <TextInput
        value={busqueda}
        onChangeText={(texto) => {
          setBusqueda(texto)
          setCantidad(POR_PAGINA)
        }}
        placeholder="Buscar por puesto o empresa"
        placeholderTextColor={colores.textoSuave}
        style={styles.buscador}
        clearButtonMode="while-editing"
        autoCorrect={false}
      />
      <FlatList
        data={visibles}
        keyExtractor={(oferta) => String(oferta.id)}
        renderItem={({ item }) => <FilaOferta oferta=
          {item} />}
```

```
        ItemSeparatorComponent={() => <View style=
          {styles.separador} />}
        ListEmptyComponent={<Text style=
          {styles.vacio}>No hay ofertas para
          “{busqueda}”.</Text>}
        ListFooterComponent={
          visibles.length < filtradas.length ? (
            <Text style={styles.pie}>Cargando más…
              </Text>
          ) : undefined
        }
        // Se llama al acercarse al final: acá iría el
          pedido de la página siguiente
        onEndReached={() => setCantidad((c) => c +
          POR_PAGINA)}
        onEndReachedThreshold={0.5}
        contentContainerStyle={styles.lista}
        keyboardShouldPersistTaps="handled"
      />
    </View>
  )
}

function FilaOferta({ oferta }: { oferta: Oferta }) {
  return (
    <Pressable style={({ pressed }) => [styles.fila,
      pressed && { backgroundColor: colores.fondo }]}>
      <View style={styles.flex}>
        <Text style={styles.titulo}>{oferta.titulo}
          </Text>
        <Text style={styles.suave}>
          {oferta.empresa} ·
            {nombreModalidad[oferta.modalidad]} ·
            {oferta.pais}
        </Text>
      </View>
      <Text style={styles.salario}>US$
        {oferta.salarioMax.toLocaleString('es')}</Text>
    </Pressable>
  )
}
```

```
const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.tarjeta
    },
  buscador: {
    margin: espacio.m,
    backgroundColor: colores.fondo,
    borderRadius: 10,
    paddingHorizontal: espacio.m,
    paddingVertical: 10,
    fontSize: 15,
    color: colores.texto,
  },
  lista: { paddingBottom: espacio.xl },
  fila: {
    flexDirection: 'row',
    alignItems: 'center',
    paddingHorizontal: espacio.l,
    paddingVertical: espacio.m,
    gap: espacio.m,
  },
  flex: { flex: 1 },
  titulo: { fontSize: 15, fontWeight: '600', color:
    colores.texto },
  suave: { fontSize: 13, color: colores.textoSuave,
    marginTop: 2 },
  salario: { fontSize: 13, fontWeight: '600', color:
    colores.exito },
  separador: { height: StyleSheet.hairlineWidth,
    backgroundColor: colores.borde, marginLeft:
    espacio.l },
  vacio: { textAlign: 'center', color:
    colores.textoSuave, marginTop: espacio.xl },
  pie: { textAlign: 'center', color: colores.textoSuave,
    padding: espacio.m },
})
```

Las props que vas a usar siempre:

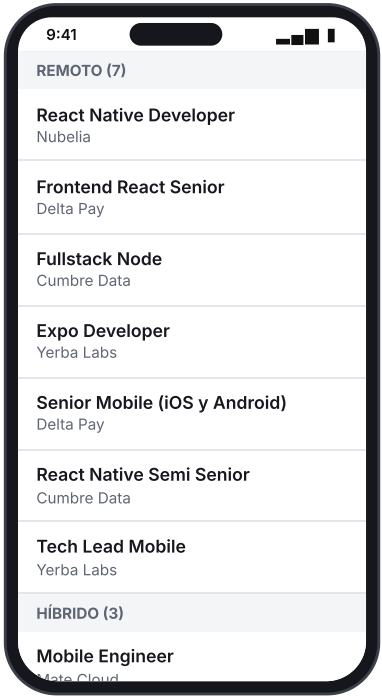
Prop	Para qué
`data`	El array.
`renderItem`	`({ item, index }) => …`: dibuja una fila.
`keyExtractor`	Una key estable por fila (el id, como string). No uses el índice.
`ItemSeparatorComponent`	Lo que va entre filas (una línea).
`ListEmptyComponent`	Lo que se ve si `data` está vacío: "No hay resultados".
`ListHeaderComponent` / `ListFooterComponent`	Arriba y abajo de todo. El footer es el lugar del spinner de "cargando más".
`onEndReached` + `onEndReachedThreshold`	Llegaste cerca del final: cargá la siguiente página. `0.5` = a media pantalla del final.
`refreshing` + `onRefresh`	El "tirar para recargar".
`numColumns`	Una grilla.
`horizontal`	Una fila que se desliza de costado (un carrusel).
`inverted`	Empieza desde abajo: para un chat.
`keyboardDismissMode="on-drag"`	Cierra el teclado al scrollear.

> ⚠️ **onEndReached se llama más de una vez**
>
> Mientras llega la siguiente página, `onEndReached` puede dispararse de nuevo. Siempre chequeá que no estés cargando ya (`if (!cargando && hayMas) cargarMas()`). Con TanStack Query es `hasNextPage && !isFetchingNextPage`, como en la pantalla de ofertas de la app.

SectionList

Para una lista agrupada (por modalidad, por letra, por fecha), con encabezados que quedan pegados arriba mientras scrolleás:

DEMO SectionList



app-expo/src/demos/listas/ListaPorModalidad.tsx

```
import { SectionList, StyleSheet, Text, View } from
  'react-native'
import { nombreModalidad, ofertas, type Modalidad } from
  '../datos'
import { colores, espacio } from '../tema'

// SectionList es una FlatList agrupada: recibe
  secciones, cada una con su título y sus datos.
const secciones = (['remoto', 'hibrido', 'presencial']
  as Modalidad[]).map((modalidad) => ({
```

```
  titulo: nombreModalidad[modalidad],
  data: ofertas.filter((o) => o.modalidad ===
    modalidad),
}))

export default function ListaPorModalidad() {
  return (
    <SectionList
      style={styles.pantalla}
      sections={secciones}
      keyExtractor={(oferta) => String(oferta.id)}
      renderSectionHeader={({ section }) => (
        <Text style={styles.encabezado}>
          {section.titulo} ({section.data.length})
        </Text>
      )}
      renderItem={({ item }) => (
        <View style={styles.fila}>
          <Text style={styles.titulo}>{item.titulo}
            </Text>
          <Text style={styles.suave}>{item.empresa}
            </Text>
        </View>
      )}
      // En iOS los encabezados quedan pegados arriba
        mientras scrolleás su sección
      stickySectionHeadersEnabled
    />
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.tarjeta
    },
  encabezado: {
    backgroundColor: colores.fondo,
    paddingHorizontal: espacio.l,
    paddingVertical: espacio.s,
    fontSize: 13,
    fontWeight: '700',
    color: colores.textoSuave,
```

```
    textTransform: 'uppercase',
  },
  fila: {
    paddingHorizontal: espacio.l,
    paddingVertical: espacio.m,
    borderBottomWidth: StyleSheet.hairlineWidth,
    borderBottomColor: colores.borde,
  },
  titulo: { fontSize: 15, fontWeight: '600', color:
    colores.texto },
  suave: { fontSize: 13, color: colores.textoSuave,
    marginTop: 2 },
})
```

Los datos van como `sections: [{ title, data: [...] }]`, y además de `renderItem` hay un `renderSectionHeader`. Los encabezados pegajosos (`stickySectionHeadersEnabled`) vienen activados en iOS y desactivados en Android.

FlashList: la que se usa en producción

`FlatList` crea y destruye filas mientras scrolleás, y en listas largas con filas complejas se nota: aparecen espacios en blanco si scrolleás rápido. **FlashList**, de Shopify, **recicla** las filas: la vista que sale por arriba se reutiliza para la que entra por abajo, cambiándole los datos. La API es casi idéntica:

```
import { FlashList } from '@shopify/flash-list'

<FlashList
```

```
  data={lista}
  keyExtractor={(oferta) => String(oferta.id)}
  renderItem={({ item }) => <TarjetaOferta oferta={item}
    />}
  onEndReached={cargarMas}
/>
```

Es lo que usa la pantalla principal de la app de la guía, con scroll infinito y "tirar para recargar":

app-expo/src/app/(tabs)/index.tsx

```
import { FlashList } from '@shopify/flash-list'
import { useDeferredValue, useState } from 'react'
import { ActivityIndicator, StyleSheet, Text, TextInput
  } from 'react-native'
import { SafeAreaView } from 'react-native-safe-area-
  context'
import { Cargando, ErrorConReintento } from
  '@/components/Estados'
import { TarjetaOferta } from
  '@/components/TarjetaOferta'
import { Titulo } from '@/components/Titulo'
import { colores, espacio } from '@/demos/tema'
import { useOfertas } from '@/hooks/use-ofertas'

export default function Ofertas() {
  const [texto, setTexto] = useState('')
  // La búsqueda usa una versión "atrasada" del texto:
    escribir nunca se traba
  const busqueda = useDeferredValue(texto)
  const { data, isPending, isError, error, refetch,
    isRefetching, fetchNextPage, hasNextPage,
    isFetchingNextPage } =
    useOfertas(busqueda)

  const lista = data?.pages.flatMap((pagina) =>
    pagina.ofertas) ?? []
```

```
  return (
    // edges top: el contenido empieza debajo de la isla
      dinámica; abajo lo maneja la barra de pestañas
    <SafeAreaView style={styles.pantalla} edges=
      {['top']}>
      <Titulo>Ofertas</Titulo>
      <TextInput
        value={texto}
        onChangeText={setTexto}
        placeholder="Buscar puesto, empresa o
          tecnología"
        placeholderTextColor={colores.textoSuave}
        style={styles.buscador}
        autoCorrect={false}
        clearButtonMode="while-editing"
        returnKeyType="search"
      />

      {isPending ? (
        <Cargando />
      ) : isError ? (
        <ErrorConReintento mensaje={error.message}
          reintentar={refetch} />
      ) : (
        // FlashList recicla las filas que salen de
          pantalla en vez de crear nuevas
        <FlashList
          data={lista}
          keyExtractor={(oferta) => String(oferta.id)}
          renderItem={({ item }) => <TarjetaOferta
            oferta={item} />}
          onEndReached={() => {
            if (hasNextPage && !isFetchingNextPage)
              fetchNextPage()
          }}
          onEndReachedThreshold={0.4}
          refreshing={isRefetching}
          onRefresh={refetch}
          ListEmptyComponent={<Text style=
            {styles.vacio}>No hay ofertas para
```

```
            “{busqueda}”.</Text>}
          ListFooterComponent={isFetchingNextPage ?
            <ActivityIndicator style={styles.pie} /> :
            null}
          contentContainerStyle={styles.lista}
          keyboardDismissMode="on-drag"
        />
      )}
    </SafeAreaView>
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.fondo },
  buscador: {
    marginHorizontal: espacio.l,
    marginBottom: espacio.m,
    backgroundColor: colores.tarjeta,
    borderRadius: 12,
    paddingHorizontal: espacio.l,
    paddingVertical: 12,
    fontSize: 16,
    color: colores.texto,
  },
  lista: { paddingTop: espacio.xs, paddingBottom: 100 },
    // lugar para la barra de pestañas
  vacio: { textAlign: 'center', color:
    colores.textoSuave, marginTop: espacio.xl },
  pie: { padding: espacio.l },
})
```

La versión 2 (la actual) solo funciona con la Nueva Arquitectura y ya no necesita que le digas cuánto mide cada fila (`estimatedItemSize`): lo mide sola. Si una lista tiene filas de tipos distintos (un anuncio cada 10 ofertas), pasale `getItemType` para que recicle cada tipo con los de su tipo.

> ℹ️ **Legend List**
>
> Otra opción que crece mucho es **Legend List** (`@legendapp/list`): hecha en JavaScript puro (anda en Expo Go), muy rápida, y con soporte para chats (empezar desde abajo y mantener la posición cuando llegan mensajes nuevos). FlashList y Legend List son las dos que se recomiendan hoy.

Que la lista vaya fluida

1. **Filas livianas.** Cada fila se renderiza muchas veces mientras scrolleás. Nada de cálculos pesados ni `fetch` adentro de una fila.
2. **Componentes de fila memoizados.** Con el React Compiler activado (como en esta app) es automático. Sin el compilador, `memo(TarjetaOferta)` y un `renderItem` estable con `useCallback`.
3. **Imágenes con tamaño fijo y con caché**: `expo-image`, nunca una imagen que cambia de alto al cargar.
4. **Nunca una lista adentro de un** `ScrollView` en la misma dirección: la lista pierde la virtualización y renderiza todo (React Native te avisa con *"VirtualizedLists should never be nested inside plain ScrollViews"*). Lo que iba arriba de la lista va en `ListHeaderComponent`.

5. **Keys estables**: el id, no el índice. Con el índice, al filtrar se mezclan los estados de las filas.
6. Si todas las filas miden lo mismo, en `FlatList` pasá `getItemLayout`: se saltea medir y puede ir directo a cualquier posición.

> **🎤 En una entrevista**
>
> "¿Cómo optimizás una lista lenta?" Primero medir (React Native DevTools, el monitor de rendimiento). Después: FlashList o Legend List en lugar de FlatList, filas memoizadas y livianas, imágenes cacheadas con tamaño fijo, keys estables, sacar trabajo pesado de `renderItem`, y paginar desde el servidor en lugar de traer todo.

Capítulo 7

Formularios y teclado

TextInput a fondo, el teclado que tapa todo, y react-hook-form con Zod.

Los formularios en el celular tienen dos problemas que en la web casi no existen: **el teclado tapa la mitad de la pantalla** y **cada campo necesita el teclado correcto**. Un formulario cómodo en el teléfono se nota en esos detalles.

TextInput

```
const [email, setEmail] = useState('')

<TextInput
  value={email}
  onChangeText={setEmail}          // recibe el texto
    directo, no un evento
  placeholder="tu@email.com"
  placeholderTextColor="#94a3b8"   // en Android el gris
    por defecto casi no se ve
  keyboardType="email-address"     // el teclado con @
  autoCapitalize="none"            // si no, la primera
    letra va en mayúscula
  autoCorrect={false}
```

```
  autoComplete="email"              // sugerencias y
    autocompletado del sistema
  textContentType="emailAddress"    // lo mismo, para iOS
  returnKeyType="next"              // el texto de la
    tecla Enter
  onSubmitEditing={() => passwordRef.current?.focus()}
/>
```

Las diferencias con un `<input>`:

- **`onChangeText`** recibe el string. Existe `onChange`, pero casi nunca se usa.
- **No hay `type`**: el tipo de teclado sale de `keyboardType` (`'default'`, `'email-address'`, `'numeric'`, `'number-pad'`, `'decimal-pad'`, `'phone-pad'`, `'url'`) y la contraseña de `secureTextEntry`.
- **`multiline`** lo convierte en un textarea.
- **El estilo no viene de fábrica**: sin `style`, un `TextInput` es invisible (sin borde ni fondo).

Autocompletar contraseñas y códigos

Con `autoComplete` y `textContentType`, iOS y Android ofrecen las contraseñas guardadas, sugieren una segura al registrarse y leen el código que llega por SMS:

Campo	`autoComplete`	`textContentType` (iOS)
Email de login	`'email'`	`'username'`

Campo	`autoComplete`	`textContentType` (iOS)
Contraseña	`'current-password'`	`'password'`
Contraseña nueva	`'new-password'`	`'newPassword'`
Código por SMS	`'one-time-code'` (iOS), `'sms-otp'` (Android)	`'oneTimeCode'`
Nombre	`'name'`	`'name'`
Teléfono	`'tel'`	`'telephoneNumber'`

Es un detalle que los revisores de las tiendas y los usuarios notan. Y para que iOS asocie las contraseñas con tu sitio web, se configuran los *associated domains* en `app.json`.

El flujo entre campos

En un formulario de celular, la tecla Enter del teclado dice **"Siguiente"** y te lleva al próximo campo; en el último dice **"Enviar"** y envía. Se hace con refs:

DEMO Formulario, validación y teclado



app-expo/src/demos/formularios/Postulacion.tsx

```
import { useRef, useState } from 'react'
import {
  ActivityIndicator,
  KeyboardAvoidingView,
  Platform,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
```

```
  TextInput,
  View,
  type TextInputInstance,
} from 'react-native'
import { colores, espacio } from '../tema'

type Campos = { nombre: string; email: string; mensaje:
  string }

function validar({ nombre, email, mensaje }: Campos) {
  const errores: Partial<Record<keyof Campos, string>> =
    {}
  if (nombre.trim().length < 2) errores.nombre =
    'Escribí tu nombre'
  if (!/^\S+@\S+\.\S+$/.test(email)) errores.email = 'El
    email no es válido'
  if (mensaje.trim().length < 20) errores.mensaje =
    'Contá un poco más (20 letras o más)'
  return errores
}

export default function Postulacion() {
  const [campos, setCampos] = useState<Campos>({ nombre:
    '', email: '', mensaje: '' })
  const [intentado, setIntentado] = useState(false)
  const [estado, setEstado] = useState<'editando' |
    'enviando' | 'enviado'>('editando')
  // Refs para pasar al campo siguiente con el botón
    "Siguiente" del teclado
  const emailRef = useRef<TextInputInstance>(null)
  const mensajeRef = useRef<TextInputInstance>(null)

  const errores = validar(campos)
  const cambiar = (campo: keyof Campos) => (texto:
    string) => setCampos({ ...campos, [campo]: texto })

  function enviar() {
    setIntentado(true)
    if (Object.keys(errores).length > 0) return
    setEstado('enviando')
    setTimeout(() => setEstado('enviado'), 1200)
```

```
}

if (estado === 'enviado') {
  return (
    <View style={[styles.pantalla, styles.centro]}>
      <Text style={styles.listo}>¡Listo,
        {campos.nombre.trim()}!</Text>
      <Text style={styles.ayuda}>Te vamos a escribir a
        {campos.email}.</Text>
    </View>
  )
}

return (
  // Sube el contenido cuando aparece el teclado, para
    que no tape el campo activo
  <KeyboardAvoidingView style={styles.pantalla}
    behavior={Platform.OS === 'ios' ? 'padding' :
    undefined}>
    <ScrollView contentContainerStyle=
      {styles.contenido}
      keyboardShouldPersistTaps="handled">
      <Text style={styles.etiqueta}>Nombre</Text>
      <TextInput
        style={[styles.input, intentado &&
          errores.nombre && styles.inputError]}
        value={campos.nombre}
        onChangeText={cambiar('nombre')}
        autoComplete="name"
        textContentType="name"
        returnKeyType="next"
        onSubmitEditing={() =>
          emailRef.current?.focus()}
        submitBehavior="submit"
      />
      {intentado && errores.nombre && <Text style=
        {styles.error}>{errores.nombre}</Text>}

      <Text style={styles.etiqueta}>Email</Text>
      <TextInput
        ref={emailRef}
```

```
        style={[styles.input, intentado &&
          errores.email && styles.inputError]}
        value={campos.email}
        onChangeText={cambiar('email')}
        keyboardType="email-address"
        autoCapitalize="none"
        autoComplete="email"
        textContentType="emailAddress"
        returnKeyType="next"
        onSubmitEditing={() =>
          mensajeRef.current?.focus()}
        submitBehavior="submit"
      />
      {intentado && errores.email && <Text style=
        {styles.error}>{errores.email}</Text>}

      <Text style={styles.etiqueta}>¿Por qué te
        interesa?</Text>
      <TextInput
        ref={mensajeRef}
        style={[styles.input, styles.multilinea,
          intentado && errores.mensaje &&
          styles.inputError]}
        value={campos.mensaje}
        onChangeText={cambiar('mensaje')}
        multiline
        textAlignVertical="top"
      />
      {intentado && errores.mensaje && <Text style=
        {styles.error}>{errores.mensaje}</Text>}

      <Pressable style={styles.boton} onPress={enviar}
        disabled={estado === 'enviando'}>
        {estado === 'enviando' ? (
          <ActivityIndicator color="#fff" />
        ) : (
          <Text style={styles.botonTexto}>Enviar
            postulación</Text>
        )}
      </Pressable>
    </ScrollView>
```

```
    </KeyboardAvoidingView>
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.fondo },
  centro: { alignItems: 'center', justifyContent:
    'center', padding: espacio.xl, gap: espacio.s },
  contenido: { padding: espacio.l, gap: espacio.xs },
  etiqueta: { fontSize: 13, fontWeight: '600', color:
    colores.textoSuave, marginTop: espacio.m },
  input: {
    backgroundColor: colores.tarjeta,
    borderWidth: 1,
    borderColor: colores.borde,
    borderRadius: 10,
    paddingHorizontal: espacio.m,
    paddingVertical: 10,
    fontSize: 16, // con menos de 16, Safari en iOS hace
      zoom al enfocar (en la web)
    color: colores.texto,
  },
  inputError: { borderColor: colores.error },
  multilinea: { minHeight: 90 },
  error: { color: colores.error, fontSize: 13 },
  boton: {
    backgroundColor: colores.primario,
    borderRadius: 12,
    minHeight: 48,
    alignItems: 'center',
    justifyContent: 'center',
    marginTop: espacio.xl,
  },
  botonTexto: { color: '#fff', fontWeight: '600',
    fontSize: 16 },
  listo: { fontSize: 22, fontWeight: '700', color:
    colores.texto },
  ayuda: { color: colores.textoSuave, fontSize: 15,
    textAlign: 'center' },
})
```

Tres detalles del código:

1. Las refs se tipan con `TextInputInstance`. Desde React Native 0.87 los tipos son más estrictos ("Strict TypeScript API") y `useRef<TextInput>` ya no es correcto. Si venís de la guía de TypeScript, es la misma idea que `useRef<HTMLInputElement>`.
2. `submitBehavior="submit"` en los campos intermedios hace que al apretar "Siguiente" el teclado no se cierre y se vuelva a abrir.
3. Los errores se muestran **recién después del primer intento** de enviar: marcar en rojo un campo que el usuario todavía no tocó es molesto.

El teclado que tapa todo

Cuando aparece el teclado, ocupa un 40 % de la pantalla y puede tapar el campo donde estás escribiendo o el botón de enviar.

`KeyboardAvoidingView` viene con React Native: achica o corre su contenido cuando aparece el teclado. En iOS se usa `behavior="padding"`; en Android el sistema ya hace algo parecido solo.

```
<KeyboardAvoidingView style={{ flex: 1 }} behavior=
  {Platform.OS === 'ios' ? 'padding' : undefined}>
  <ScrollView keyboardShouldPersistTaps="handled">{/*
    campos */}</ScrollView>
</KeyboardAvoidingView>
```

- `keyboardShouldPersistTaps="handled"` en el `ScrollView`: sin esto, tocar el botón "Enviar" con el teclado abierto solo cierra el teclado, y hay que tocarlo dos veces.
- `Keyboard.dismiss()` cierra el teclado desde código.
- `keyboardDismissMode="on-drag"` lo cierra al scrollear.

> 💡 **react-native-keyboard-controller**
>
> `KeyboardAvoidingView` tiene sus mañas (sobre todo en Android, con las apps de borde a borde). En proyectos reales se usa **react-native-keyboard-controller**: su `KeyboardAwareScrollView` scrollea solo hasta el campo activo, `KeyboardToolbar` agrega la barra con "anterior / siguiente / listo", y anima al mismo ritmo que el teclado. Viene incluida en Expo Go.

react-hook-form + Zod

Para formularios de verdad, la combinación más usada es la misma que en la web: **react-hook-form** maneja el estado y los errores, y **Zod** define las reglas. La diferencia: `TextInput` no es un `<input>`, así que en vez de `register` se usa `Controller`. Así es el login de la app:

app-expo/src/app/login.tsx

```
import { zodResolver } from '@hookform/resolvers/zod'
import { Controller, useForm } from 'react-hook-form'
import {
  ActivityIndicator,
  KeyboardAvoidingView,
  Platform,
  Pressable,
  StyleSheet,
  Text,
  TextInput,
  View,
} from 'react-native'
import { SafeAreaView } from 'react-native-safe-area-
  context'
import { z } from 'zod'
import { colores, espacio } from '@/demos/tema'
import { useSesion } from '@/stores/sesion'

const esquema = z.object({
  email: z.email('Escribí un email válido'),
  password: z.string().min(8, 'La contraseña tiene al
    menos 8 caracteres'),
})
type Datos = z.infer<typeof esquema>

export default function Login() {
  const iniciarSesion = useSesion((s) =>
    s.iniciarSesion)
  const {
    control,
    handleSubmit,
    setError,
    setFocus,
    formState: { errors, isSubmitting },
  } = useForm<Datos>({ resolver: zodResolver(esquema),
    defaultValues: { email: '', password: '' } })
```

```
  async function entrar({ email, password }: Datos) {
    try {
      await iniciarSesion(email, password)
      // No hace falta navegar: el Stack.Protected del
        layout muestra las pestañas solo
    } catch (error) {
      setError('root', { message: error instanceof Error
        ? error.message : 'No se pudo entrar' })
    }
  }

  return (
    <SafeAreaView style={styles.pantalla}>
      <KeyboardAvoidingView style={styles.contenido}
        behavior={Platform.OS === 'ios' ? 'padding' :
        undefined}>
        <Text style={styles.titulo}>Búsqueda RN</Text>
        <Text style={styles.subtitulo}>Usá cualquier
          email y la contraseña expo1234.</Text>

        {/* Controller conecta un TextInput (que no es
          un <input>) con react-hook-form */}
        <Controller
          control={control}
          name="email"
          render={({ field: { onChange, onBlur, value,
            ref } }) => (
            <TextInput
              ref={ref}
              style={[styles.input, errors.email &&
                styles.inputError]}
              placeholder="Email"
              value={value}
              onChangeText={onChange}
              onBlur={onBlur}
              keyboardType="email-address"
              autoCapitalize="none"
              autoComplete="email"
              textContentType="username"
              returnKeyType="next"
```

```
            onSubmitEditing={() =>
              setFocus('password')}
            submitBehavior="submit"
          />
        )}
      />
      {errors.email && <Text style={styles.error}>
        {errors.email.message}</Text>}

      <Controller
        control={control}
        name="password"
        render={({ field: { onChange, onBlur, value,
          ref } }) => (
          <TextInput
            ref={ref}
            style={[styles.input, errors.password &&
              styles.inputError]}
            placeholder="Contraseña"
            value={value}
            onChangeText={onChange}
            onBlur={onBlur}
            secureTextEntry
            autoComplete="current-password"
            textContentType="password"
            returnKeyType="go"
            onSubmitEditing={() =>
              handleSubmit(entrar)()}
          />
        )}
      />
      {errors.password && <Text style={styles.error}>
        {errors.password.message}</Text>}
      {errors.root && <Text style={styles.error}>
        {errors.root.message}</Text>}

      {/* handleSubmit(entrar) devuelve una función
        que espera un evento web: en RN se llama sin
        evento */}
      <Pressable style={styles.boton} onPress={() =>
        handleSubmit(entrar)()} disabled=
        {isSubmitting}>
```

```
          {isSubmitting ? <ActivityIndicator
            color="#fff" /> : <Text style=
            {styles.botonTexto}>Entrar</Text>}
        </Pressable>
        <View style={styles.flex} />
      </KeyboardAvoidingView>
    </SafeAreaView>
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.fondo },
  contenido: { flex: 1, padding: espacio.xl, gap:
    espacio.s, justifyContent: 'center' },
  titulo: { fontSize: 34, fontWeight: '800', color:
    colores.texto },
  subtitulo: { fontSize: 15, color: colores.textoSuave,
    marginBottom: espacio.l },
  input: {
    backgroundColor: colores.tarjeta,
    borderWidth: 1,
    borderColor: colores.borde,
    borderRadius: 12,
    paddingHorizontal: espacio.l,
    paddingVertical: 14,
    fontSize: 16,
    color: colores.texto,
  },
  inputError: { borderColor: colores.error },
  error: { color: colores.error, fontSize: 13 },
  boton: {
    backgroundColor: colores.primario,
    borderRadius: 12,
    minHeight: 50,
    alignItems: 'center',
    justifyContent: 'center',
    marginTop: espacio.m,
  },
  botonTexto: { color: '#fff', fontSize: 16, fontWeight:
    '600' },
  flex: { flex: 0.3 },
```

```
})
```

- `zodResolver(esquema)` conecta las reglas de Zod; `z.infer` saca el tipo de los datos.
- El `ref` que da `Controller` va al `TextInput`, y así `setFocus('password')` pasa al campo siguiente.
- `setError('root', …)` muestra un error que no es de un campo ("contraseña incorrecta").
- El botón llama `handleSubmit(entrar)()` dentro de una función: con los tipos estrictos de React Native, `onPress={handleSubmit(entrar)}` no compila, porque el evento de un toque no es el de un formulario web.

Una alternativa que crece: **TanStack Form**, con validación por campo y muy buen tipado.

> ℹ️ **¿Y las Actions de React 19?**
>
> `<form action={...}>` es de la web: en React Native no hay `<form>`. Pero `useActionState`, `useOptimistic` y `useTransition` funcionan igual: llamás a la acción desde el `onPress`. Lo de la guía de React 19 aplica tal cual.

Selects, fechas y otros controles

No hay `<select>` ni `<input type="date">`. Las opciones:

Necesitás	Opción
Elegir una opción de pocas	Botones tipo chip, o un menú nativo con `@expo/ui`
Elegir de una lista larga	Una ruta modal o un bottom sheet con una lista y buscador
Una fecha u hora	`@react-native-community/datetimepicker`, o el `DateTimePicker` de `@expo/ui`
Un menú de acciones (iOS)	`ActionSheetIOS.showActionSheetWithOptions`
Un slider	`@expo/ui` o `@react-native-community/slider`

Capítulo 8

Toques, gestos y animaciones

Pressable, háptica, Animated, Reanimated 4 y Gesture Handler.

Lo que hace que una app se sienta "nativa" no son los colores: es que **responde al toque al instante**, que las cosas se mueven siguiendo al dedo, y que vibra apenas cuando tiene que hacerlo.

Feedback al tocar

Todo lo tocable tiene que reaccionar: con `Pressable` cambiás el estilo mientras está presionado (lo vimos en componentes). En Android también podés usar el efecto de onda del sistema:

```
<Pressable android_ripple={{ color: 'rgba(0,0,0,0.1)' }}
  style={styles.fila}>…</Pressable>
```

Háptica: la vibración suave

`expo-haptics` da acceso al motor de vibración del teléfono. Bien usado, hace que la app se sienta cara; mal usado (vibrar en cada toque), cansa.

```
import * as Haptics from 'expo-haptics'

Haptics.selectionAsync()
  // un "tic": elegir algo, un switch
Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light)
  // un golpe suave: soltar una tarjeta
Haptics.notificationAsync(Haptics.NotificationFeedbackTy
  pe.Success)  // terminó bien (o Error, Warning)
```

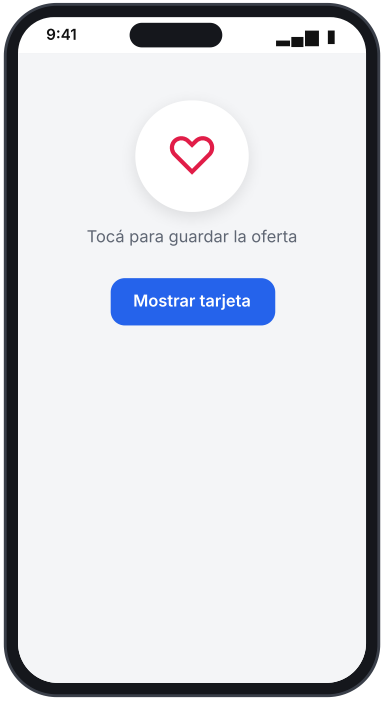
En la app de la guía, el corazón de "guardar oferta" hace `selectionAsync` y "Postularme" hace `notificationAsync(Success)`.

Animated: las animaciones que vienen con React Native

La API `Animated` viene incluida. Creás un `Animated.Value`, lo animás con `timing` o `spring`, y lo conectás a un `Animated.View`:

DEMO

Animated: secuencias, resortes e interpolación



app-expo/src/demos/interaccion/Rebote.tsx

```
import { useRef, useState } from 'react'
import { Animated, Pressable, StyleSheet, Text, View }
  from 'react-native'
import { colores, espacio } from '../tema'

// La API Animated viene con React Native. Un
  Animated.Value guarda un número que cambia
// sin pasar por el estado de React, y Animated.View lo
  aplica directo al estilo.
export default function Rebote() {
```

```
const escala = useRef(new Animated.Value(1)).current
const entrada = useRef(new Animated.Value(0)).current
const [guardada, setGuardada] = useState(false)

function guardar() {
  setGuardada((g) => !g)
  // Achica y vuelve con un resorte
  Animated.sequence([
    Animated.timing(escala, { toValue: 0.8, duration:
      90, useNativeDriver: true }),
    Animated.spring(escala, { toValue: 1, friction: 3,
      tension: 200, useNativeDriver: true }),
  ]).start()
}

function mostrar() {
  entrada.setValue(0)
  Animated.timing(entrada, { toValue: 1, duration:
    500, useNativeDriver: true }).start()
}

return (
  <View style={styles.pantalla}>
    <Pressable onPress={guardar}>
      <Animated.View style={[styles.corazon, {
        transform: [{ scale: escala }] }]}>
        <Text style={styles.icono}>{guardada ? '♥' :
          '♡'}</Text>
      </Animated.View>
    </Pressable>
    <Text style={styles.ayuda}>{guardada ? 'Guardada'
      : 'Tocá para guardar la oferta'}</Text>

    <Pressable style={styles.boton} onPress={mostrar}>
      <Text style={styles.botonTexto}>Mostrar
        tarjeta</Text>
    </Pressable>

    <Animated.View
      style={[
        styles.tarjeta,
```

```
          {
            opacity: entrada,
            // interpolate convierte 0→1 en otro rango:
              acá, de 40 px abajo a su lugar
            transform: [{ translateY:
              entrada.interpolate({ inputRange: [0, 1],
              outputRange: [40, 0] }) }],
          },
        ]}
      >
        <Text style={styles.titulo}>Nueva oferta</Text>
        <Text style={styles.suave}>Tech Lead Mobile en
          Yerba Labs</Text>
      </Animated.View>
    </View>
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.fondo,
    padding: espacio.l, alignItems: 'center', gap:
    espacio.m },
  corazon: {
    width: 96,
    height: 96,
    borderRadius: 48,
    backgroundColor: colores.tarjeta,
    alignItems: 'center',
    justifyContent: 'center',
    marginTop: espacio.xl,
    boxShadow: '0 4px 14px rgba(0,0,0,0.1)',
  },
  icono: { fontSize: 44, color: '#e11d48' },
  ayuda: { color: colores.textoSuave },
  boton: { backgroundColor: colores.primario,
    paddingVertical: 12, paddingHorizontal: 20,
    borderRadius: 12, marginTop: espacio.l },
  botonTexto: { color: '#fff', fontWeight: '600' },
  tarjeta: { alignSelf: 'stretch', backgroundColor:
    colores.tarjeta, borderRadius: 14, padding:
    espacio.l },
```

```
  titulo: { fontWeight: '700', fontSize: 16, color:
    colores.texto },
  suave: { color: colores.textoSuave, marginTop: 2 },
})
```

- El valor se crea una vez por componente: `useAnimatedValue(0)` (o, en código viejo, `useRef(new Animated.Value(0)).current`, como en la demo, que también tiene que correr en la web).
- **`useNativeDriver: true`** manda la animación al lado nativo antes de empezar: después corre sin pasar por JavaScript. Funciona solo con `transform` y `opacity` (no con `width` ni colores).
- **`interpolate`** convierte un rango en otro: el mismo valor de 0 a 1 mueve, achica y desvanece.
- `Animated.sequence`, `parallel`, `stagger` y `loop` combinan animaciones.

Seguir al dedo con PanResponder

Lo mismo para gestos: `PanResponder` viene con React Native y avisa dónde está el dedo. Arrastrá la tarjeta hacia los costados:

DEMO Arrastrar una tarjeta con PanResponder



app-expo/src/demos/interaccion/Deslizar.tsx

```
import { useRef, useState } from 'react'
import { Animated, PanResponder, StyleSheet, Text, View
  } from 'react-native'
import { ofertas } from '../datos'
import { colores, espacio } from '../tema'

const UMBRAL = 110 // px que hay que arrastrar para
  decidir

// Arrastrar una tarjeta: a la derecha la guarda, a la
  izquierda la descarta.
```

```
// PanResponder viene con React Native; en una app real
  se usa react-native-gesture-handler
// con Reanimated, que corren en el hilo de UI (ver el
  capítulo).
export default function Deslizar() {
  const [indice, setIndice] = useState(0)
  const [guardadas, setGuardadas] = useState(0)
  const posicion = useRef(new
    Animated.ValueXY()).current

  function decidir(direccion: 1 | -1) {
    Animated.timing(posicion, { toValue: { x: direccion
      * 500, y: 0 }, duration: 200, useNativeDriver:
      true }).start(() => {
      if (direccion === 1) setGuardadas((n) => n + 1)
      setIndice((i) => i + 1)
      posicion.setValue({ x: 0, y: 0 })
    })
  }

  const responder = useRef(
    PanResponder.create({
      onMoveShouldSetPanResponder: (_, gesto) =>
        Math.abs(gesto.dx) > 5,
      onPanResponderMove: Animated.event([null, { dx:
        posicion.x, dy: posicion.y }], {
        useNativeDriver: false }),
      onPanResponderRelease: (_, gesto) => {
        if (gesto.dx > UMBRAL) decidir(1)
        else if (gesto.dx < -UMBRAL) decidir(-1)
        else Animated.spring(posicion, { toValue: { x:
          0, y: 0 }, useNativeDriver: true }).start()
      },
    }),
  ).current

  const oferta = ofertas[indice % ofertas.length]
  const rotacion = posicion.x.interpolate({ inputRange:
    [-200, 0, 200], outputRange: ['-12deg', '0deg',
    '12deg'] })

  return (
```

```
    <View style={styles.pantalla}>
      <Text style={styles.contador}>Guardadas:
        {guardadas}</Text>
      <Animated.View
        {...responder.panHandlers}
        style={[styles.tarjeta, { transform:
          [...posicion.getTranslateTransform(), {
          rotate: rotacion }] }]}
      >
        <Text style={styles.empresa}>{oferta.empresa}
          </Text>
        <Text style={styles.titulo}>{oferta.titulo}
          </Text>
        <Text style={styles.salario}>Hasta US$
          {oferta.salarioMax.toLocaleString('es')}
          </Text>
      </Animated.View>
      <Text style={styles.ayuda}>← Descartar · Guardar
        →</Text>
    </View>
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.fondo,
    alignItems: 'center', justifyContent: 'center',
    padding: espacio.xl, gap: espacio.l, overflow:
    'hidden' },
  contador: { color: colores.textoSuave, fontWeight:
    '600' },
  tarjeta: {
    alignSelf: 'stretch',
    height: 220,
    backgroundColor: colores.tarjeta,
    borderRadius: 20,
    padding: espacio.xl,
    justifyContent: 'flex-end',
    gap: espacio.xs,
    boxShadow: '0 8px 24px rgba(0,0,0,0.12)',
  },
  empresa: { color: colores.primario, fontWeight: '700',
    textTransform: 'uppercase', fontSize: 12 },
```

```
  titulo: { fontSize: 22, fontWeight: '700', color:
    colores.texto },
  salario: { color: colores.exito, fontWeight: '600' },
  ayuda: { color: colores.textoSuave },
})
```

Funciona, pero el gesto pasa por el hilo de JavaScript: si JS está ocupado, la tarjeta se traba. Por eso, para gestos y animaciones serias, se usan las dos librerías de abajo.

Reanimated 4

Reanimated es la librería de animaciones de React Native (viene en la plantilla de Expo). La idea clave: las animaciones corren en el **hilo de UI**, con funciones especiales llamadas *worklets*. Aunque JavaScript esté ocupado, la animación sigue a 60 o 120 cuadros por segundo.

```
import Animated, { useAnimatedStyle, useSharedValue,
  withSequence, withSpring } from 'react-native-
  reanimated'

function Corazon() {
  const escala = useSharedValue(1)
    // un valor que vive en el hilo de UI
  const estilo = useAnimatedStyle(() => ({
    // esta función es un worklet
    transform: [{ scale: escala.get() }],
  }))

  return (
```

```
    <Pressable onPress={() =>
      escala.set(withSequence(withSpring(1.3),
      withSpring(1)))}>
      <Animated.View style={estilo}>…</Animated.View>
    </Pressable>
  )
}
```

- `useSharedValue`: como un `useRef`, pero visible desde el hilo de UI. Cambiarlo **no** re-renderiza. Se lee y se cambia con `.value`, o con `.get()` y `.set()`: con el React Compiler activado conviene `get` / `set`, porque el compilador no permite asignar propiedades de lo que devuelve un hook.
- `useAnimatedStyle`: un estilo que se recalcula en el hilo de UI cada vez que cambia un shared value.
- `withTiming`, `withSpring`, `withDecay`, `withSequence`, `withRepeat`: animan el valor.
- **Animaciones de layout**: `entering={FadeInDown}` y `exiting={FadeOut}` animan un elemento al aparecer y desaparecer, y `layout={LinearTransition}` cuando cambia de lugar. Ideal para listas que se filtran.

Transiciones al estilo CSS

La novedad de Reanimated 4: entiende **transiciones y animaciones con la sintaxis de CSS**. Para animar un cambio de estado no hace falta ni un shared value:

app-expo/src/ejemplos/TransicionCss.tsx

```tsx
import { useState } from 'react'
import { Pressable, Text } from 'react-native'
import Animated from 'react-native-reanimated'

// Reanimated 4 también entiende transiciones al estilo
  CSS: cuando cambia el estilo,
// anima el cambio solo. Para animaciones simples de
  estado no hace falta nada más.
export function BotonExpandible() {
  const [abierto, setAbierto] = useState(false)

  return (
    <Pressable onPress={() => setAbierto((a) => !a)}>
      <Animated.View
        style={{
          width: abierto ? 260 : 140,
          backgroundColor: abierto ? '#2563eb' :
            '#94a3b8',
          borderRadius: 14,
          padding: 16,
          transitionProperty: ['width',
            'backgroundColor'],
          transitionDuration: 300,
          transitionTimingFunction: 'ease-in-out',
        }}
      >
        <Text style={{ color: '#fff', fontWeight: '600'
          }}>{abierto ? 'Filtros abiertos' : 'Filtros'}
          </Text>
      </Animated.View>
    </Pressable>
  )
}
```

También hay `animationName` con keyframes (`{ from: {...}, to: {...} }`), `animationDuration` y `animationIterationCount: 'infinite'`, como `@keyframes` en CSS.

Gesture Handler: gestos de verdad

react-native-gesture-handler reconoce gestos en el hilo de UI: arrastrar, pellizcar, rotar, deslizar, mantener presionado. Combinado con Reanimated, el gesto y la animación nunca pasan por JavaScript. Esta es la tarjeta de la demo anterior, hecha como se hace en una app real:

app-expo/src/ejemplos/TarjetaDeslizable.tsx

```
import { StyleSheet, Text } from 'react-native'
import { GestureDetector, usePanGesture } from 'react-
  native-gesture-handler'
import Animated, {
  FadeInDown,
  interpolate,
  useAnimatedStyle,
  useSharedValue,
  withSpring,
  withTiming,
} from 'react-native-reanimated'
import { scheduleOnRN } from 'react-native-worklets'

const UMBRAL = 120

// La demo "Arrastrar una tarjeta", como se hace en una
  app de verdad: Gesture Handler
```

```
// detecta el gesto y Reanimated mueve la tarjeta, los
  dos en el hilo de UI. Aunque el hilo
// de JavaScript esté ocupado, la tarjeta sigue al dedo
  sin saltos.
export function TarjetaDeslizable({ titulo, alDecidir }:
  { titulo: string; alDecidir: (guardar: boolean) =>
  void }) {
  // Un valor que vive en el hilo de UI. Se lee con
    get() y se cambia con set():
  // x.value también funciona, pero get/set es lo que
    entiende el React Compiler.
  const x = useSharedValue(0)

  const arrastre = usePanGesture({
    onUpdate: (evento) => {
      x.set(evento.translationX)
    },
    onDeactivate: (evento) => {
      if (Math.abs(evento.translationX) < UMBRAL) {
        x.set(withSpring(0)) // vuelve a su lugar con un
          resorte
        return
      }
      const guardar = evento.translationX > 0
      x.set(
        withTiming(guardar ? 500 : -500, { duration: 200
          }, () => {
          // Este callback corre en el hilo de UI: para
            llamar a código de React, scheduleOnRN
          scheduleOnRN(alDecidir, guardar)
        }),
      )
    },
  })

  const estilo = useAnimatedStyle(() => ({
    transform: [{ translateX: x.get() }, { rotate:
      `${interpolate(x.get(), [-200, 0, 200], [-12, 0,
      12])}deg` }],
  }))

  return (
```

```
    <GestureDetector gesture={arrastre}>
      {/* entering: una animación de layout al aparecer,
        sin escribir nada más */}
      <Animated.View entering={FadeInDown} style=
        {[styles.tarjeta, estilo]}>
        <Text style={styles.titulo}>{titulo}</Text>
      </Animated.View>
    </GestureDetector>
  )
}

const styles = StyleSheet.create({
  tarjeta: { height: 200, borderRadius: 20, padding: 24,
    justifyContent: 'flex-end', backgroundColor: '#fff'
    },
  titulo: { fontSize: 22, fontWeight: '700' },
})
```

- La versión 3 usa **hooks** por gesto: `usePanGesture`, `useTapGesture`, `usePinchGesture`, `useLongPressGesture` … y `GestureDetector` envuelve la vista.
- Los callbacks (`onUpdate`, `onDeactivate`) son worklets: corren en el hilo de UI.
- Para llamar a una función de React desde un worklet (avisar que se decidió), se usa **`scheduleOnRN`** de `react-native-worklets` (antes se llamaba `runOnJS`).
- La app tiene que estar envuelta en **`GestureHandlerRootView`**; en la de la guía está en `src/app/_layout.tsx`.

> **⚠️ Respetá 'reducir movimiento'**
>
> Hay usuarios que activan "Reducir movimiento" en el sistema porque las animaciones les causan mareo. Reanimated respeta esa opción por defecto en las animaciones de layout, y el hook `useReducedMotion()` te dice si está activa para tus animaciones propias.

Más librerías de interacción

Necesitás	Librería
Deslizar una fila para borrar	`ReanimatedSwipeable`, incluida en Gesture Handler
Un panel que sube desde abajo (bottom sheet)	**@gorhom/bottom-sheet**, o una ruta con `presentation: 'formSheet'`
Animaciones de After Effects	**lottie-react-native**
Dibujar con alto rendimiento (gráficos, efectos)	**@shopify/react-native-skia**
Animaciones declarativas simples	**Moti** (sobre Reanimated)
Gráficos	**victory-native** (sobre Skia)

🎤 En una entrevista

"¿Cuál es la diferencia entre Animated y Reanimated?" Animated viene con React Native y, con `useNativeDriver`, corre en el lado nativo solo transform y opacity, y los gestos pasan por JS. Reanimated corre worklets en el hilo de UI: cualquier propiedad, gestos sincronizados con Gesture Handler, animaciones de layout, y desde la v4 transiciones estilo CSS. Para algo más que un fade, Reanimated.

Parte 2 · Una app de verdad

Lo que tiene cualquier app: navegación, datos del servidor, estado guardado, login y funciones del teléfono.

Capítulo 9

Navegación con Expo Router

Rutas como archivos, Stack, pestañas nativas, parámetros, modales, rutas protegidas y deep links.

Expo Router es la navegación de Expo: cada archivo de `src/app/` es una pantalla, como en Next.js. Por debajo usa las pilas y pestañas **nativas** del sistema (con sus animaciones y gestos), y además cada pantalla tiene una URL, así que los deep links y la web funcionan solos.

Rutas como archivos

Archivo	URL	Qué es
`app/_layout.tsx`	—	El layout raíz: envuelve todo (providers, Stack)
`app/index.tsx`	`/`	La pantalla inicial
`app/login.tsx`	`/login`	
`app/oferta/[id].tsx`	`/oferta/5`	Ruta dinámica: `id = '5'`
`app/(tabs)/_layout.tsx`	—	Las pestañas del grupo
`app/(tabs)/guardadas.tsx`	`/guardadas`	Los paréntesis no suman a la URL
`app/[...resto].tsx`	`/lo/que/sea`	Atrapa todo (`resto = ['lo','que','sea']`)
`app/+not-found.tsx`	—	Cuando ninguna ruta coincide

Un **`_layout.tsx`** define cómo se muestran las pantallas de su carpeta: como una pila (`Stack`), como pestañas (`NativeTabs`) o como lo que quieras (`Slot` muestra la pantalla sin nada alrededor). Los layouts se anidan: la app de la guía tiene un `Stack` en la raíz, y adentro un grupo `(tabs)` con pestañas.

Stack: la pila de pantallas

En el celular, navegar es **apilar**: abrís una oferta y la pantalla nueva entra desde la derecha (en iOS) con su botón "atrás" y el gesto de deslizar desde el borde para volver. Eso es un `Stack`, y viene configurado en el layout raíz:

app-expo/src/app/_layout.tsx

```
import { QueryClient, QueryClientProvider } from
  '@tanstack/react-query'
import { Stack } from 'expo-router'
import * as SplashScreen from 'expo-splash-screen'
import { StatusBar } from 'expo-status-bar'
import { useEffect, useSyncExternalStore } from 'react'
import { GestureHandlerRootView } from 'react-native-
  gesture-handler'
import { useSesion } from '@/stores/sesion'

// La pantalla de carga (splash) queda visible hasta que
  leamos la sesión guardada
SplashScreen.preventAutoHideAsync()

const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 60_000, retry:
    2 } },
})

export default function RootLayout() {
  const logueado = useSesion((s) => s.token !== null)
  // true cuando zustand terminó de leer la sesión del
    almacenamiento seguro
  const hidratada =
    useSyncExternalStore(useSesion.persist.onFinishHydra
    tion, useSesion.persist.hasHydrated)
```

```
  useEffect(() => {
    if (hidratada) SplashScreen.hideAsync()
  }, [hidratada])

  if (!hidratada) return null

  return (
    // Gesture Handler necesita esta vista en la raíz
      para reconocer gestos en toda la app
    <GestureHandlerRootView style={{ flex: 1 }}>
      <QueryClientProvider client={queryClient}>
        <Stack screenOptions={{
          headerBackButtonDisplayMode: 'minimal' }}>
          {/* Con sesión: las pestañas y las pantallas
            de detalle */}
          <Stack.Protected guard={logueado}>
            <Stack.Screen name="(tabs)" options={{
              headerShown: false }} />
            <Stack.Screen name="oferta/[id]" options={{
              title: '' }} />
            <Stack.Screen name="demo/[id]" />
          </Stack.Protected>
          {/* Sin sesión: solo el login. Al cambiar
            `logueado`, Expo Router redirige solo */}
          <Stack.Protected guard={!logueado}>
            <Stack.Screen name="login" options={{
              headerShown: false }} />
          </Stack.Protected>
        </Stack>
        <StatusBar style="dark" />
      </QueryClientProvider>
    </GestureHandlerRootView>
  )
}
```

Las opciones del header se configuran en el layout (`<Stack.Screen name="…" options={…} />`) o **desde la**

pantalla misma, que es útil cuando el título depende de datos:

```
// app/oferta/[id].tsx
<Stack.Screen options={{ title: oferta.empresa }} />
```

Opciones útiles: `title`, `headerShown`, `headerLargeTitle` (el título grande de iOS), `headerRight` (botones), `headerBackButtonDisplayMode: 'minimal'` (solo la flecha), `presentation` (cómo se presenta: lo vemos abajo).

Navegar

```
import { Link, router } from 'expo-router'

// Declarativo: como un <a>
<Link href="/guardadas">Ver guardadas</Link>
<Link href={{ pathname: '/oferta/[id]', params: { id: 5
  } }}>Ver oferta</Link>

// Con asChild, el Link le pasa el toque a su hijo (un
  Pressable con tu estilo)
<Link href="/perfil" asChild>
  <Pressable style={styles.boton}><Text>Perfil</Text>
    </Pressable>
</Link>

// Imperativo: desde una función
router.push('/oferta/5')       // apila
router.replace('/login')       // reemplaza la actual
  (no se puede volver)
router.back()                  // vuelve
```

```
router.dismissAll()            // cierra todas las
  pantallas apiladas
router.canGoBack()
```

Con `experiments.typedRoutes` en `app.json`, las rutas están tipadas: `router.push('/ofertaa/5')` no compila, y los `params` de una ruta dinámica son obligatorios. La tarjeta de oferta navega así:

app-expo/src/components/TarjetaOferta.tsx

```
import * as Haptics from 'expo-haptics'
import { router } from 'expo-router'
import { memo } from 'react'
import { Pressable, StyleSheet, Text, View } from
  'react-native'
import { nombreModalidad, type Oferta } from
  '@/demos/datos'
import { colores, espacio } from '@/demos/tema'
import { useEstaGuardada, useGuardadas } from
  '@/stores/guardadas'

// memo: en una lista larga, una fila solo se vuelve a
  renderizar si cambian sus props.
// (Con el React Compiler activado esto se hace solo,
  pero conviene saber por qué importa.)
export const TarjetaOferta = memo(function
  TarjetaOferta({ oferta }: { oferta: Oferta }) {
  const guardada = useEstaGuardada(oferta.id)
  const alternar = useGuardadas((s) => s.alternar)

  return (
    // router.push navega desde código; con typedRoutes,
      la ruta y sus parámetros están tipados
    <Pressable
      onPress={() => router.push({ pathname:
        '/oferta/[id]', params: { id: oferta.id } })}
```

```
      style={({ pressed }) => [styles.tarjeta, pressed
        && styles.presionada]}
    >
      <View style={styles.flex}>
        <Text style={styles.empresa}>{oferta.empresa}
          </Text>
        <Text style={styles.titulo}>{oferta.titulo}
          </Text>
        <Text style={styles.suave}>
          {nombreModalidad[oferta.modalidad]} ·
            {oferta.pais} · hasta US$
            {oferta.salarioMax.toLocaleString('es')}
        </Text>
      </View>
      <Pressable
        hitSlop={12}
        accessibilityRole="button"
        accessibilityLabel={guardada ? 'Quitar de
          guardadas' : 'Guardar oferta'}
        onPress={() => {
          Haptics.selectionAsync()
          alternar(oferta)
        }}
      >
        <Text style={[styles.corazon, guardada &&
          styles.corazonActivo]}>{guardada ? '♥' : '♡'}
          </Text>
      </Pressable>
    </Pressable>
  )
})

const styles = StyleSheet.create({
  tarjeta: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: espacio.m,
    backgroundColor: colores.tarjeta,
    marginHorizontal: espacio.l,
    marginBottom: espacio.m,
    padding: espacio.l,
```

```
    borderRadius: 14,
    boxShadow: '0 1px 4px rgba(0,0,0,0.06)',
  },
  presionada: { opacity: 0.75 },
  flex: { flex: 1, gap: 2 },
  empresa: { fontSize: 12, fontWeight: '700', color:
    colores.primario, textTransform: 'uppercase' },
  titulo: { fontSize: 16, fontWeight: '600', color:
    colores.texto },
  suave: { fontSize: 13, color: colores.textoSuave },
  corazon: { fontSize: 26, color: colores.textoSuave },
  corazonActivo: { color: '#e11d48' },
})
```

Parámetros

```
// app/oferta/[id].tsx
const { id } = useLocalSearchParams<{ id: string }>()
const oferta = useOferta(Number(id))
```

Los parámetros **siempre llegan como string** (vienen de una URL), aunque hayas pasado un número. También llegan los de la query: `/ofertas?modalidad=remoto` da `{ modalidad: 'remoto' }`. Para cambiarlos sin navegar: `router.setParams({ modalidad: 'hibrido' })`.

> 💡 **Pasá ids, no objetos**
>
> No pases la oferta entera como parámetro: pasá el `id` y que la pantalla de detalle la busque. Con TanStack Query, si la lista ya la trajo, puede estar en caché y aparece al instante. Así la pantalla también funciona si se abre desde un deep link o una notificación.

Pestañas nativas

`NativeTabs` usa la barra de pestañas **del sistema**: en iOS 26, la de Liquid Glass, y en Android, la de Material. Cada `Trigger` apunta a un archivo de la carpeta; el ícono es un SF Symbol en iOS (`sf`) y un Material Symbol en Android (`md`):

app-expo/src/app/(tabs)/_layout.tsx

```
import { NativeTabs } from 'expo-router/native-tabs'
import { colores } from '@/demos/tema'

// Pestañas nativas: la barra de pestañas del sistema
  (UITabBar en iOS, con Liquid Glass,
// y la de Material en Android). Cada Trigger apunta a
  un archivo de esta carpeta.
export default function TabsLayout() {
  return (
    <NativeTabs tintColor={colores.primario}>
      <NativeTabs.Trigger name="index">

          <NativeTabs.Trigger.Label>Ofertas</NativeTabs.
          Trigger.Label>
```

```
        <NativeTabs.Trigger.Icon sf="briefcase"
          md="work" />
      </NativeTabs.Trigger>
      <NativeTabs.Trigger name="guardadas">

          <NativeTabs.Trigger.Label>Guardadas</NativeTab
          s.Trigger.Label>
        <NativeTabs.Trigger.Icon sf={{ default: 'heart',
          selected: 'heart.fill' }} md="favorite" />
      </NativeTabs.Trigger>
      <NativeTabs.Trigger name="demos">

          <NativeTabs.Trigger.Label>Demos</NativeTabs.Tr
          igger.Label>
        <NativeTabs.Trigger.Icon sf="square.grid.2x2"
          md="apps" />
      </NativeTabs.Trigger>
      <NativeTabs.Trigger name="perfil">

          <NativeTabs.Trigger.Label>Perfil</NativeTabs.T
          rigger.Label>
        <NativeTabs.Trigger.Icon sf="person.crop.circle"
          md="account_circle" />
      </NativeTabs.Trigger>
    </NativeTabs>
  )
}
```

En la web no hay barra nativa: la app usa `(tabs)/_layout.web.tsx` con `Tabs` de `expo-router/js-tabs` (las pestañas hechas en JavaScript, que también sirven si necesitás una barra de pestañas totalmente personalizada).

Cada pestaña puede tener su propio `Stack` adentro (una carpeta con su `_layout.tsx`), así navegar dentro de "Ofertas" no te saca de la pestaña.

Modales y hojas

Una pantalla que se abre desde abajo es una ruta más, con `presentation`:

```
// app/_layout.tsx
<Stack.Screen name="filtros" options={{ presentation:
  'modal' }} />

// Una hoja que ocupa media pantalla y se puede agrandar
  arrastrando (como en Apple Maps)
<Stack.Screen
  name="compartir"
  options={{ presentation: 'formSheet',
    sheetAllowedDetents: [0.5, 1], sheetGrabberVisible:
    true }}
/>
```

Se abre con `router.push('/filtros')` y se cierra con `router.back()` o deslizando hacia abajo. A diferencia del componente `Modal`, tiene URL, animación nativa y el gesto de cerrar.

Rutas protegidas: el login

La app de la guía muestra el login o las pestañas según haya sesión, con `Stack.Protected` (en el layout raíz de arriba). Las pantallas dentro de `guard={logueado}` solo existen si hay sesión; si `logueado` cambia, Expo Router redirige solo. Por eso el login no hace `router.replace('/')`: guarda el token, y el layout hace el resto. Lo vemos completo en login y sesión.

Deep links

Como cada pantalla tiene URL, los links funcionan sin configurar nada:

- **Esquema propio**: con `"scheme": "busquedarn"` en `app.json`, el link `busquedarn://oferta/5` abre la app en esa oferta. Probalo con `npx uri-scheme open busquedarn://oferta/5 --ios`.
- **Universal Links (iOS) y App Links (Android)**: `https://tusitio.com/oferta/5` abre la app si está instalada, y el sitio si no. Necesitan `ios.associatedDomains` y `android.intentFilters` en `app.json`, y un archivo de verificación en tu dominio (`apple-app-site-association` y `assetlinks.json`).

Hooks de navegación

Hook	Para qué
`useLocalSearchParams()`	Los parámetros de esta pantalla.
`useRouter()` / `router`	Navegar (`router` sirve también fuera de componentes).
`usePathname()`	La URL actual: `/oferta/5`.
`useSegments()`	La ruta en partes: `['(tabs)', 'guardadas']`.
`useFocusEffect(cb)`	Como `useEffect`, pero corre cada vez que la pantalla **vuelve a verse**.

Hook	Para qué
`useNavigation()`	El objeto de React Navigation, para cosas avanzadas.

> ⚠️ **Las pantallas no se desmontan**
>
> En una pila, la pantalla de atrás **sigue montada** cuando apilás otra encima. Un `useEffect` con `[]` no vuelve a correr cuando el usuario vuelve atrás. Si querés recargar algo al volver a una pantalla, usá `useFocusEffect` (o deja que TanStack Query lo haga).

Cosas que trae Expo Router y conviene saber que existen

- `Link.Preview`: en iOS, mantener presionado un link muestra una vista previa de la pantalla, como en Safari.
- **API routes** (`app/api/ofertas+api.ts`): endpoints de servidor en el mismo proyecto, cuando la web se publica con `output: "server"`.
- **Loaders** (estables desde el SDK 58): una función `loader` que trae datos en el servidor antes de mostrar la pantalla en la web, y `useLoaderData` para leerlos.

ℹ️ ¿Y React Navigation?

Expo Router nació sobre **React Navigation**, la librería de navegación clásica de React Native, y vas a ver mucho código con `createNativeStackNavigator` y `navigation.navigate('Oferta', { id })`. Desde el SDK 56, Expo Router ya no depende de ella, pero los conceptos (stack, tabs, opciones de pantalla, `useFocusEffect`) son los mismos. Si en una entrevista te preguntan por React Navigation, todo lo de este capítulo aplica.

Capítulo 10

Datos del servidor

fetch, TanStack Query, paginación infinita, variables de entorno y modo offline.

Traer datos de una API en React Native es casi igual que en la web: `fetch` existe y funciona igual. Lo que cambia es el entorno: la red se corta seguido, la app pasa a segundo plano, y "localhost" no es lo que pensás.

fetch, con tres diferencias

```
const respuesta = await
  fetch('https://api.ejemplo.com/ofertas')
const ofertas = await respuesta.json()
```

1. **No hay CORS** en el celular. CORS es una regla de los navegadores; tu app puede llamar a cualquier API. (En la versión web de tu app, sí aplica).
2. `localhost` **es el teléfono**, no tu computadora. Para llamar a tu backend local desde el celular, usá la IP de tu compu en la red (`http://192.168.0.10:3000`).

En el emulador de Android, tu computadora es `10.0.2.2`.

3. **HTTP sin S está bloqueado** por defecto en iOS y en Android (en las builds de producción). Tu API tiene que ser `https://`.

Si necesitás leer una respuesta de a pedazos (streaming, como un chat con IA), `import { fetch } from 'expo/fetch'` es un `fetch` compatible con el estándar que soporta streams.

Un cliente de API

En lugar de repetir la URL base, los headers y el manejo de errores en cada llamada, se arma una función:

app-expo/src/ejemplos/cliente-api.ts

```
import { useSesion } from '@/stores/sesion'

// Las variables que empiezan con EXPO_PUBLIC_ se copian
  dentro del bundle al compilar.
// Sirven para URLs y claves públicas; NUNCA pongas acá
  secretos (cualquiera puede leerlas).
const BASE_URL = process.env.EXPO_PUBLIC_API_URL ??
  'http://localhost:3000'

export class ErrorApi extends Error {
  constructor(
    public status: number,
    mensaje: string,
  ) {
    super(mensaje)
```

```
  }
}

/** fetch con la URL base, el token de la sesión,
  timeout y errores con status. */
export async function api<T>(ruta: string, init:
  RequestInit = {}): Promise<T> {
  const token = useSesion.getState().token // fuera de
    un componente: getState()
  const respuesta = await fetch(`${BASE_URL}${ruta}`, {
    ...init,
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } :
        {}),
      ...init.headers,
    },
    // En el celular la red se corta seguido: sin
      timeout, la promesa puede quedar colgada
    signal: init.signal ?? AbortSignal.timeout(15_000),
  })

  if (respuesta.status === 401) {
    useSesion.getState().cerrarSesion() // token
      vencido: Stack.Protected lo manda al login
    throw new ErrorApi(401, 'Tu sesión venció')
  }
  if (!respuesta.ok) throw new
    ErrorApi(respuesta.status, await respuesta.text())
  if (respuesta.status === 204) return undefined as T
  return (await respuesta.json()) as T
}

// Uso con TanStack Query:
//   useQuery({ queryKey: ['oferta', id], queryFn: () =>
  api<Oferta>(`/ofertas/${id}`) })
//   useMutation({ mutationFn: (datos) =>
  api('/postulaciones', { method: 'POST', body:
  JSON.stringify(datos) }) })
```

El token se lee con `useSesion.getState()`: los stores de Zustand se pueden leer fuera de un componente. Y el `AbortSignal.timeout` es importante en el celular: con una conexión mala, un `fetch` sin timeout puede quedar colgado para siempre.

TanStack Query

Para datos del servidor, **TanStack Query** (ex React Query) es el estándar, igual que en la web: caché, reintentos, estados de carga y error, paginación, y refresco automático. Se configura una vez en el layout raíz:

```
// app/_layout.tsx
const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 60_000, retry:
    2 } },  // los datos valen 1 minuto
})

<QueryClientProvider client={queryClient}>…
  </QueryClientProvider>
```

Y cada pantalla usa un hook. La app tiene dos: una lista paginada y el detalle:

app-expo/src/hooks/use-ofertas.ts

```
import { useInfiniteQuery, useQuery } from
  '@tanstack/react-query'
import { buscarOfertas, obtenerOferta } from
  '@/api/ofertas'
```

```
// TanStack Query guarda cada respuesta en caché por su
  queryKey: volver a una pantalla
// muestra los datos al instante y los actualiza por
  detrás.
export function useOfertas(texto: string) {
  return useInfiniteQuery({
    queryKey: ['ofertas', texto],
    queryFn: ({ pageParam }) => buscarOfertas(texto,
      pageParam),
    initialPageParam: 0,
    getNextPageParam: (ultimaPagina) =>
      ultimaPagina.siguiente,
  })
}

export function useOferta(id: number) {
  return useQuery({ queryKey: ['oferta', id], queryFn:
    () => obtenerOferta(id) })
}
```

`useInfiniteQuery` junta las páginas en `data.pages`, y `fetchNextPage` trae la siguiente usando lo que devuelve `getNextPageParam`. Conectado a `onEndReached` de la lista, es el scroll infinito de la pantalla de ofertas. La API de la app es falsa (devuelve datos con un retraso para simular la red):

app-expo/src/api/ofertas.ts

```
import { ofertas, type Oferta } from '@/demos/datos'

// Una API falsa que responde con demora, como un
  servidor. Para usar una de verdad,
// cada función hace un fetch a
  process.env.EXPO_PUBLIC_API_URL (ver el capítulo de
```

```
  datos).
const esperar = (ms: number) => new Promise((resolver)
  => setTimeout(resolver, ms))
const POR_PAGINA = 6

export type Pagina = { ofertas: Oferta[]; siguiente:
  number | null }

export async function buscarOfertas(texto: string,
  pagina: number): Promise<Pagina> {
  await esperar(500)
  const filtradas = ofertas.filter((o) =>
    `${o.titulo} ${o.empresa} ${o.stack.join('
      ')}`.toLowerCase().includes(texto.trim().toLowerCa
      se()),
  )
  const desde = pagina * POR_PAGINA
  return {
    ofertas: filtradas.slice(desde, desde + POR_PAGINA),
    siguiente: desde + POR_PAGINA < filtradas.length ?
      pagina + 1 : null,
  }
}

export async function obtenerOferta(id: number):
  Promise<Oferta> {
  await esperar(300)
  const oferta = ofertas.find((o) => o.id === id)
  if (!oferta) throw new Error('La oferta no existe')
  return oferta
}

export async function iniciarSesion(email: string,
  password: string) {
  await esperar(700)
  if (password !== 'expo1234') throw new Error('Email o
    contraseña incorrectos')
  return { token: `token-de-${email}`, nombre:
    email.split('@')[0] }
}
```

Mutaciones con actualización optimista

```
const queryClient = useQueryClient()

const postularme = useMutation({
  mutationFn: (id: number) =>
    api(`/ofertas/${id}/postulaciones`, { method: 'POST'
    }),
  // Antes de que responda el servidor: marcala como
    postulada en la caché
  onMutate: async (id) => {
    await queryClient.cancelQueries({ queryKey:
      ['oferta', id] })
    const anterior = queryClient.getQueryData<Oferta>
      (['oferta', id])
    queryClient.setQueryData(['oferta', id], {
      ...anterior, postulado: true })
    return { anterior }
  },
  onError: (_error, id, contexto) =>
    queryClient.setQueryData(['oferta', id],
    contexto?.anterior), // deshacer
  onSettled: (_data, _error, id) =>
    queryClient.invalidateQueries({ queryKey: ['oferta',
    id] }),
})
```

Lo que en el celular hay que configurar a mano

En la web, TanStack Query refresca los datos cuando volvés a la pestaña del navegador y pausa las consultas cuando se corta internet. En el celular no hay `window`: hay que contarle cuándo la app vuelve a

primer plano (con `AppState`) y cuándo cambia la conexión (con `expo-network`):

app-expo/src/ejemplos/conectividad.ts

```ts
import { focusManager, onlineManager } from
  '@tanstack/react-query'
import * as Network from 'expo-network'
import { useEffect } from 'react'
import { AppState, Platform, type AppStateStatus } from
  'react-native'

// En la web TanStack Query ya escucha "online" y
  "visibilitychange" del navegador.
// En el celular no hay window: se lo contamos nosotros.

// 1) Sin internet, las consultas se pausan y se
  reanudan solas al volver la conexión
onlineManager.setEventListener((setOnline) => {
  const suscripcion =
    Network.addNetworkStateListener((estado) => {
    setOnline(!!estado.isConnected)
  })
  return () => suscripcion.remove()
})

// 2) Al volver a la app (desde otra o desde el fondo),
  refresca lo que esté viejo
function alCambiarEstado(estado: AppStateStatus) {
  if (Platform.OS !== 'web')
    focusManager.setFocused(estado === 'active')
}

export function useRefrescarAlVolver() {
  useEffect(() => {
    const suscripcion =
      AppState.addEventListener('change',
      alCambiarEstado)
    return () => suscripcion.remove()
```

```
  }, [])
}

/** Para mostrar un aviso de "Sin conexión" arriba de la
  pantalla. */
export function useSinConexion() {
  const estado = Network.useNetworkState()
  return estado.isConnected === false
}
```

Y como las pantallas de un Stack no se desmontan, una consulta de una pantalla que quedó atrás sigue escuchando cambios. Para pausarla mientras no se ve:

```
import { useIsFocused } from 'expo-router'

const enPantalla = useIsFocused()
const { data } = useQuery({ queryKey: ['guardadas'],
  queryFn: traerGuardadas, subscribed: enPantalla })
```

Offline

Una app de celular se abre en el subte, en un avión, con una señal. Lo mínimo:

1. **Mostrar algo útil sin conexión**: con `useSinConexion()` (del ejemplo de arriba), un aviso arriba de la pantalla.
2. **Guardar la caché en el disco**: con `@tanstack/query-async-storage-persister` y `PersistQueryClientProvider`, la app abre

mostrando los últimos datos que vio, y los actualiza cuando hay red.

3. **Encolar lo que el usuario hace sin red**: las mutaciones de TanStack Query se pausan solas sin conexión y se reintentan al volver (con `networkMode` por defecto).

Para apps que tienen que funcionar 100 % offline (con sincronización), hay librerías específicas: **PowerSync**, **ElectricSQL**, **Legend State** con sync, o SQLite con tu propia sincronización.

Subir archivos

`FormData` funciona, con una particularidad de React Native: un archivo es un objeto con `uri`, `name` y `type`:

```
const datos = new FormData()
datos.append('cv', { uri: archivo.uri, name: 'cv.pdf',
  type: 'application/pdf' } as unknown as Blob)
await fetch(`${BASE_URL}/cv`, { method: 'POST', body:
  datos })
```

Otras opciones

Necesitás	Opción
Ya usás Redux Toolkit	**RTK Query** (lo viste en la guía de estado): mismo concepto que TanStack Query.

Necesitás	Opción
GraphQL	**Apollo Client** o **urql**.
Backend propio en TypeScript con tipos de punta a punta	**tRPC** u **oRPC**.
Backend listo (auth, base de datos, archivos)	**Supabase** o **Firebase** (lo vemos en librerías).
Tiempo real	`WebSocket` viene incluido; o los canales de Supabase / Firebase.

Capítulo 11

Estado y almacenamiento

Zustand con persistencia, AsyncStorage, MMKV, SecureStore y SQLite: qué va en cada lado.

Cuando cerrás una app del todo y la volvés a abrir, React arranca de cero: todo lo que estaba en `useState` se perdió. Lo que tiene que sobrevivir (la sesión, las preferencias, lo que guardaste) va al almacenamiento del teléfono. La pregunta es **dónde**, porque hay varios lugares y cada uno sirve para otra cosa.

Qué va en cada lado

Qué	Dónde	Por qué
Estado de una pantalla (un campo, un modal abierto)	`useState`	No tiene que sobrevivir.
Datos del servidor (ofertas, perfil)	**TanStack Query**	Ya tiene caché; el servidor es la fuente de verdad.

Qué	Dónde	Por qué
Estado global de la app (filtros, carrito)	**Zustand** (o Redux Toolkit, Jotai)	Lo comparten varias pantallas.
Preferencias y datos chicos que no son secretos	**AsyncStorage**, **MMKV** o `expo-sqlite/kv-store`	Clave-valor, como `localStorage`.
Tokens, contraseñas, claves	**expo-secure-store**	Cifrado por el sistema (Keychain / Keystore).
Muchos datos con estructura, búsquedas, offline	**expo-sqlite** (+ Drizzle)	Una base de datos de verdad.
Archivos (fotos, PDFs, descargas)	**expo-file-system**	El disco de la app.

> 🎤 **En una entrevista**
>
> "¿Dónde guardás el token de sesión?" En **SecureStore** (Keychain en iOS, Keystore en Android), nunca en AsyncStorage: AsyncStorage es texto plano en el disco de la app. Y nada de secretos de la app (claves de APIs privadas) en el bundle: van en el backend.

Zustand con persistencia

Zustand se usa igual que en la web (lo viste en la guía de estado). El middleware `persist` guarda el estado en el almacenamiento que le digas, y lo vuelve a cargar

al abrir la app. Las ofertas guardadas van a AsyncStorage:

app-expo/src/stores/guardadas.ts

```
import AsyncStorage from '@react-native-async-
  storage/async-storage'
import { create } from 'zustand'
import { createJSONStorage, persist } from
  'zustand/middleware'
import type { Oferta } from '@/demos/datos'

type Guardadas = {
  ofertas: Oferta[]
  alternar: (oferta: Oferta) => void
}

// Las ofertas guardadas no son secretas: van a
  AsyncStorage (un almacenamiento clave-valor
// simple, el equivalente del localStorage).
export const useGuardadas = create<Guardadas>()(
  persist(
    (set) => ({
      ofertas: [],
      alternar: (oferta) =>
        set((estado) => ({
          ofertas: estado.ofertas.some((o) => o.id ===
            oferta.id)
            ? estado.ofertas.filter((o) => o.id !==
              oferta.id)
            : [oferta, ...estado.ofertas],
        })),
    }),
    { name: 'guardadas', storage: createJSONStorage(()
      => AsyncStorage) },
  ),
)
```

```
export const useEstaGuardada = (id: number) =>
  useGuardadas((s) => s.ofertas.some((o) => o.id ===
  id))
```

AsyncStorage es el `localStorage` de React Native: clave-valor, solo strings, pero **asincrónico** (todo devuelve promesas). Para unos cientos de ofertas está perfecto; para miles de registros con búsquedas, SQLite.

La sesión, cifrada

La sesión usa el mismo `persist`, pero con un adaptador sobre **expo-secure-store**:

app-expo/src/lib/almacenamiento-seguro.ts

```
import * as SecureStore from 'expo-secure-store'
import { Platform } from 'react-native'
import type { StateStorage } from 'zustand/middleware'

// expo-secure-store guarda en el Keychain (iOS) o el
  Keystore (Android): el lugar para
// tokens y secretos. En la web no existe, así que ahí
  cae al localStorage.
export const almacenamientoSeguro: StateStorage =
  Platform.OS === 'web'
    ? {
        getItem: (clave) => localStorage.getItem(clave),
        setItem: (clave, valor) =>
          localStorage.setItem(clave, valor),
        removeItem: (clave) =>
          localStorage.removeItem(clave),
      }
```

```
    : {
        getItem: (clave) =>
          SecureStore.getItemAsync(clave),
        setItem: (clave, valor) =>
          SecureStore.setItemAsync(clave, valor),
        removeItem: (clave) =>
          SecureStore.deleteItemAsync(clave),
      }
```

app-expo/src/stores/sesion.ts

```
import { create } from 'zustand'
import { createJSONStorage, persist } from
  'zustand/middleware'
import * as api from '@/api/ofertas'
import { almacenamientoSeguro } from
  '@/lib/almacenamiento-seguro'

type Sesion = {
  token: string | null
  nombre: string | null
  iniciarSesion: (email: string, password: string) =>
    Promise<void>
  cerrarSesion: () => void
}

// La sesión se guarda cifrada en el teléfono: al abrir
  la app de nuevo, seguís logueado.
export const useSesion = create<Sesion>()(
  persist(
    (set) => ({
      token: null,
      nombre: null,
      iniciarSesion: async (email, password) => {
        const { token, nombre } = await
          api.iniciarSesion(email, password)
        set({ token, nombre })
      },
```

```
      cerrarSesion: () => set({ token: null, nombre:
        null }),
    }),
    {
      name: 'sesion',
      storage: createJSONStorage(() =>
        almacenamientoSeguro),
      partialize: ({ token, nombre }) => ({ token,
        nombre }), // solo los datos, no las funciones
    },
  ),
)
```

- `partialize` elige qué guardar: los datos, no las funciones.
- SecureStore está pensado para valores **chicos** (un token, un id), no para un JSON grande.
- En la web no hay Keychain: el adaptador cae a `localStorage` (con Platform, como vimos en proyecto).

Esperar a que cargue

Como el almacenamiento es asincrónico, durante unos milisegundos al abrir la app Zustand todavía no leyó la sesión: `token` es `null` aunque el usuario esté logueado. Si mostrás el login en ese momento, parpadea. La solución es esperar a que termine de **hidratar** con el splash todavía visible:

```
// app/_layout.tsx
SplashScreen.preventAutoHideAsync()   // el splash no se
  va solo
```

```
export default function RootLayout() {
  const hidratada =
    useSyncExternalStore(useSesion.persist.onFinishHydra
    tion, useSesion.persist.hasHydrated)

  useEffect(() => {
    if (hidratada) SplashScreen.hideAsync()
  }, [hidratada])

  if (!hidratada) return null   // mientras tanto se
    sigue viendo el splash
  // ...
}
```

MMKV: cuando AsyncStorage es lento

react-native-mmkv es un almacenamiento clave-valor hecho en C++ por WeChat: **sincrónico** (sin `await`) y unas 30 veces más rápido que AsyncStorage. Es muy usado en apps grandes, sobre todo para leer preferencias al arrancar sin esperar. Necesita un development build.

```
import { createMMKV } from 'react-native-mmkv'

export const almacen = createMMKV()

almacen.set('tema', 'oscuro')
const tema = almacen.getString('tema') // sincrónico
almacen.remove('tema')
```

```
// Con Zustand: getItem/setItem/removeItem que llaman a
  almacen.getString/set/remove
```

Una alternativa que **sí anda en Expo Go**: `expo-sqlite/kv-store`, con la misma API que AsyncStorage más versiones sincrónicas (`Storage.getItemSync`). Y si una librería espera `localStorage`, `import 'expo-sqlite/localStorage/install'` lo crea sobre SQLite.

SQLite

Cuando tenés muchos datos con estructura (miles de registros, filtros, ordenar, relaciones), o la app tiene que funcionar offline, se usa **SQLite**: una base de datos de verdad, en un archivo dentro de la app. `expo-sqlite` viene en el Expo SDK y anda en Expo Go:

app-expo/src/ejemplos/BaseDeDatos.tsx

```
import { useMutation, useQuery, useQueryClient } from
  '@tanstack/react-query'
import { SQLiteProvider, useSQLiteContext, type
  SQLiteDatabase } from 'expo-sqlite'
import { useState } from 'react'
import { Button, FlatList, Text, TextInput, View } from
  'react-native'

type Nota = { id: number; ofertaId: string; texto:
  string; creadaEn: string }

// Migraciones con PRAGMA user_version: SQLite guarda un
  número en el archivo
// y cada versión nueva de la app corre solo los pasos
  que le faltan.
```

```ts
async function migrar(db: SQLiteDatabase) {
  const VERSION_ACTUAL = 2
  const fila = await db.getFirstAsync<{ user_version:
    number }>('PRAGMA user_version')
  let version = fila?.user_version ?? 0
  if (version >= VERSION_ACTUAL) return

  if (version === 0) {
    await db.execAsync(`
      PRAGMA journal_mode = 'wal';
      CREATE TABLE notas (
        id INTEGER PRIMARY KEY NOT NULL,
        oferta_id TEXT NOT NULL,
        texto TEXT NOT NULL
      );
    `)
    version = 1
  }
  if (version === 1) {
    await db.execAsync(`
      ALTER TABLE notas ADD COLUMN creada_en TEXT NOT
        NULL DEFAULT (datetime('now'));
      CREATE INDEX idx_notas_oferta ON notas
        (oferta_id);
    `)
    version = 2
  }
  await db.execAsync(`PRAGMA user_version =
    ${VERSION_ACTUAL}`)
}

// El provider abre la base una vez y corre las
  migraciones antes de mostrar a los hijos
export function ConBaseDeDatos({ children }: { children:
  React.ReactNode }) {
  return (
    <SQLiteProvider databaseName="busqueda.db" onInit=
      {migrar}>
      {children}
    </SQLiteProvider>
  )
```

```
}

export function NotasDeOferta({ ofertaId }: { ofertaId:
  string }) {
  const db = useSQLiteContext()
  const queryClient = useQueryClient()
  const [texto, setTexto] = useState('')
  const clave = ['notas', ofertaId]

  // Leer de SQLite también es asincrónico: TanStack
    Query maneja la carga y la caché igual que con una
    API
  const { data: notas = [] } = useQuery({
    queryKey: clave,
    // Tagged template: los ${valores} viajan como
      parámetros, nunca pegados al SQL
    queryFn: async () => await db.sql<Nota>`
      SELECT id, oferta_id AS ofertaId, texto, creada_en
        AS creadaEn
      FROM notas WHERE oferta_id = ${ofertaId} ORDER BY
        id DESC`,
  })

  const agregar = useMutation({
    // La API "clásica" con ? también es segura
    mutationFn: (nuevo: string) => db.runAsync('INSERT
      INTO notas (oferta_id, texto) VALUES (?, ?)',
      ofertaId, nuevo),
    onSuccess: () => {
      setTexto('')
      return queryClient.invalidateQueries({ queryKey:
        clave }) // vuelve a leer la lista
    },
  })

  const borrarTodas = useMutation({
    // Varias escrituras juntas: todas o ninguna
    mutationFn: () =>
      db.withTransactionAsync(async () => {
        await db.runAsync('DELETE FROM notas WHERE
          oferta_id = ?', ofertaId)
```

```
      }),
    onSuccess: () => queryClient.invalidateQueries({
      queryKey: clave }),
  })

  return (
    <View style={{ gap: 8, padding: 16 }}>
      <TextInput value={texto} onChangeText={setTexto}
        placeholder="Ej: pedir rango salarial" />
      <Button title="Guardar nota" disabled=
        {!texto.trim()} onPress={() =>
        agregar.mutate(texto.trim())} />
      <FlatList
        data={notas}
        keyExtractor={(n) => String(n.id)}
        renderItem={({ item }) => <Text>• {item.texto}
          </Text>}
      />
      {notas.length > 0 && <Button title="Borrar todas"
        color="#dc2626" onPress={() =>
        borrarTodas.mutate()} />}
    </View>
  )
}
```

- `SQLiteProvider` abre la base una vez y corre `onInit` (las migraciones) antes de mostrar la app.
- `useSQLiteContext()` da la base en cualquier componente.
- `db.sql` es un *tagged template*: los `${valores}` se mandan como parámetros, así que no hay inyección SQL. La API clásica (`runAsync` , `getAllAsync` con `?`) es igual de segura.
- Leer y escribir es asincrónico, así que se usa **TanStack Query** igual que con una API: `useQuery` para leer, `useMutation` para escribir, e

`invalidateQueries` para que la lista se vuelva a leer.

- Las **migraciones con** `PRAGMA user_version`: la app se actualiza en teléfonos que tienen la base vieja, así que cada cambio de esquema es un paso numerado.

Si venís de la guía de MySQL, el SQL es prácticamente el mismo.

> 💡 **Drizzle ORM**
>
> En proyectos reales se suele usar **Drizzle** sobre expo-sqlite: definís las tablas en TypeScript, las consultas quedan tipadas, `drizzle-kit` genera las migraciones, y `useLiveQuery` re-renderiza el componente cuando cambian los datos.

Archivos

`expo-file-system` tiene una API con clases: `File`, `Directory` y `Paths`.

```
import { Directory, File, Paths } from 'expo-file-
  system'

const archivo = new File(Paths.document,
  'postulaciones.json')   // Paths.cache se puede borrar
  solo
await archivo.write(JSON.stringify(postulaciones))
  // desde el SDK 58, write() es async
const texto = await archivo.text()
```

```
const carpeta = new Directory(Paths.document, 'cvs')
if (!carpeta.exists) carpeta.create()

// Descargar un archivo
const pdf = await
  File.downloadFileAsync('https://ejemplo.com/cv.pdf',
  carpeta)
```

`Paths.document` es permanente (y entra en el backup de iCloud); `Paths.cache` es para lo que se puede volver a descargar, y el sistema lo puede borrar si falta espacio.

Capítulo 12

Login y sesión

El flujo de sesión con rutas protegidas, OAuth, Sign in with Apple, biometría y proveedores.

El login en una app tiene las mismas piezas que en la web (un formulario, un token, rutas protegidas), más tres cosas propias del celular: el token se guarda **cifrado**, los logins con Google o Apple abren **una ventana del sistema** y vuelven a la app, y Apple tiene **reglas** sobre qué logins tenés que ofrecer.

El flujo de la app de la guía

1. Al abrir, el splash queda visible mientras Zustand lee la sesión de SecureStore (almacenamiento).
2. El layout raíz usa `Stack.Protected`: con token, existen las pestañas; sin token, solo existe `/login`.
3. El login valida con Zod y llama a `iniciarSesion`, que pide el token y lo guarda en el store.
4. Al cambiar el token, Expo Router redirige solo: no hay `router.replace` en ningún lado.

5. Cerrar sesión borra el token y vuelve al login, también solo.

```
// app/_layout.tsx
<Stack>
  <Stack.Protected guard={logueado}>
    <Stack.Screen name="(tabs)" options={{ headerShown:
      false }} />
    <Stack.Screen name="oferta/[id]" />
  </Stack.Protected>
  <Stack.Protected guard={!logueado}>
    <Stack.Screen name="login" options={{ headerShown:
      false }} />
  </Stack.Protected>
</Stack>
```

Si un deep link apunta a `/oferta/5` sin sesión, el usuario ve el login. Y si la API responde 401 (token vencido), el cliente de API cierra la sesión y el guard lo manda al login.

> ⚠️ **Proteger la ruta no protege los datos**
>
> `Stack.Protected` decide qué pantallas se ven, nada más. La seguridad real está en el backend: cada endpoint tiene que validar el token. Una app se puede descompilar y modificar.

Tokens: lo que conviene saber

- Guardá el **access token** (y el refresh token, si hay) en **SecureStore**, nunca en AsyncStorage.

- **Nunca guardes la contraseña** del usuario en el teléfono.
- Si el access token dura poco (15 minutos, lo normal), el cliente de API pide uno nuevo con el refresh token cuando recibe un 401, y reintenta. Si el refresh también falla, cierra la sesión.
- En el celular no hay cookies de sesión cómodas como en la web: lo normal es mandar `Authorization: Bearer <token>` en cada pedido.

Login con Google, GitHub y otros (OAuth)

Un login con un proveedor abre una ventana **del sistema** (no un WebView propio: los proveedores lo bloquean) donde el usuario entra, y el proveedor redirige de vuelta a tu app con un código. `expo-auth-session` arma todo el flujo con PKCE:

```
import * as AuthSession from 'expo-auth-session'
import * as WebBrowser from 'expo-web-browser'

WebBrowser.maybeCompleteAuthSession() // cierra la
  ventana del navegador al volver (en la web)

export function BotonGithub() {
  const [pedido, respuesta, abrir] =
    AuthSession.useAuthRequest(
    {
      clientId:
        process.env.EXPO_PUBLIC_GITHUB_CLIENT_ID!,
      scopes: ['read:user', 'user:email'],
```

```
      redirectUri: AuthSession.makeRedirectUri({ scheme:
        'busquedarn' }), // busquedarn://
    },
    {
      authorizationEndpoint:
        'https://github.com/login/oauth/authorize',
      tokenEndpoint:
        'https://github.com/login/oauth/access_token',
    },
  )

  useEffect(() => {
    if (respuesta?.type === 'success') {
      // Mandá el código y el verificador a TU backend:
        él lo cambia por un token (con el secreto)
      enviarAlBackend(respuesta.params.code,
        pedido?.codeVerifier)
    }
  }, [respuesta])

  return <Button title="Entrar con GitHub" disabled=
    {!pedido} onPress={() => abrir()} />
}
```

Para **Google** en particular, se usa la librería nativa `@react-native-google-signin/google-signin`: muestra la lista de cuentas del teléfono en lugar de pedir usuario y contraseña. Necesita un development build.

Sign in with Apple: obligatorio si ofrecés otros

La regla 4.8 de Apple dice: si tu app ofrece login con un servicio de terceros (Google, Facebook, GitHub), **tiene que ofrecer también una opción equivalente que**

cuide la privacidad, y Sign in with Apple la cumple. Es uno de los rechazos más comunes. No aplica si solo tenés email y contraseña propios, o si la app es de una empresa para sus empleados.

app-expo/src/ejemplos/BotonApple.tsx

```
import * as AppleAuthentication from 'expo-apple-
  authentication'
import { Platform } from 'react-native'

type Props = { alEntrar: (identityToken: string, nombre:
  string | null) => void }

/**
 * Si tu app ofrece "Entrar con Google" (u otra red
   social) en iOS, Apple exige
 * ofrecer también una opción equivalente que cuide la
   privacidad: Sign in with Apple.
 */
export function BotonApple({ alEntrar }: Props) {
  if (Platform.OS !== 'ios') return null

  async function entrar() {
    try {
      const credencial = await
        AppleAuthentication.signInAsync({
        requestedScopes: [

            AppleAuthentication.AppleAuthenticationScope
            .FULL_NAME,

            AppleAuthentication.AppleAuthenticationScope
            .EMAIL,
        ],
      })
      // Apple manda el nombre SOLO la primera vez:
        guardalo en tu backend en ese momento
```

```
      const nombre = credencial.fullName?.givenName ??
        null
      if (credencial.identityToken)
        alEntrar(credencial.identityToken, nombre)
    } catch (error) {
      // Cancelar no es un error para mostrar
      if ((error as { code?: string }).code ===
        'ERR_REQUEST_CANCELED') return
      throw error
    }
  }

  return (
    <AppleAuthentication.AppleAuthenticationButton
      buttonType=
        {AppleAuthentication.AppleAuthenticationButtonTy
        pe.SIGN_IN}
      buttonStyle=
        {AppleAuthentication.AppleAuthenticationButtonSt
        yle.BLACK}
      cornerRadius={12}
      style={{ height: 48 }}
      onPress={entrar}
    />
  )
}
```

- El botón tiene que ser el de Apple (`AppleAuthenticationButton`), con su diseño.
- Apple te da el nombre y el email **solo la primera vez**. Si no los guardás en ese momento, no los vas a volver a recibir.
- El usuario puede elegir "ocultar mi email": recibís una dirección de reenvío `@privaterelay.appleid.com`.

- Hace falta `"usesAppleSignIn": true` en `ios` de `app.json` (la app de la guía lo tiene) y activar la capacidad en tu cuenta de Apple (EAS lo hace al compilar).

Biometría: Face ID y huella

La biometría **no reemplaza** el login: no hay "usuario de Face ID" en tu servidor. Sirve para **desbloquear** algo que ya está en el teléfono: abrir la app sin volver a escribir la contraseña, o confirmar una acción sensible.

app-expo/src/ejemplos/biometria.ts

```
import * as LocalAuthentication from 'expo-local-
  authentication'

/**
 * Pide Face ID / huella antes de mostrar algo sensible.
   Si el celular no tiene
 * biometría configurada, deja pasar con el PIN del
   sistema (o devolvé false si preferís).
 * En iOS hace falta NSFaceIDUsageDescription: lo agrega
   el config plugin de la librería.
 */
export async function confirmarIdentidad(motivo:
  string): Promise<boolean> {
  const tieneHardware = await
    LocalAuthentication.hasHardwareAsync()
  const configurada = await
    LocalAuthentication.isEnrolledAsync()
  if (!tieneHardware || !configurada) return true

  const resultado = await
    LocalAuthentication.authenticateAsync({
    promptMessage: motivo,
```

```
    cancelLabel: 'Cancelar',
    // false = si falla la cara o la huella, ofrece el
      PIN o patrón del celular
    disableDeviceFallback: false,
  })
  return resultado.success
}
```

Un patrón común: después del primer login, preguntar "¿Querés usar Face ID para entrar?"; si dice que sí, al abrir la app pedís la biometría antes de mostrar el contenido.

Borrar la cuenta: obligatorio

Si tu app permite **crear** una cuenta, Apple y Google exigen que también permita **borrarla desde la app** (no alcanza con "mandanos un email"). Google además pide un link web para pedir el borrado sin instalar la app. Lo vemos en requisitos.

Proveedores de autenticación

Hacer el login a mano (registro, verificación de email, recuperar contraseña, OAuth, 2FA) es mucho trabajo. Lo más usado con Expo:

Proveedor	Qué es
Clerk (`@clerk/expo`)	Auth como servicio con componentes listos, OAuth, passkeys, organizaciones. El más simple de integrar.

Proveedor	Qué es
Supabase Auth	Viene con Supabase (Postgres + auth + archivos). Muy usado en apps nuevas.
Firebase Auth	El clásico de Google. `@react-native-firebase/auth` necesita development build.
Better Auth	Open source, en tu propio backend de Node, con plugin para Expo.
Auth0	Enterprise, `react-native-auth0`.

Si ya tenés tu backend de Node con JWT (como en la guía de Node), el flujo de este capítulo funciona tal cual contra tu API.

Capítulo 13

Cámara, notificaciones y más

Permisos, cámara y fotos, ubicación, notificaciones push, archivos, compartir y config plugins.

La razón para hacer una app y no una página web suele ser esto: la cámara, las notificaciones, la ubicación, la biometría. El Expo SDK tiene una librería para cada cosa, con la misma API en iOS y Android, y casi todas andan en Expo Go.

Permisos: cómo y cuándo pedirlos

Para la cámara, la ubicación, las notificaciones, los contactos o el micrófono, el sistema le pregunta al usuario. Y en iOS, si dice que no, **no podés volver a preguntar**: solo queda mandarlo a Ajustes. Por eso el *cuándo* importa tanto como el *cómo*.

1. **Pedí en el momento en que se usa**, no al abrir la app. "Sacar foto" pide la cámara; la pantalla de bienvenida no pide nada.

2. **Explicá antes** para qué lo querés, si no es obvio ("Activá las notificaciones para enterarte cuando te respondan").
3. **Manejá el "no"**: la app tiene que seguir funcionando, y ofrecer ir a Ajustes con `Linking.openSettings()`.
4. **Pedí solo lo que usás.** Cada permiso de más es un motivo de rechazo en la revisión y una razón para que el usuario desconfíe.

Cada librería tiene un hook que devuelve el estado y la función para pedir:

```
const [permiso, pedirPermiso] = useCameraPermissions()
// permiso: null (cargando) | { granted, canAskAgain,
  status: 'granted' | 'denied' | 'undetermined' }
```

Los textos de los permisos

En iOS, cada permiso muestra un texto tuyo explicando para qué lo usás, y **Apple rechaza la app si el texto es vago** ("La app necesita la cámara"). Tiene que decir para qué, en concreto. Se configuran en `app.json` con el config plugin de cada librería:

```
"plugins": [
  ["expo-camera", {
    "cameraPermission": "Usamos la cámara para que
      saques tu foto de perfil y para leer códigos QR de
      ofertas.",
    "microphonePermission": false,
    "recordAudioAndroid": false
```

```
  }],
  ["expo-location", {
    "locationWhenInUsePermission": "Usamos tu ubicación
      para mostrarte ofertas de tu ciudad.",
    "locationAlwaysPermission": false
  }]
]
```

Poner `false` **saca** ese permiso: la app de la guía no graba audio, así que bloquea el micrófono que `expo-camera` agrega por defecto. Revisá siempre qué permisos agregan tus librerías: con `npx expo config --type introspect` ves el `Info.plist` y el `AndroidManifest.xml` que se van a generar.

Fotos: galería y cámara

`expo-image-picker` abre el selector del sistema. Para la **galería no hace falta permiso**: el selector moderno de iOS y Android le da a la app solo las fotos que el usuario eligió. La **cámara sí** lo necesita:

app-expo/src/app/(tabs)/perfil.tsx

```
import * as Haptics from 'expo-haptics'
import { Image } from 'expo-image'
import * as ImagePicker from 'expo-image-picker'
import { useState } from 'react'
import { Linking, Pressable, StyleSheet, Text, View }
  from 'react-native'
import { SafeAreaView } from 'react-native-safe-area-
  context'
import { Titulo } from '@/components/Titulo'
```

```
import { colores, espacio } from '@/demos/tema'
import { useSesion } from '@/stores/sesion'

export default function Perfil() {
  const nombre = useSesion((s) => s.nombre)
  const cerrarSesion = useSesion((s) => s.cerrarSesion)
  const [foto, setFoto] = useState<string | null>(null)
  const [aviso, setAviso] = useState<string | null>
    (null)

  async function elegirFoto() {
    // La galería no necesita permiso: el selector del
      sistema solo le da a la app la foto elegida
    const resultado = await
      ImagePicker.launchImageLibraryAsync({
      mediaTypes: ['images'],
      allowsEditing: true,
      aspect: [1, 1],
      quality: 0.8,
    })
    if (!resultado.canceled)
      setFoto(resultado.assets[0].uri)
  }

  async function sacarFoto() {
    // La cámara sí: se pide en el momento de usarla, y
      si lo niega, se explica cómo activarlo
    const permiso = await
      ImagePicker.requestCameraPermissionsAsync()
    if (!permiso.granted) {
      setAviso('Sin permiso para la cámara. Podés
        activarlo en Ajustes.')
      return
    }
    const resultado = await
      ImagePicker.launchCameraAsync({ allowsEditing:
      true, aspect: [1, 1], quality: 0.8 })
    if (!resultado.canceled)
      setFoto(resultado.assets[0].uri)
  }

  return (
```

```
    <SafeAreaView style={styles.pantalla} edges=
      {['top']}>
      <Titulo>Perfil</Titulo>
      <View style={styles.contenido}>
        <View style={styles.cabecera}>
          {foto ? (
            <Image source={foto} style={styles.foto}
              contentFit="cover" transition={200} />
          ) : (
            <View style={[styles.foto, styles.sinFoto]}>
              <Text style={styles.inicial}>{nombre?.
                [0]?.toUpperCase() ?? '?'}</Text>
            </View>
          )}
          <Text style={styles.nombre}>{nombre}</Text>
        </View>

        <Boton texto="Elegir foto de la galería"
          onPress={elegirFoto} />
        <Boton texto="Sacar una foto" onPress=
          {sacarFoto} />
        {aviso && (
          <Pressable onPress={() =>
            Linking.openSettings()}>
            <Text style={styles.aviso}>{aviso} Tocá acá
              para abrirlos.</Text>
          </Pressable>
        )}
        <Boton texto="Probar la vibración" onPress={()
          =>
          Haptics.notificationAsync(Haptics.Notification
          FeedbackType.Success)} />
        <Boton texto="Cerrar sesión" onPress=
          {cerrarSesion} peligro />
      </View>
    </SafeAreaView>
  )
}

function Boton({ texto, onPress, peligro }: { texto:
  string; onPress: () => void; peligro?: boolean }) {
  return (
```

```
    <Pressable style={({ pressed }) => [styles.boton,
      pressed && styles.presionado]} onPress={onPress}>
      <Text style={[styles.botonTexto, peligro && {
        color: colores.error }]}>{texto}</Text>
    </Pressable>
  )
}

const styles = StyleSheet.create({
  pantalla: { flex: 1, backgroundColor: colores.fondo },
  contenido: { padding: espacio.l, gap: espacio.s },
  cabecera: { alignItems: 'center', gap: espacio.s,
    marginBottom: espacio.l },
  foto: { width: 96, height: 96, borderRadius: 48 },
  sinFoto: { backgroundColor: colores.primarioSuave,
    alignItems: 'center', justifyContent: 'center' },
  inicial: { fontSize: 40, fontWeight: '700', color:
    colores.primario },
  nombre: { fontSize: 20, fontWeight: '700', color:
    colores.texto },
  boton: { backgroundColor: colores.tarjeta, padding:
    espacio.l, borderRadius: 12 },
  presionado: { opacity: 0.7 },
  botonTexto: { fontSize: 16, color: colores.primario,
    fontWeight: '500' },
  aviso: { color: colores.error, fontSize: 14,
    paddingHorizontal: espacio.s },
})
```

`allowsEditing` con `aspect: [1, 1]` deja recortar en cuadrado, ideal para una foto de perfil. `quality: 0.8` comprime un poco: una foto de un iPhone puede pesar 5 MB.

Cámara propia y códigos QR

Para mostrar la cámara dentro de tu pantalla (un escáner, un filtro), `expo-camera` tiene `CameraView`:

app-expo/src/ejemplos/Escaner.tsx

```
import { CameraView, useCameraPermissions, type
  BarcodeScanningResult } from 'expo-camera'
import { useRef } from 'react'
import { Button, StyleSheet, Text, View } from 'react-
  native'

/** Escanea un QR (por ejemplo, el link a una oferta en
  un evento de empleo). */
export function EscanerQr({ alLeer }: { alLeer: (texto:
  string) => void }) {
  const [permiso, pedirPermiso] = useCameraPermissions()
  // onBarcodeScanned se dispara muchas veces por
    segundo: nos quedamos con la primera
  const leido = useRef(false)

  if (!permiso) return null // todavía cargando el
    estado del permiso
  if (!permiso.granted) {
    return (
      <View style={styles.centro}>
        <Text>Necesitamos la cámara para leer el código
          QR.</Text>
        <Button title="Dar permiso" onPress=
          {pedirPermiso} />
      </View>
    )
  }

  function alEscanear({ data }: BarcodeScanningResult) {
    if (leido.current) return
    leido.current = true
```

```
    alLeer(data)
  }

  return (
    <CameraView
      style={StyleSheet.absoluteFill}
      facing="back"
      barcodeScannerSettings={{ barcodeTypes: ['qr'] }}
      onBarcodeScanned={alEscanear}
    />
  )
}

const styles = StyleSheet.create({
  centro: { flex: 1, alignItems: 'center',
    justifyContent: 'center', gap: 12, padding: 24 },
})
```

Para cosas más avanzadas (procesar cada cuadro, reconocer texto o caras en tiempo real) se usa **react-native-vision-camera**, que necesita un development build.

Ubicación

app-expo/src/ejemplos/Ubicacion.tsx

```
import * as Location from 'expo-location'
import { useState } from 'react'
import { Button, Linking, Text, View } from 'react-
  native'

export function CiudadActual() {
```

```
// El hook trae el estado del permiso y la función
  para pedirlo
const [permiso, pedirPermiso] =
  Location.useForegroundPermissions()
const [ciudad, setCiudad] = useState<string | null>
  (null)

async function buscar() {
  let respuesta = permiso
  if (!respuesta?.granted) respuesta = await
    pedirPermiso()
  if (!respuesta.granted) return

  // Balanced alcanza para saber la ciudad y gasta
    menos batería que High
  const { coords } = await
    Location.getCurrentPositionAsync({ accuracy:
    Location.Accuracy.Balanced })
  const [direccion] = await
    Location.reverseGeocodeAsync(coords)
  setCiudad(direccion?.city ?? direccion?.region ??
    'Desconocida')
}

// Si dijo "no" y ya no se puede volver a preguntar,
  solo queda mandarlo a Ajustes
if (permiso && !permiso.granted &&
  !permiso.canAskAgain) {
  return (
    <View style={{ padding: 16, gap: 8 }}>
      <Text>Activá la ubicación en Ajustes para ver
        ofertas cerca tuyo.</Text>
      <Button title="Abrir Ajustes" onPress={() =>
        Linking.openSettings()} />
    </View>
  )
}

return (
  <View style={{ padding: 16, gap: 8 }}>
    <Button title="Usar mi ubicación" onPress={buscar}
      />
```

```
      {ciudad && <Text>Estás en {ciudad}</Text>}
    </View>
  )
}
```

- `Accuracy.Balanced` alcanza para saber la ciudad y gasta mucho menos batería que `High` (GPS).
- La ubicación **en segundo plano** (cuando la app está cerrada) es otro permiso, mucho más difícil de que te aprueben: Apple y Google piden justificarlo y Google exige un video mostrando la función. Si no lo necesitás de verdad, no lo pidas.

Notificaciones

Hay dos tipos:

- **Locales**: las programa la app misma ("recordatorio en 3 días"). No necesitan servidor.
- **Push (remotas)**: las manda tu servidor. El teléfono se registra, obtiene un **token**, se lo manda a tu backend, y el backend lo usa para mandar la notificación.

`app-expo/src/ejemplos/notificaciones.ts`

```
import Constants from 'expo-constants'
import * as Device from 'expo-device'
import * as Notifications from 'expo-notifications'
import { router, type Href } from 'expo-router'
import { useEffect } from 'react'
```

```ts
import { Platform } from 'react-native'

// Desde SDK 58 las notificaciones se muestran aunque la
  app esté abierta.
// El handler sirve para cambiar eso: acá, sin sonido
  mientras la usás.
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowBanner: true,
    shouldShowList: true,
    shouldPlaySound: false,
    shouldSetBadge: false,
  }),
})

/** Pide permiso y devuelve el token para mandarle push
  a este celular (o null). */
export async function registrarParaPush():
  Promise<string | null> {
  // En Android 13+ el canal tiene que existir antes de
    pedir permiso
  if (Platform.OS === 'android') {
    await
      Notifications.setNotificationChannelAsync('default
      ', {
      name: 'General',
      importance:
        Notifications.AndroidImportance.DEFAULT,
    })
  }
  // Los simuladores y emuladores no reciben push
    remotas
  if (!Device.isDevice) return null

  let { status } = await
    Notifications.getPermissionsAsync()
  if (status !== 'granted') {
    ;({ status } = await
      Notifications.requestPermissionsAsync())
  }
  if (status !== 'granted') return null
```

```
  // El projectId lo agrega `eas init` en app.json
    (extra.eas.projectId)
  const projectId =
    Constants.expoConfig?.extra?.eas?.projectId ??
    Constants.easConfig?.projectId
  const { data } = await
    Notifications.getExpoPushTokenAsync({ projectId })
  return data // "ExponentPushToken[xxxx]": se lo mandás
    a tu backend
}

/** Una notificación local: no necesita servidor ni
  token. */
export function recordarPostulacion(empresa: string,
  ofertaId: string) {
  return Notifications.scheduleNotificationAsync({
    content: {
      title: '¿Te respondieron?',
      body: `Pasaron 3 días desde que te postulaste a
        ${empresa}.`,
      data: { url: `/oferta/${ofertaId}` },
    },
    trigger: {
      type:
        Notifications.SchedulableTriggerInputTypes.TIME_
        INTERVAL,
      seconds: 60 * 60 * 24 * 3,
    },
  })
}

/** Al tocar una notificación (con la app abierta o
  cerrada), navega a la URL que trae. */
export function useAbrirDesdeNotificacion() {
  const respuesta =
    Notifications.useLastNotificationResponse()

  useEffect(() => {
    const url =
      respuesta?.notification.request.content.data?.url
    if (typeof url === 'string' &&
      respuesta?.actionIdentifier ===
```

```
      Notifications.DEFAULT_ACTION_IDENTIFIER) {
      router.push(url as Href)
      void
        Notifications.clearLastNotificationResponseAsync
        ()
    }
  }, [respuesta])
}
```

El token que devuelve `getExpoPushTokenAsync` es del **servicio de push de Expo**: tu backend le manda la notificación a Expo, y Expo se la pasa a Apple (APNs) o Google (FCM). Es un `POST` simple:

```
// En tu backend de Node
await fetch('https://exp.host/--/api/v2/push/send', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    to: 'ExponentPushToken[xxxxxxxxxxxx]',
    title: 'Nueva oferta',
    body: 'Expo Developer en Yerba Labs: hasta US$
      3600',
    data: { url: '/oferta/5' },  // al tocarla, la app
      abre esa pantalla
  }),
})
```

Para que funcione, EAS necesita las credenciales de Apple (una clave APNs, que crea solo al compilar) y de Firebase (subís la clave de FCM a tu proyecto de EAS). Si preferís no depender de Expo, también podés usar el token nativo (`getDevicePushTokenAsync`) con FCM o APNs directo, o servicios como **OneSignal**.

> **⚠️ Push en Expo Go**
>
> Las notificaciones push **no funcionan en Expo Go en Android**: necesitás un development build. Las locales sí. Y en simuladores no llegan push remotas (en el simulador de iOS se pueden simular arrastrando un archivo `.apns`).

Compartir, abrir links y otras utilidades

```
import { Linking, Share } from 'react-native'
import * as Clipboard from 'expo-clipboard'
import * as WebBrowser from 'expo-web-browser'

await Share.share({ message: 'Mirá esta oferta:
  https://ejemplo.com/oferta/5' })  // la hoja de
  compartir del sistema
await Linking.openURL('mailto:rrhh@empresa.com')
  // también tel:, sms:, https:, y otras apps
await Linking.openSettings()
  // los ajustes de tu app
await WebBrowser.openBrowserAsync('https://ejemplo.com')
  // un navegador dentro de la app (Safari View / Custom
  Tabs)
await Clipboard.setStringAsync('ABC123')
```

Para links a páginas externas (términos, ayuda), `WebBrowser.openBrowserAsync` es mejor que sacar al usuario a Safari o Chrome.

Más librerías del Expo SDK

Necesitás	Librería
Video y audio	`expo-video`, `expo-audio`
Mapas	`expo-maps` (Apple Maps / Google Maps nativos) o `react-native-maps`
Contactos, calendario	`expo-contacts`, `expo-calendar`
Sensores (acelerómetro, giroscopio, podómetro)	`expo-sensors`
Elegir un archivo (un PDF)	`expo-document-picker`
Info del teléfono y la app	`expo-device`, `expo-application`, `expo-constants`
Idioma y región del usuario	`expo-localization`
Mantener la pantalla encendida	`expo-keep-awake`
Tareas en segundo plano	`expo-background-task` (el sistema decide cuándo, cada 15 minutos como mínimo)
Pedir una reseña en la tienda	`expo-store-review`
Widgets, Live Activities	`expo-widgets` (y librerías de la comunidad)

Cuando no existe una librería

Dos herramientas de Expo para tocar lo nativo **sin perder el prebuild**:

Config plugins: una función que modifica el proyecto nativo generado (el `Info.plist`, el `AndroidManifest.xml`, el `build.gradle`). Así hacen las librerías para agregar sus permisos, y podés escribir los tuyos:

```
// plugins/con-clave-de-mapas.js
const { withInfoPlist } = require('expo/config-plugins')

module.exports = function conClaveDeMapas(config, {
  clave }) {
  return withInfoPlist(config, (c) => {
    c.modResults.MiClaveDeMapas = clave
    return c
  })
}
```

```
// app.json
"plugins": [["./plugins/con-clave-de-mapas.js", {
  "clave": "abc123" }]]
```

Expo Modules API: para escribir tu propio módulo nativo en **Swift** y **Kotlin** con una API simple, sin tocar el proyecto a mano. `npx create-expo-module@latest --local` crea la carpeta `modules/` con el esqueleto. Así están hechas todas las librerías `expo-*`.

💡 Componentes DOM: código web adentro de la app

Con la directiva `'use dom'` al principio de un archivo, Expo renderiza ese componente con React DOM dentro de un WebView. Sirve para reutilizar algo que solo existe para la web (un editor de texto, una librería de gráficos) sin reescribirlo. No es para pantallas enteras, pero salva en casos puntuales.

Capítulo 14

Las librerías más usadas

Una tabla por necesidad: qué se usa hoy, si es de Expo, y si anda en Expo Go.

Una tabla por necesidad, con lo que se usa en proyectos reales a fines de 2026. La columna **Expo** dice si es parte del Expo SDK; las marcadas con **DB** necesitan un *development build* (no andan en Expo Go).

> ℹ️ **Cómo elegir una librería**
>
> Antes de instalar algo, mirá en reactnative.directory: si soporta la Nueva Arquitectura (obligatoria desde el SDK 55), si anda en Expo Go, cuándo fue el último release y cuántas descargas tiene. Y siempre `npx expo install`, para que elija la versión compatible con tu SDK.

Navegación y pantalla

Necesitás	Lo más usado	Expo
Navegación	**expo-router**	✓
Navegación (proyectos viejos)	React Navigation	
Safe area	react-native-safe-area-context	
Splash screen	expo-splash-screen	✓
Barra de estado	expo-status-bar	✓
Íconos	**expo-symbols** (SF Symbols + Material Symbols)	✓
Íconos (sets clásicos: Ionicons, FontAwesome...)	@react-native-vector-icons/* (uno por familia)	
Componentes nativos de SwiftUI / Compose	**@expo/ui**	✓
Efecto vidrio (iOS 26)	expo-glass-effect	✓
Degradados, desenfoque	expo-linear-gradient, expo-blur	✓

`@expo/vector-icons`, el paquete de íconos que usaban todas las plantillas, se está deprecando: Expo recomienda pasar a `@react-native-vector-icons` o a `expo-symbols`.

Estilos y componentes

Necesitás	Lo más usado	Expo
Tailwind	**NativeWind**, **Uniwind**	
StyleSheet con temas y breakpoints	Unistyles 3	DB
Kit de componentes + compilador	Tamagui	
Material Design	React Native Paper	
Componentes accesibles listos	HeroUI Native, gluestack-ui	
Bottom sheet	**@gorhom/bottom-sheet** (o una ruta `formSheet`)	
Toasts	sonner-native, burnt	

Listas, animaciones y gestos

Necesitás	Lo más usado	Expo
Listas largas	**@shopify/flash-list**, @legendapp/list	
Animaciones	**react-native-reanimated**	
Gestos	**react-native-gesture-handler**	
Teclado	**react-native-keyboard-controller**	
Animaciones de After Effects	lottie-react-native	

Necesitás	Lo más usado	Expo
Dibujo y efectos 2D	@shopify/react-native-skia	
Gráficos	victory-native, react-native-gifted-charts	
Háptica	expo-haptics	✓

Datos, estado y almacenamiento

Necesitás	Lo más usado	Expo	
Datos del servidor	**@tanstack/react-query**		
Estado global	**zustand**, jotai, Redux Toolkit		
Formularios	**react-hook-form** + **zod**, TanStack Form		
Clave-valor	@react-native-async-storage/async-storage		
Clave-valor rápido y sincrónico	react-native-mmkv		DB
Tokens y secretos	expo-secure-store	✓	
Base de datos	**expo-sqlite** + drizzle-orm	✓	
Archivos	expo-file-system	✓	
Offline con sincronización	PowerSync, Legend State		
Backend como servicio	**Supabase**, Firebase (@react-native-firebase)		Firebase: DB

Login y pagos

Necesitás	Lo más usado	Expo	
OAuth (GitHub, cualquier proveedor)	expo-auth-session + expo-web-browser	✓	
Sign in with Apple	expo-apple-authentication	✓	
Google Sign-In nativo	@react-native-google-signin/google-signin		DB
Auth como servicio	Clerk (@clerk/expo), Supabase Auth, Better Auth		
Biometría	expo-local-authentication	✓	
Suscripciones y compras dentro de la app	**react-native-purchases** (RevenueCat), expo-iap		DB
Pagos de cosas físicas o servicios	@stripe/stripe-react-native		Apple/Google Pay: DB

> **⚠️ Compras dentro de la app: no podés usar Stripe para todo**
>
> Si vendés algo **digital que se usa dentro de la app** (una suscripción premium, monedas, desbloquear funciones), Apple y Google exigen usar **su** sistema de pagos (In-App Purchase / Google Play Billing), que se queda con un 15–30 %. Stripe es para cosas físicas o servicios fuera de la app (un envío, un turno, una entrada). Lo vemos en requisitos. Por eso existe RevenueCat: maneja las suscripciones de las dos tiendas con una sola API.

Funciones del teléfono

Necesitás	Lo más usado	Expo	
Elegir fotos	expo-image-picker	✓	
Cámara, QR	expo-camera	✓	
Cámara avanzada (procesar cuadros)	react-native-vision-camera		DB
Ubicación	expo-location	✓	
Mapas	react-native-maps, expo-maps	expo-maps ✓	expo-maps: DB

Necesitás	Lo más usado	Expo	
Notificaciones	**expo-notifications**	✓	push en Android: DB
Notificaciones con más control / campañas	Notifee, OneSignal		DB
Video, audio	expo-video, expo-audio	✓	
Imágenes	**expo-image**	✓	
WebView	react-native-webview		
Idioma del usuario	expo-localization	✓	
Pagos con Apple Pay / Google Pay	@stripe/stripe-react-native		DB

Calidad y producción

Necesitás	Lo más usado	Expo
Tests	**jest-expo** + **@testing-library/react-native**	✓ (jest-expo)
Tests de punta a punta	**Maestro**	
Errores en producción	**@sentry/react-native**	
Analytics y feature flags	PostHog, Firebase Analytics, Amplitude	
Traducciones	**i18next** + react-i18next, Lingui	

Necesitás	Lo más usado	Expo
Fechas	date-fns, dayjs (y `Intl`, que Hermes soporta)	
Builds, envío a tiendas, actualizaciones	**EAS** (Build, Submit, Update, Workflows)	✓

Traducciones, por ser lo más pedido

```
// src/lib/i18n.ts
import { getLocales } from 'expo-localization'
import i18n from 'i18next'
import { initReactI18next } from 'react-i18next'

i18n.use(initReactI18next).init({
  resources: {
    es: { translation: { ofertas: 'Ofertas', guardar:
      'Guardar' } },
    en: { translation: { ofertas: 'Jobs', guardar:
      'Save' } },
  },
  lng: getLocales()[0]?.languageCode ?? 'es',  // el
    idioma del teléfono
  fallbackLng: 'es',
})

// En un componente
const { t } = useTranslation()
<Text>{t('ofertas')}</Text>
```

Para que la tienda muestre tu app como disponible en varios idiomas, también se traducen el nombre de la app y los textos de los permisos, con la clave `locales` de `app.json` (un JSON por idioma).

Parte 3 · Calidad

Que la app ande bien en teléfonos de verdad: rendimiento, accesibilidad, tests y depuración.

Capítulo 15

Buenas prácticas y rendimiento

Estructura del proyecto, el React Compiler, hilos, imágenes, accesibilidad y errores comunes.

Lo que separa una app que "anda" de una que se siente bien en el teléfono de cualquier persona (incluido un Android de gama baja con la letra agrandada).

Estructura del proyecto

```
src/
├── app/            ← solo pantallas y layouts (Expo
  Router)
├── components/     ← componentes reutilizables
  (TarjetaOferta, Boton, Estados)
```

```
├── features/       ← en apps grandes: una carpeta por
  funcionalidad (ofertas/, perfil/)
├── hooks/          ← hooks propios (use-ofertas)
├── stores/         ← estado global (zustand)
├── api/            ← el cliente de la API y las
  funciones que llaman al backend
├── lib/            ← utilidades y adaptadores
  (almacenamiento, i18n)
└── constants/      ← tema, colores, espacios
```

- **En `app/` solo rutas**: cualquier archivo ahí se convierte en pantalla. Las pantallas deberían ser finas: leen parámetros, llaman hooks y arman componentes.
- **Un tema centralizado** (colores, espacios, tipografía) en lugar de números mágicos por todos lados. La app de la guía lo tiene en `src/demos/tema.ts`.
- **Alias `@/`** en lugar de `../../../components`.
- **TypeScript estricto** y `experiments.typedRoutes`: los errores de rutas y de props aparecen antes de abrir la app.

React Compiler: chau useMemo y memo

La plantilla de Expo trae el React Compiler activado (`experiments.reactCompiler` en `app.json`). Memoiza componentes y valores solo, así que **no hace falta** `useMemo`, `useCallback` ni `memo` para evitar re-renders. Lo que sí hace falta: seguir las reglas de React (no mutar props ni estado, hooks siempre en el mismo

orden). Si un componente rompe las reglas, el compilador lo saltea sin romper nada.

Rendimiento

Primero medí: el monitor de rendimiento (menú de desarrollo → *Perf Monitor*) muestra los cuadros por segundo del hilo de UI y del de JavaScript, y el profiler de React Native DevTools muestra qué se re-renderizó. Siempre probá en una **build de release** (`npx expo run:android --variant release`) y en un **Android barato**: el modo desarrollo es mucho más lento, y tu iPhone es mucho más rápido que el teléfono promedio de tus usuarios.

Lo que más impacta:

1. **Listas**: FlashList o Legend List, filas livianas, imágenes con tamaño fijo (listas).
2. **Re-renders de árboles grandes**: con Zustand, elegí solo lo que usás (`useSesion((s) => s.nombre)`, no `useSesion()` entero). Con Context, separá los que cambian seguido.
3. **Imágenes**: `expo-image` con caché, y pedile al servidor la imagen del tamaño que mostrás (no una foto de 4000 px para un avatar de 48).
4. **Animaciones y gestos** en el hilo de UI con Reanimated y Gesture Handler, nunca con `setState` en cada cuadro.
5. **Arranque**: no cargues todo antes de mostrar la primera pantalla. El splash queda hasta que tengas

lo mínimo (la sesión), y el resto se carga después.

6. **Trabajo pesado**: algo que bloquea el hilo de JS más de 16 ms se nota. Partilo, movelo al servidor, o usá `useTransition` / `useDeferredValue` para que la UI responda primero (como el buscador de la app).
7. **Tamaño del bundle**: `EXPO_ATLAS=true npx expo export` y después `npx expo-atlas .expo/atlas.jsonl` muestra qué ocupa lugar. Una librería de fechas entera o un SDK enorme se notan en el arranque.

> ℹ️ **InteractionManager ya no existe**
>
> En código viejo vas a ver `InteractionManager.runAfterInteractions(() => …)` para hacer algo después de una animación. Se eliminó en el SDK 58: usá `requestIdleCallback`, `useTransition`, o el evento `transitionEnd` de la navegación.

Accesibilidad

Una de cada cinco personas usa alguna función de accesibilidad: letra agrandada, VoiceOver (iOS), TalkBack (Android), contraste alto, reducir movimiento. Y las tiendas lo miran cada vez más.

```
<Pressable
  onPress={alternar}
```

```
  accessibilityRole="button"                              //
    o role="button"
  accessibilityLabel={guardada ? 'Quitar de guardadas' :
    'Guardar oferta'}  // o aria-label
  accessibilityState={{ selected: guardada }}
  hitSlop={12}
>
  <Text>{guardada ? '♥' : '♡'}</Text>   {/* sin label,
    VoiceOver diría "corazón blanco" */}
</Pressable>

<Text accessibilityRole="header">Ofertas</Text>      {/*
  los lectores de pantalla saltan entre encabezados */}
```

- **Todo lo tocable que no tiene texto** (un ícono) necesita `accessibilityLabel`.
- **Áreas tocables de 44×44 puntos** como mínimo (48 en Android): `hitSlop` o padding.
- **La letra agrandada**: probá la app con el tamaño de letra al máximo. Los textos tienen que poder crecer (nada de alturas fijas en contenedores de texto).
- **Contraste**: 4,5:1 para texto normal. Ese gris clarito sobre blanco no se lee al sol.
- **Probalo**: activá VoiceOver o TalkBack y recorré tu app con los ojos cerrados. Es la mejor prueba.

React Native acepta también las props de la web: `role`, `aria-label`, `aria-hidden`, `aria-checked`, etc.

Errores

- **Error boundaries por ruta**: una pantalla puede exportar `ErrorBoundary` y Expo Router lo muestra si

esa pantalla explota, en lugar de cerrar la app.

```
// app/oferta/[id].tsx
export function ErrorBoundary({ error, retry }:
  ErrorBoundaryProps) {
  return <ErrorConReintento mensaje={error.message}
    reintentar={retry} />
}
```

- **Estados de carga, error y vacío** en cada pantalla que trae datos (la app tiene `Cargando` y `ErrorConReintento` en `components/Estados.tsx`).
- **Sentry** (o similar) en producción: sin eso, te enterás de los crashes por las reseñas de una estrella.

Seguridad

- **No hay secretos en la app**: todo lo que va en el bundle (incluidas las variables `EXPO_PUBLIC_`) se puede leer.
- **Tokens en SecureStore**, nunca en AsyncStorage, y nunca en los logs.
- **HTTPS siempre.**
- **Validá los parámetros de los deep links**: cualquiera puede armar un link `busquedarn://…` con lo que quiera.
- La seguridad de verdad está en el **backend**: validar el token y los permisos en cada endpoint.

Respetar las convenciones de cada plataforma

- **Android tiene botón (o gesto) "atrás"**: los modales tienen que cerrarse con él (`onRequestClose`), y la navegación de Expo Router ya lo maneja.
- **iOS tiene el gesto de deslizar desde el borde** para volver: no lo rompas con un gesto horizontal propio en toda la pantalla.
- **Componentes del sistema** cuando existen: pestañas nativas, menús, selectores de fecha, la hoja de compartir.
- **Modo oscuro y letra grande**: los usuarios esperan que la app los respete.

Errores comunes

Error	Solución
Text strings must be rendered within a `<Text>`	Texto suelto en un `View`, o un `{0 && …}`.
Un texto largo se sale por la derecha	`flex: 1` o `flexShrink: 1` en su contenedor.
La lista tarda en aparecer	`ScrollView` con `map`: usá FlatList/FlashList.

Error	Solución
Hay que tocar dos veces el botón con el teclado abierto	`keyboardShouldPersistTaps="handled"`.
El contenido queda debajo de la isla o la barra de gestos	Safe area.
Una librería nativa "no se encuentra" en Expo Go	Necesita un development build.
Instalaste con `npm install` y la app crashea	`npx expo install --fix`.
El efecto no corre al volver a la pantalla	Las pantallas no se desmontan: `useFocusEffect`.
En el teléfono no llega a tu API local	`localhost` es el teléfono: usá la IP de tu compu.
Todo anda en desarrollo, la build de release crashea	Probá release seguido: `npx expo run:ios --configuration Release`.

Capítulo 16

Tests y depuración

Jest con jest-expo, Testing Library, tests de navegación, E2E con Maestro y React Native DevTools.

Los tests en React Native son los mismos de la web: **Jest** como runner y **Testing Library** para renderizar componentes y usarlos como lo haría una persona. Lo que cambia: los módulos nativos no existen en Node, así que se reemplazan por versiones falsas (*mocks*).

Jest con jest-expo

`jest-expo` es una configuración de Jest preparada para Expo: transforma el código, simula los módulos del Expo SDK y entiende los archivos `.ios.tsx` / `.android.tsx`. En el `package.json` de la app:

```
"scripts": { "test": "jest" },
"jest": {
  "preset": "jest-expo",
  "transformIgnorePatterns": [
    "/node_modules/(?!(.pnpm|react-native|@react-
      native|@react-native-
```

```
      community|expo|@expo|@shopify/flash-list))"
  ]
}
```

`transformIgnorePatterns` dice qué paquetes de `node_modules` hay que transformar: muchas librerías de React Native se publican sin compilar, y si una tira *"SyntaxError: Cannot use import statement outside a module"*, hay que agregarla a esa lista.

```
npx expo install jest-expo jest @types/jest @testing-
  library/react-native test-renderer -- --save-dev
npm test
```

Testing Library

`@testing-library/react-native` tiene la misma filosofía que en la web: buscás elementos por lo que ve el usuario (texto, label, rol) y los usás con `userEvent`. Este es el test del formulario de postulación:

app-expo/src/demos/formularios/__tests__/Postulacion.test.tsx

```
import { act, render, screen, userEvent } from
  '@testing-library/react-native'
import Postulacion from '../Postulacion'

// Se testea como lo usaría una persona: escribir, tocar
  y leer lo que aparece en pantalla.
describe('Postulacion', () => {
```

```
  beforeEach(() => jest.useFakeTimers())
  afterEach(() => jest.useRealTimers())

  test('muestra los errores si se envía vacío', async ()
    => {
    const user = userEvent.setup()
    await render(<Postulacion />)

    await user.press(screen.getByText('Enviar
      postulación'))

    expect(screen.getByText('Escribí tu
      nombre')).toBeOnTheScreen()
    expect(screen.getByText('El email no es
      válido')).toBeOnTheScreen()
  })

  test('con los datos completos, envía y confirma',
    async () => {
    const user = userEvent.setup()
    await render(<Postulacion />)

    const [nombre, email, mensaje] =
      screen.getAllByDisplayValue('')
    await user.type(nombre, 'Ana')
    await user.type(email, 'ana@ejemplo.com')
    await user.type(mensaje, 'Me encanta React Native y
      busco un equipo chico.')
    await user.press(screen.getByText('Enviar
      postulación'))

    expect(screen.queryByText('Escribí tu
      nombre')).not.toBeOnTheScreen()
    // La demo simula un envío de 1,2 segundos:
      adelantamos el reloj falso
    await act(() => jest.advanceTimersByTime(1200))
    expect(screen.getByText('¡Listo,
      Ana!')).toBeOnTheScreen()
  })
})
```

- La versión 14 es **asincrónica**: `await render(...)`.
- `userEvent.setup()` simula a una persona: `user.press` (con pressIn y pressOut, como un dedo de verdad), `user.type` (letra por letra), `user.scrollTo`.
- **Consultas**: `getByText`, `getByRole('button', { name: 'Entrar' })`, `getByLabelText`, `getByPlaceholderText`, `getByDisplayValue`. Con `findBy…` espera a que aparezca (para lo asincrónico); con `queryBy…` devuelve `null` si no está.
- `toBeOnTheScreen()`, `toHaveTextContent`, `toBeDisabled` vienen incluidos.
- Con **relojes falsos** (`jest.useFakeTimers()`), `jest.advanceTimersByTime` adelanta el tiempo sin esperarlo de verdad.

Mocks de módulos nativos

`jest-expo` ya simula los módulos de Expo. Para otras librerías nativas, muchas traen su propio mock. Así se testea el store de guardadas, que usa AsyncStorage:

app-expo/src/stores/__tests__/guardadas.test.ts

```
import { ofertas } from '@/demos/datos'
import { useGuardadas } from '../guardadas'
```

```
// AsyncStorage es nativo: en los tests se reemplaza por
  la versión en memoria que trae la librería
jest.mock('@react-native-async-storage/async-storage',
  () =>
  require('@react-native-async-storage/async-
    storage/jest/async-storage-mock'),
)

// Un store de zustand se puede testear sin renderizar
  nada: getState() y sus acciones
test('alternar guarda y quita una oferta', () => {
  const { alternar } = useGuardadas.getState()

  alternar(ofertas[0])
  alternar(ofertas[1])
  expect(useGuardadas.getState().ofertas.map((o) =>
    o.id)).toEqual([2, 1])

  alternar(ofertas[0])
  expect(useGuardadas.getState().ofertas.map((o) =>
    o.id)).toEqual([2])
})
```

Un store de Zustand se testea sin renderizar nada: `getState()` y sus acciones.

Tests de navegación

`expo-router/testing-library` arma un router en memoria con las pantallas que le pases, y agrega matchers como `toHavePathname`:

app-expo/src/app/__tests__/navegacion.test.tsx

```tsx
import { QueryClient, QueryClientProvider } from
  '@tanstack/react-query'
import { Stack } from 'expo-router'
import { userEvent } from '@testing-library/react-
  native'
import { renderRouter, screen } from 'expo-
  router/testing-library'
import Ofertas from '../(tabs)/index'
import DetalleOferta from '../oferta/[id]'

jest.mock('@react-native-async-storage/async-storage',
  () =>
  require('@react-native-async-storage/async-
    storage/jest/async-storage-mock'),
)

// renderRouter arma un Expo Router en memoria con las
  pantallas que le pasamos
test('tocar una oferta abre su detalle', async () => {
  const queryClient = new QueryClient()
  const Layout = () => (
    <QueryClientProvider client={queryClient}>
      <Stack />
    </QueryClientProvider>
  )
  await renderRouter({ _layout: Layout, index: Ofertas,
    'oferta/[id]': DetalleOferta }, { initialUrl: '/' })
  const user = userEvent.setup()

  await user.press(await screen.findByText('Expo
    Developer', {}, { timeout: 3000 }))

  expect(screen).toHavePathname('/oferta/5')
  expect(await screen.findByText('Hasta US$ 3600 por
    mes')).toBeOnTheScreen()
})
```

Este test usa las pantallas reales de la app (la lista y el detalle) con la API falsa: escribe, toca una oferta, y comprueba que se abrió `/oferta/5` con los datos correctos.

Tests de punta a punta con Maestro

Jest corre en Node: no prueba la app de verdad en un teléfono. Para eso están los tests **E2E** (*end to end*), que abren la app en un simulador y la usan. **Maestro** es el más usado hoy con Expo: los tests son archivos YAML, sin código.

```yaml
# app-expo/.maestro/login.yaml
appId: com.ejemplo.busquedarn
---
- launchApp:
    clearState: true
- tapOn: "Email"
- inputText: "ana@ejemplo.com"
- tapOn: "Contraseña"
- inputText: "expo1234"
- tapOn: "Entrar"
- assertVisible: "Ofertas"
- tapOn: "Expo Developer"
- assertVisible: "Hasta US$ 3600 por mes"
```

```
maestro test .maestro/login.yaml   # con la app
  instalada en el simulador
maestro studio                     # una interfaz para
  armar los pasos tocando la app
```

Se pueden correr en la nube con **EAS Workflows** después de cada build. La alternativa clásica es **Detox**, más potente pero bastante más difícil de configurar.

Qué testear

- **Lógica pura** (validaciones, cálculos, stores): tests unitarios, rápidos y baratos.
- **Componentes con comportamiento** (formularios, listas con filtro): Testing Library.
- **Flujos críticos** (login, comprar, postularse): uno o dos tests E2E con Maestro.
- No vale la pena testear que un `View` tiene cierto color.

Depurar

Con `npx expo start` corriendo, apretá `j` para abrir **React Native DevTools** (las DevTools de Chrome, adaptadas):

Panel	Para qué
Console	Los `console.log` (también salen en la terminal), y ejecutar código en la app.
Sources	Breakpoints y `debugger;` en tu código TypeScript.
Network	Los pedidos de red que hace la app.
Components	El árbol de React con props y estado, como React DevTools en la web.

Panel	Para qué
Profiler	Qué se renderizó, cuántas veces y cuánto tardó.
Memory	Para buscar fugas de memoria.

En el teléfono, el **menú de desarrollo** (sacudir el teléfono, o `m` en la terminal) tiene el **Element Inspector** (tocás algo y ves sus estilos), el **Perf Monitor** y recargar.

Otras herramientas:

- **LogBox**: las advertencias amarillas y los errores rojos en pantalla, con el stack trace.
- **Radon IDE**: una extensión de VS Code y Cursor (de Software Mansion) con el simulador adentro del editor, breakpoints e inspector.
- **Expo Atlas**: qué ocupa lugar en el bundle.
- **Logs nativos**: si la app se cierra sin error de JavaScript, el problema es nativo. Mirá los logs del sistema: Console.app en Mac para iOS, `adb logcat` para Android.
- **Sentry**: errores en producción, con el stack trace de tu código (subiendo los *source maps* en cada build).

Parte 4 · Publicar

De tu computadora a las tiendas: builds en la nube, envío, revisión y actualizaciones.

Capítulo 17

Builds con EAS

eas.json, perfiles, credenciales, versiones y development builds.

Hasta acá corriste la app en Expo Go. Para instalarla como una app de verdad (con su ícono, su nombre y sus permisos) o subirla a las tiendas, hay que **compilarla**: generar un `.ipa` para iOS y un `.aab` o `.apk` para Android, firmados con tus certificados. **EAS Build** lo hace en la nube, así que no necesitás una Mac para compilar para iOS.

EAS en una tabla

Servicio	Qué hace
EAS Build	Compila tu app en servidores de Expo (macOS para iOS, Linux para Android) y maneja las firmas.

Servicio	Qué hace
EAS Submit	Sube la build a App Store Connect y a Google Play Console.
EAS Update	Manda cambios de JavaScript a las apps ya instaladas, sin pasar por la tienda (actualizaciones).
EAS Workflows	CI/CD: encadena build, tests, submit y update cuando hacés push.

Tienen un plan gratuito (con un límite de builds por mes y una cola más lenta) que alcanza para proyectos personales. Y todo se puede hacer sin EAS: `npx expo prebuild` genera los proyectos nativos y compilás con Xcode y Android Studio, o `eas build --local` usa EAS en tu máquina.

Empezar

```
npm install --global eas-cli
eas login              # tu cuenta de expo.dev
eas init               # crea el proyecto en EAS y
  agrega su id a app.json
eas build:configure    # crea eas.json
```

eas.json: los perfiles

Un **perfil** es una forma de compilar. Lo normal es tener tres:

app-expo/eas.json

```
{
  "cli": {
    "version": ">= 27.0.0",
    "appVersionSource": "remote"
  },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal",
      "channel": "development",
      "env": { "APP_VARIANT": "development" }
    },
    "preview": {
      "distribution": "internal",
      "channel": "preview",
      "android": { "buildType": "apk" },
      "env": { "APP_VARIANT": "preview" }
    },
    "production": {
      "channel": "production",
      "autoIncrement": true
    }
  },
  "submit": {
    "production": {
      "ios": { "ascAppId": "1234567890" },
      "android": { "track": "internal", "releaseStatus":
        "draft" }
    }
  }
}
```

Perfil	Para qué	Cómo se instala
development	Tu propia "Expo Go", con tus librerías nativas. Se conecta a `npx expo start`.	Link o QR (distribución interna)
preview	Una build de release para que la pruebe tu equipo o un cliente.	Link o QR; en Android, un `.apk`
production	La que va a las tiendas.	Por la tienda (TestFlight / Play)

- `distribution: "internal"` genera una build que se instala con un link, sin tienda. En iOS, los iPhone tienen que estar **registrados** en tu cuenta de Apple (`eas device:create` arma un link para registrarlos).
- `channel` une la build con un canal de EAS Update.
- `env` define variables al compilar: acá `APP_VARIANT`, que usa el `app.config.ts` de ejemplo para cambiar el nombre y el identificador, y poder tener la versión de desarrollo y la de producción instaladas a la vez.
- `autoIncrement` + `appVersionSource: "remote"`: EAS lleva la cuenta del número de build y lo sube solo en cada build de producción.

```
eas build --profile development --platform ios
eas build --profile preview --platform android
eas build --profile production --platform all
```

Al terminar, EAS te da un link y un QR para instalar la build.

> 💡 **Sin cuenta de Apple, para el simulador**
>
> Para instalar en un iPhone de verdad necesitás la cuenta paga de Apple (USD 99 por año), aunque sea una build interna. Para el **simulador** de iOS no: un perfil con `"ios": { "simulator": true }` compila sin firmar.

Versión y número de build

Cada tienda pide dos números:

	Qué es	Dónde	Ejemplo
Versión	La que ven los usuarios. La subís vos.	`version` en `app.json`	`1.2.0`
Número de build	Un entero que tiene que crecer en **cada** subida, aunque la versión sea la misma.	`ios.buildNumber` / `android.versionCode` (o EAS, con `autoIncrement`)	`14`

Si subís dos builds con el mismo número, la tienda rechaza la segunda. Con `appVersionSource: "remote"` y `autoIncrement`, no te tenés que acordar.

Credenciales: las firmas

Toda app tiene que estar **firmada**, para que el sistema sepa que viene de vos. Es la parte más temida del desarrollo móvil, y EAS la resuelve casi entera:

- **iOS**: un certificado de distribución y un *provisioning profile* por app. EAS los crea en tu cuenta de Apple la primera vez (te pide tu Apple ID) y los guarda.
- **Android**: un *keystore* (un archivo con la clave). EAS lo genera y lo guarda. Con **Play App Signing** (obligatorio para apps nuevas), Google firma la versión final, y tu keystore es solo la "clave de subida".

```
eas credentials   # ver, descargar o cambiar las
  credenciales
```

> ⚠️ **Hacé un backup del keystore**
>
> Si perdés la clave de subida de Android, se puede pedir a Google que la reemplace, pero es un trámite. Descargá un backup con `eas credentials` y guardalo en un lugar seguro (no en git).

Variables de entorno y secretos de build

Las variables `EXPO_PUBLIC_` del `.env` sirven en tu máquina, pero el `.env` no se sube a git y EAS compila desde tu repo. Para las builds en la nube, las variables se guardan en EAS por ambiente (development, preview, production):

```
eas env:create --name EXPO_PUBLIC_API_URL --value
  https://api.ejemplo.com --environment production
eas env:create --name SENTRY_AUTH_TOKEN --value xxxx --
  environment production --visibility secret
eas env:pull --environment development   # las baja a tu
  .env local
```

Una variable **secreta** sirve para el proceso de build (subir source maps a Sentry, bajar un paquete privado), pero nunca debe terminar en el código de la app.

Development builds en el día a día

Cuando tu app necesita una librería nativa que no está en Expo Go, el flujo pasa a ser:

1. `eas build --profile development --platform android` (una vez, o cuando cambiás algo nativo).
2. Instalás esa build en el teléfono.
3. `npx expo start` y la abrís: funciona igual que Expo Go, con Fast Refresh, pero con tus librerías.

Solo hace falta compilar de nuevo cuando **agregás o actualizás una librería nativa** o cambiás `app.json` (permisos, plugins). Si solo cambiás JavaScript, alcanza con recargar.

Capítulo 18

Subir a App Store y Google Play

Cuentas, TestFlight, pistas de prueba de Google, envío con EAS Submit y la revisión.

Subir una app es más trámite que código: crear las cuentas, completar la ficha de la tienda, responder cuestionarios, probar con testers y pasar la revisión. Este capítulo es el recorrido completo; el que sigue es la lista de requisitos que revisan.

Las cuentas

	Apple Developer Program	Google Play Console
Costo	**USD 99 por año**	**USD 25, una vez**
Cuenta personal	Con tu Apple ID y tu identidad	Con tu identidad verificada (documento)

	Apple Developer Program	Google Play Console
Cuenta de empresa	Necesita un número **D-U-N-S** (gratis, de Dun & Bradstreet; tarda días o semanas)	También pide D-U-N-S
Lo que figura como desarrollador	Tu nombre, o el de la empresa	Tu nombre, o el de la empresa
Apps pagas o con compras	Firmar el acuerdo de apps pagas y cargar datos bancarios e impositivos	Crear un perfil de pagos

> ℹ️ **¿Personal o empresa?**
>
> Con una cuenta personal, en la tienda aparece **tu nombre** como desarrollador, y en Google además tenés el requisito de los 12 testers (abajo). Si la app es de un cliente, lo correcto es que la publique con **su** cuenta, y te agregue como miembro del equipo.

El recorrido en Apple

1. **Crear la app en App Store Connect** (appstoreconnect.apple.com): nombre, idioma, bundle id (el de `app.json`) y un SKU (cualquier código interno).
2. **Compilar y subir**:

```
eas build --platform ios --profile production
eas submit --platform ios          # o las dos juntas:
  eas build --platform ios --auto-submit
```

3. **TestFlight**: en 10–15 minutos la build aparece en TestFlight. Los **testers internos** (hasta 100 personas de tu equipo en App Store Connect) la instalan enseguida con la app TestFlight. Los **externos** (hasta 10.000, con un link público) necesitan que Apple apruebe la primera build de cada versión (*Beta App Review*, suele ser rápido).
4. **Completar la ficha**: capturas, descripción, palabras clave, URL de soporte y de política de privacidad, categoría, la **clasificación por edad** (un cuestionario), las **etiquetas de privacidad** (qué datos juntás) y la información para el revisor (un usuario de prueba).
5. **Enviar a revisión**: elegís la build y "Add for Review". La mayoría se revisa en **24 a 48 horas**.
6. **Publicar**: apenas se aprueba, en una fecha, o manualmente. Podés hacer un **lanzamiento gradual** de una actualización en 7 días.

Si te rechazan, llega un mensaje en **Resolution Center** citando la regla (por ejemplo "Guideline 5.1.1"). Respondés ahí, corregís, y volvés a enviar. Si creés que el revisor se equivocó, podés explicarlo en la respuesta o apelar.

El recorrido en Google

1. **Crear la app en Play Console** (play.google.com/console): nombre, idioma, si es app o juego, gratis o paga.
2. **Completar "Configurar tu app"**: una lista de tareas con declaraciones obligatorias (las vemos en requisitos): política de privacidad, acceso a la app, anuncios, clasificación de contenido, público objetivo, seguridad de los datos, y más.
3. **Compilar y subir** un `.aab` (Android App Bundle; Google ya no acepta `.apk` para apps nuevas):

```
eas build --platform android --profile production
eas submit --platform android      # necesita una clave
  de cuenta de servicio de Google
```

Para que `eas submit` pueda subir, creás una **cuenta de servicio** en Google Cloud con acceso a tu Play Console y le das la clave JSON a EAS (`eas credentials`). La primera vez sube a la **pista interna** como borrador.

4. **Las pistas de prueba**:

Pista	Para quién	Revisión
Prueba interna	Hasta 100 testers por email	Casi instantánea
Prueba cerrada	Grupos por email o Google Groups	Sí

Pista	Para quién	Revisión
Prueba abierta	Cualquiera que se anote desde la tienda	Sí
Producción	Todo el mundo	Sí

5. **Producción**: promovés la build y elegís un **porcentaje** (lanzamiento escalonado: 10 %, 50 %, 100 %). La revisión puede tardar **horas o hasta 7 días**, más en cuentas nuevas.

> **⚠️ Cuentas personales nuevas: 12 testers durante 14 días**
>
> Si tu cuenta de Google Play es **personal** y la creaste después del 13 de noviembre de 2023, antes de poder publicar en producción tenés que hacer una **prueba cerrada con al menos 12 testers** que se hayan anotado y la tengan instalada durante **14 días seguidos**. Recién ahí podés pedir el acceso a producción (respondiendo unas preguntas sobre la prueba). Las cuentas de empresa no tienen este requisito. Planificalo: son al menos dos semanas antes del lanzamiento.

La ficha de la tienda

	App Store	Google Play
Nombre	30 caracteres	30 caracteres
Subtítulo / descripción corta	Subtítulo, 30 caracteres	Descripción corta, 80 caracteres
Descripción	4.000 caracteres	4.000 caracteres
Palabras clave	Campo propio, 100 caracteres	No hay: se usan el título y las descripciones
Ícono	1024×1024, sin transparencia (lo saca de la build)	512×512 PNG, en la ficha
Capturas	iPhone de 6,9" (Apple escala al resto); iPad de 13" si soporta tablet	Al menos 2 del teléfono; tablet recomendado
Imagen destacada	—	**1024×500, obligatoria**
Video	App Preview, opcional	Un video de YouTube, opcional
Política de privacidad	URL obligatoria	URL obligatoria

Las capturas son lo que más influye en las descargas: muestran la app con un texto corto encima que explica el beneficio ("Encontrá trabajo remoto en minutos"), no la pantalla de login.

Antes de enviar: checklist

- [ ] Probaste la **build de producción** (no solo el modo desarrollo) en un iPhone y un Android de verdad.
- [ ] `version` y número de build actualizados.
- [ ] Ícono, splash y nombre definitivos.
- [ ] Textos de los permisos claros y específicos; ningún permiso de más.
- [ ] Política de privacidad publicada en una URL.
- [ ] Usuario y contraseña de prueba para el revisor (si hay login), que funcionen, con datos cargados.
- [ ] Si se pueden crear cuentas: se pueden **borrar** desde la app.
- [ ] Si hay login con Google u otro: también Sign in with Apple (en iOS).
- [ ] Las compras digitales usan el sistema de pagos de la tienda.
- [ ] Ningún botón que no haga nada, ningún "próximamente", ningún texto de ejemplo.
- [ ] Capturas reales de la app.

Después de publicar

- **Respondé reseñas**, sobre todo las malas: las dos tiendas lo permiten, y los usuarios cambian la calificación cuando les resolvés el problema.
- **Pedí reseñas en el momento justo** con `expo-store-review` (después de algo que salió bien, no al abrir

la app). El sistema decide si muestra el diálogo.

- **Mirá los crashes**: Sentry, Firebase Crashlytics, el *Android vitals* de Play Console y la sección de crashes de App Store Connect / TestFlight.
- **Mantené la app al día**: si no actualizás el *target SDK* de Android cada año, Google deja de mostrar la app a usuarios nuevos; y Apple puede sacar apps abandonadas que no funcionan en versiones nuevas de iOS.

Capítulo 19

Lo que piden las tiendas

La lista de requisitos de Apple y Google en 2026, y los motivos de rechazo más comunes.

Lo que Apple y Google revisan antes de aprobar una app, y lo que te van a pedir completar en sus consolas. Las reglas cambian todos los años; esta es la foto de **noviembre de 2026**. Las fuentes oficiales: las *App Review Guidelines* de Apple (developer.apple.com/app-store/review/guidelines) y el *Policy Center* de Google Play.

Requisitos técnicos con fecha

Tienda	Requisito	Desde	Con Expo
Apple	Compilar con **Xcode 26** y el **SDK de iOS 26**	28 de abril de 2026	EAS usa la imagen correcta; el SDK 58 pide Xcode 26.4+.

Tienda	Requisito	Desde	Con Expo
Google	Apps nuevas y actualizaciones con **target API 36** (Android 16)	31 de agosto de 2026 (se podía pedir prórroga hasta el 1 de noviembre)	El SDK 58 compila con target 36.
Google	Las apps existentes con target menor a **35** dejan de mostrarse a usuarios nuevos	31 de agosto de 2026	Actualizá el SDK al menos una vez por año.
Google	Soportar páginas de memoria de **16 KB**	1 de noviembre de 2025	React Native y Expo ya lo soportan.
Google	Publicar en formato **AAB** (no APK)	2021	El perfil de producción genera `.aab`.
Apple	**Privacy manifest** y razones para APIs sensibles	1 de mayo de 2024	Las librerías de Expo traen el suyo; lo tuyo va en `ios.privacyManifests`.

La regla práctica: **un SDK de Expo de menos de un año cumple los requisitos técnicos de las dos tiendas**. Si te quedás dos o tres SDK atrás, un día no vas a poder subir una actualización.

Lo que piden las dos

Política de privacidad

Una URL pública con tu política de privacidad, aunque la app no junte datos. Tiene que decir qué datos juntás, para qué, con quién los compartís (incluidos los SDK de terceros como analytics o Sentry), cuánto tiempo los guardás y cómo pedir que los borren. Además, tiene que haber un link a la política **dentro de la app** (en el perfil o la configuración).

Declarar qué datos juntás

- **Apple: "App Privacy"** (las etiquetas de privacidad que se ven en la ficha): qué datos juntás (email, ubicación, identificadores, datos de uso, crashes…), si se vinculan a la identidad del usuario y si se usan para *tracking*.
- **Google: "Seguridad de los datos"** (*Data safety*): lo mismo, más si los datos viajan cifrados y si el usuario puede pedir borrarlos.

Tienen que coincidir con lo que la app hace de verdad, **incluyendo lo que hacen tus librerías**: si usás Sentry, juntás datos de crashes; si usás Firebase Analytics, datos de uso e identificadores. Declarar de menos es motivo de rechazo o de suspensión.

Borrar la cuenta desde la app

Si la app permite **crear una cuenta**, tiene que permitir **borrarla desde la app**, de forma fácil de encontrar (en el perfil o la configuración). No alcanza con desactivarla ni con "escribinos un email". Apple lo exige desde 2022; Google, además, pide una **URL web** donde se pueda pedir el borrado sin tener la app instalada.

```
// En el perfil: un botón que confirma y llama a tu
  backend
async function borrarCuenta() {
  await api('/cuenta', { method: 'DELETE' })   // el
    backend borra los datos de verdad
  useSesion.getState().cerrarSesion()
}
```

Un usuario de prueba para el revisor

Si hay login, en la información para la revisión ponés un **usuario y contraseña que funcionen**, con datos cargados para que vea la app completa. Si el login es por SMS o con un dispositivo especial, explicá cómo probarla (o dejá un modo demo). "No pudimos entrar" es uno de los rechazos más comunes (*Guideline 2.1, App Completeness*). En Google es la declaración **"Acceso a la app"**.

Permisos justificados

- Pedí **solo** los permisos que usás, y en el momento en que los usás.
- En iOS, el texto de cada permiso tiene que explicar **para qué** en concreto (*Guideline 5.1.1*). "Usamos la cámara para que saques tu foto de perfil" se aprueba; "La app necesita acceso a la cámara" no.
- Revisá los permisos que agregan tus librerías (`npx expo config --type introspect`) y bloqueá los que no usás:

```
"android": {
  "blockedPermissions":
    ["android.permission.RECORD_AUDIO",
    "android.permission.ACCESS_BACKGROUND_LOCATION"]
}
```

Compras: el sistema de pagos de la tienda

Vendés	Cómo cobrás
Algo **digital que se usa en la app**: suscripción premium, monedas, desbloquear funciones, contenido	**Obligatorio** el sistema de la tienda: In-App Purchase (Apple) / Google Play Billing. Comisión del 15–30 %.
Bienes físicos o servicios fuera de la app: un producto, un viaje, un turno, una entrada	Tu procesador de pagos (Stripe, Mercado Pago…). Sin comisión de la tienda.
Una app "lectora" (contenido que se compra en otro lado: Netflix, Spotify)	Puede no vender en la app; las reglas para poner un link afuera dependen del país.

La comisión es del **15 %** para desarrolladores que facturan menos de un millón de dólares por año (*Small Business Program* de Apple, y el primer millón en Google) y para suscripciones después del primer año. En algunos países (Estados Unidos, la Unión Europea, Corea, Japón) hoy se permiten links o sistemas de pago alternativos, con condiciones que cambiaron varias veces desde 2024: si tu modelo depende de eso, leé las reglas vigentes para cada país.

Las suscripciones también tienen reglas: mostrar el precio y el período claramente antes de comprar, un botón de **restaurar compras**, y links a los términos y a la política de privacidad en la pantalla de compra.

Funcionalidad mínima y contenido

- **No una web envuelta**: una app que solo muestra un sitio en un WebView se rechaza (*Guideline 4.2, Minimum Functionality*). Tiene que aportar algo de app.
- **Nada a medio hacer**: botones que no hacen nada, pantallas de "próximamente", texto de ejemplo, links rotos.
- **Metadatos honestos**: las capturas muestran la app real, la descripción no promete lo que no hace, y el nombre no usa marcas ajenas.
- **Contenido generado por usuarios** (un chat, comentarios, perfiles): hace falta poder **reportar** contenido, **bloquear** usuarios, filtrar lo ofensivo, y tener un contacto.

- **No copiar**: apps casi iguales a otras ya publicadas (o muchas apps tuyas casi iguales) se rechazan como spam (*Guideline 4.3*).

Solo Apple

Requisito	Qué hay que hacer
Sign in with Apple (o equivalente)	Si ofrecés login con Google, Facebook u otro tercero, también una opción que cuide la privacidad: Sign in with Apple la cumple (auth).
Clasificación por edad	El cuestionario nuevo (clasificaciones 4+, 9+, 13+, 16+ y 18+) tenía que estar respondido para enero de 2026 para poder publicar actualizaciones.
App Tracking Transparency	Si rastreás al usuario entre apps de otras empresas (publicidad personalizada, compartir datos con data brokers), pedir permiso con el diálogo del sistema (`expo-tracking-transparency`). Sin permiso, no podés usar el identificador de publicidad.
Privacy manifest	Un archivo que declara qué APIs "sensibles" usa la app y por qué. Las librerías de Expo traen el suyo; si tu código o una librería sin manifest usa una de esas APIs, se declara en `app.json` (ejemplo abajo).
Cifrado (export compliance)	En cada build Apple pregunta si usás cifrado. Si solo usás HTTPS, con `ios.config.usesNonExemptEncryption: false` (como la app de la guía) no pregunta más.

Requisito	Qué hay que hacer
Estado de "trader" en la UE	Por la ley de servicios digitales europea, declarar si sos comerciante (con datos de contacto públicos). Sin eso, la app no se publica en la Unión Europea.
Ícono para iOS 26	Los íconos de iOS 26 tienen capas (Liquid Glass). La plantilla de Expo usa un archivo `.icon` de Icon Composer (`ios.icon` en `app.json`).

```
// app.json: declarar el uso de UserDefaults (lo usan
  AsyncStorage y otras librerías)
"ios": {
  "privacyManifests": {
    "NSPrivacyAccessedAPITypes": [
      {
        "NSPrivacyAccessedAPIType":
          "NSPrivacyAccessedAPICategoryUserDefaults",
        "NSPrivacyAccessedAPITypeReasons": ["CA92.1"]
      }
    ]
  }
}
```

Solo Google

Las declaraciones de la sección **"Contenido de la app"** de Play Console (se completan una vez y se actualizan si cambia algo):

Declaración	Qué preguntan
Política de privacidad	La URL.

Declaración	Qué preguntan
Acceso a la app	Si hay partes con login, las credenciales de prueba.
Anuncios	Si la app muestra publicidad (se muestra un sello en la ficha).
Clasificación del contenido	Un cuestionario (IARC) que da la clasificación para cada país.
Público objetivo	Para qué edades es. Si incluye menores de 13, se aplican las reglas de *Familias*, mucho más estrictas.
Seguridad de los datos	Qué datos juntás y para qué (ver arriba).
Apps de gobierno, financieras, de salud, de noticias	Si la app es de alguno de esos tipos, declaraciones y documentación extra.
ID de publicidad	Si usás el identificador de publicidad.

Y las políticas de permisos más estrictas:

- **Fotos y videos**: las apps solo pueden pedir acceso a toda la galería (`READ_MEDIA_IMAGES`) si es su función principal (una galería, un editor). El resto tiene que usar el **selector de fotos del sistema**, que no pide permiso. `expo-image-picker` ya lo usa.
- **Ubicación en segundo plano**: una declaración con un **video** que muestre la función, y que sea esencial para la app.
- **Servicios en primer plano** (*foreground services*, como un reproductor o un GPS activo): declarar el tipo y justificarlo.

- **Alarmas exactas**, **acceso a todos los archivos**, **lista de apps instaladas**, **accesibilidad**: permisos restringidos que se aprueban solo para casos puntuales.

Los rechazos más comunes

Motivo	Regla	Cómo evitarlo
El revisor no pudo entrar o algo no funcionó	Apple 2.1	Usuario de prueba con datos; probá la build de producción.
Textos de permisos vagos	Apple 5.1.1	Explicá para qué, en concreto.
Login con Google sin opción de Apple	Apple 4.8	Agregá Sign in with Apple.
No se puede borrar la cuenta	Apple 5.1.1(v) / Google	Botón de borrar cuenta en la app.
Vender contenido digital con Stripe	Apple 3.1.1 / Google Payments	In-App Purchase / Play Billing.
Es solo una web	Apple 4.2	Funciones nativas, contenido propio, que funcione sin ser una web.
Capturas que no son la app, metadatos engañosos	Apple 2.3 / Google Metadata	Capturas reales, descripción honesta.
Declaración de datos incompleta	Google Data safety / Apple 5.1.2	Incluí lo que juntan tus librerías.

Motivo	Regla	Cómo evitarlo
Permiso de fotos o ubicación sin justificar	Google	Selector de fotos; ubicación solo en uso.
Crash al abrir	Apple 2.1 / Google	Probá en un teléfono real con la build de producción.

🎤 En una entrevista

"¿Qué tenés en cuenta para publicar una app?" Una respuesta completa: cuentas (Apple USD 99 por año, Google USD 25), builds firmadas con EAS, TestFlight y las pistas de prueba de Google (con los 12 testers por 14 días en cuentas personales nuevas), política de privacidad y declaraciones de datos, borrar la cuenta desde la app, Sign in with Apple si hay login social, In-App Purchase para contenido digital, textos de permisos claros, un usuario de prueba para la revisión, y mantener el target SDK de Android al día cada año.

Capítulo 20

Actualizaciones OTA y CI/CD

EAS Update, runtime version, qué se puede actualizar sin pasar por la tienda, y EAS Workflows.

Publicar una corrección en la tienda lleva tiempo: compilar, subir, esperar la revisión, y que cada usuario actualice. **EAS Update** permite mandar cambios de **JavaScript, estilos e imágenes** directo a las apps ya instaladas, en minutos. Es una de las grandes ventajas de React Native frente a una app 100 % nativa.

Cómo funciona

Una build tiene dos capas:

- **La capa nativa**: el código compilado (React Native, Hermes, las librerías nativas, los permisos, el ícono). No se puede cambiar sin una build nueva.
- **La capa de JavaScript**: tu bundle y tus imágenes. Esta se puede reemplazar con una actualización.

Al abrir la app, `expo-updates` pregunta si hay una actualización para su **canal** y su **runtime version**. Si hay, la descarga en segundo plano y la aplica **la próxima vez que se abre** la app (por defecto).

Configurar

```
npx expo install expo-updates
eas update:configure    # agrega runtimeVersion y
  updates.url a app.json
```

Las builds se conectan a un canal con `channel` en `eas.json` (la app de la guía ya lo tiene: `development`, `preview`, `production`). Después de compilar con esos perfiles, publicar es:

```
eas update --channel production --environment production
  --message "Arreglo del filtro de búsqueda"
```

Runtime version: que la actualización sea compatible

Una actualización solo puede llegar a las builds cuya parte nativa sea compatible. Si agregaste `expo-camera` en el JavaScript nuevo, pero la build instalada no tiene el código nativo de la cámara, la app crashearía. La **runtime version** es la etiqueta que lo evita: una

actualización solo llega a builds con la misma runtime version.

```
// app.json
"runtimeVersion": { "policy": "fingerprint" }
```

Política	Cuándo cambia la runtime version
`"appVersion"`	Cuando subís `version`. Simple, pero si agregás una librería nativa y te olvidás de subir la versión, mandás una actualización incompatible.
`"fingerprint"`	Cuando cambia **cualquier cosa nativa** (calcula una huella de las dependencias nativas y la configuración). La más segura.
Un string fijo (`"1.0.0"`)	Cuando vos lo cambiás a mano.

Qué se puede mandar por OTA y qué no

Se puede	No se puede (necesita build y tienda)
Corregir un bug en tu código	Agregar o actualizar una librería nativa
Cambiar textos, estilos, colores, pantallas	Cambiar permisos, el ícono, el nombre, el splash
Agregar imágenes y pantallas nuevas	Actualizar el SDK de Expo o React Native

Se puede	No se puede (necesita build y tienda)
Cambiar la lógica de un flujo	Cambiar cualquier cosa de `app.json` que afecte lo nativo

> **⚠️ Las reglas de las tiendas también aplican**
>
> Apple y Google permiten actualizar código interpretado (JavaScript) sin revisión, **siempre que no cambies el propósito de la app** ni agregues funciones que esquiven la revisión. Corregir bugs y mejorar pantallas, sí; convertir una app de notas en un casino, no. Cambios grandes, por la tienda.

Desplegar con cuidado

- **Probar antes**: publicá primero en el canal `preview` (las builds de preview lo leen) y probalo. Desde un development build también podés abrir cualquier actualización.
- **Rollouts**: `eas update --channel production --rollout-percentage 10` la manda al 10 % de los usuarios. Si los errores no suben, la ampliás.
- **Rollback**: `eas update:rollback` vuelve a la actualización anterior (o a la que venía en la build).
- **Recuperación automática**: si una actualización crashea al arrancar, `expo-updates` vuelve sola a la versión anterior que funcionaba.

Aplicar la actualización enseguida

Por defecto se aplica al **siguiente** arranque. Para una corrección urgente, la app puede chequear y recargar:

```
import * as Updates from 'expo-updates'

function AvisoDeActualizacion() {
  const { isUpdatePending } = Updates.useUpdates()
  if (!isUpdatePending) return null
  return <Button title="Hay una versión nueva:
    reiniciar" onPress={() => Updates.reloadAsync()} />
}
```

CI/CD con EAS Workflows

EAS Workflows es el CI/CD de Expo: archivos YAML en `.eas/workflows/` que corren en la nube de EAS cuando hacés push (o a mano). Un flujo típico: en cada push a `main`, si cambió algo nativo hace una build nueva y la manda a las tiendas; si no, publica una actualización OTA.

```
# .eas/workflows/produccion.yml (solo Android; para iOS
  se repiten los mismos pasos)
name: Producción

on:
  push:
    branches: ['main']

jobs:
  huella:
```

```
    type: fingerprint                  # una huella de
      todo lo nativo del proyecto
    environment: production

  buscar_build:
    needs: [huella]
    type: get-build                    # ¿ya hay una
      build de producción con esta huella?
    params:
      fingerprint_hash: ${{
        needs.huella.outputs.android_fingerprint_hash }}
      profile: production

  build:
    needs: [buscar_build]
    if: ${{ !needs.buscar_build.outputs.build_id }}   #
      no hay: cambió algo nativo, compilar
    type: build
    params:
      platform: android
      profile: production

  enviar:
    needs: [build]
    type: submit
    params:
      build_id: ${{ needs.build.outputs.build_id }}

  actualizar:
    needs: [buscar_build]
    if: ${{ needs.buscar_build.outputs.build_id }}    #
      hay: alcanza con una actualización OTA
    type: update
    params:
      branch: production
      platform: android
```

```
eas workflow:run produccion.yml   # correrlo a mano
```

Hay tipos de job prehechos para casi todo: `build`, `submit`, `testflight`, `update`, `fingerprint`, `get-build`, `maestro` (tests E2E), `slack` (avisar). Es el flujo que Expo llama *golden workflow*. Si ya usás GitHub Actions, también podés correr `eas build` y `eas update` desde ahí con un token (`EXPO_TOKEN`).

El ciclo completo

1. Desarrollás con `npx expo start` (Expo Go o development build).
2. Push a una rama → build de **preview** → la prueba el equipo (o un cliente) con un link.
3. Merge a `main` → build de **producción** + `eas submit` → TestFlight y prueba interna de Google → revisión → tiendas.
4. Bug en producción → corregís → `eas update` al canal `production` → los usuarios lo tienen al abrir la app.
5. Cambio nativo (librería, permiso, SDK nuevo) → otra vez el paso 3.

Parte 5 · Repaso

Todo junto para repasar antes de una entrevista.

Capítulo 21

Chuleta y preguntas de entrevista

React Native y Expo en una página, y las preguntas que más se repiten.

Todo en una página, para repasar antes de una entrevista o tener a mano en un proyecto.

Componentes y estilos

```
import { FlatList, Modal, Pressable, ScrollView,
  StyleSheet, Text, TextInput, View } from 'react-
  native'
import { Image } from 'expo-image'
import { SafeAreaView, useSafeAreaInsets } from 'react-
  native-safe-area-context'

<View style={[styles.caja, activo && styles.activo]}>
  {/* array: el último gana */}
  <Text numberOfLines={1}>Texto, siempre dentro de
    Text</Text>
```

```
</View>
<Pressable onPress={f} onLongPress={g} hitSlop={8}
  style={({ pressed }) => [styles.boton, pressed && {
  opacity: 0.7 }]} />
<TextInput value={v} onChangeText={setV}
  keyboardType="email-address" autoCapitalize="none"
  returnKeyType="next" />
<Image source={{ uri }} style={{ width: 48, height: 48
  }} contentFit="cover" placeholder={{ blurhash }} />
<ScrollView contentContainerStyle={{ padding: 16, gap:
  12 }} keyboardShouldPersistTaps="handled" />
<Modal visible={abierto} transparent
  animationType="fade" onRequestClose={cerrar} />
<SafeAreaView edges={['top']} style={{ flex: 1 }} />

const styles = StyleSheet.create({
  caja: { flex: 1, flexDirection: 'row', alignItems:
    'center', gap: 8, padding: 16 },  // sin unidades
  sombra: { boxShadow: '0 2px 8px rgba(0,0,0,0.12)' },
})
// flexDirection: 'column' y flexShrink: 0 por defecto ·
  sin cascada · sin hover
Platform.OS; Platform.select({ ios: a, android: b,
  default: c }); Archivo.ios.tsx / .android.tsx /
  .web.tsx
const { width } = useWindowDimensions(); const esquema =
  useColorScheme()
```

Listas

```
<FlashList                                          // o
  FlatList / @legendapp/list
  data={items}
  keyExtractor={(i) => String(i.id)}                // nunca
    el índice
  renderItem={({ item }) => <Fila item={item} />}
  onEndReached={() => hasNextPage && !isFetchingNextPage
    && fetchNextPage()}
  onEndReachedThreshold={0.5}
```

```
  refreshing={recargando} onRefresh={refetch}
  ListEmptyComponent={<Vacio />} ListFooterComponent=
    {cargandoMas ? <ActivityIndicator /> : null}
/>
// SectionList: sections=[{ title, data }] +
  renderSectionHeader
```

Navegación (Expo Router)

```
// app/_layout.tsx · app/(tabs)/_layout.tsx ·
  app/oferta/[id].tsx · app/+not-found.tsx
<Stack>
  <Stack.Protected guard={logueado}><Stack.Screen name="
    (tabs)" options={{ headerShown: false }} />
    </Stack.Protected>
  <Stack.Protected guard={!logueado}><Stack.Screen
    name="login" /></Stack.Protected>
  <Stack.Screen name="filtros" options={{ presentation:
    'formSheet', sheetAllowedDetents: [0.5, 1] }} />
</Stack>

<Link href={{ pathname: '/oferta/[id]', params: { id }
  }}>Ver</Link>
router.push('/oferta/5'); router.replace('/login');
  router.back(); router.setParams({ q: 'expo' })
const { id } = useLocalSearchParams<{ id: string }>()
  // siempre strings
useFocusEffect(useCallback(() => { /* al volver a la
  pantalla */ }, []))
<Stack.Screen options={{ title: oferta.empresa }} />
  // opciones desde la pantalla
```

Datos, estado y almacenamiento

```
const { data, isPending, isError, refetch } = useQuery({
  queryKey: ['oferta', id], queryFn: () =>
  api(`/ofertas/${id}`) })
useInfiniteQuery({ queryKey, queryFn: ({ pageParam }) =>
  …, initialPageParam: 1, getNextPageParam: (u) =>
  u.siguiente })
// En el celular: onlineManager + expo-network,
  focusManager + AppState

create<Estado>()(persist((set) => ({ … }), { name:
  'clave', storage: createJSONStorage(() =>
  AsyncStorage) }))
await SecureStore.setItemAsync('token', token)
  // tokens: SecureStore, nunca AsyncStorage
const db = useSQLiteContext(); await db.sql`SELECT *
  FROM notas WHERE id = ${id}`
process.env.EXPO_PUBLIC_API_URL
  // público: nada secreto en la app
```

Animaciones y gestos

```
const x = useSharedValue(0)
const estilo = useAnimatedStyle(() => ({ transform: [{
  translateX: x.get() }] }))   // o x.value
x.set(withSpring(0)); x.set(withTiming(100, { duration:
  200 }))
const pan = usePanGesture({ onUpdate: (e) =>
  x.set(e.translationX), onDeactivate: () =>
  x.set(withSpring(0)) })
<GestureDetector gesture={pan}><Animated.View style=
  {estilo} entering={FadeInDown} /></GestureDetector>
scheduleOnRN(funcionDeReact, arg)   // desde un worklet
  al hilo de JS
```

```
// Reanimated 4: transitionProperty / transitionDuration
  / animationName como en CSS
```

Comandos

```
npx create-expo-app@latest mi-app        # proyecto
  nuevo
npx expo start                            # servidor +
  QR (i, a, w, j, r, m)
npx expo install <paquete>                # versión
  compatible con el SDK
npx expo install --fix · npx expo-doctor  # arreglar
  versiones · revisar el proyecto
npx expo prebuild · npx expo run:ios      # generar
  nativo · compilar local
npx expo config --type introspect         # ver el
  Info.plist y el AndroidManifest que se generan

eas init · eas build:configure
eas build --profile development|preview|production --
  platform ios|android|all
eas submit --platform ios|android         # o eas build
  --auto-submit
eas update --channel production --environment production
  --message "…"
eas update:rollback · eas credentials · eas env:create ·
  eas device:create
eas workflow:run archivo.yml
```

Publicar: la lista corta

	Apple	Google
Cuenta	USD 99/año	USD 25 una vez

	Apple	Google
Formato	`.ipa`	`.aab`
Pruebas	TestFlight (interno sin revisión, externo con revisión)	Pistas interna, cerrada, abierta
Antes de producción	—	Cuentas personales nuevas: 12 testers × 14 días
Revisión	24–48 h normalmente	Horas a 7 días
Requisito técnico 2026	Xcode 26 / SDK iOS 26	Target API 36
Siempre	Política de privacidad, declarar datos, borrar cuenta en la app, usuario de prueba, permisos justificados, IAP para contenido digital	Lo mismo + declaraciones de contenido, selector de fotos
Solo Apple	Sign in with Apple si hay login social, privacy manifest, ATT para tracking, estado de trader en la UE	

Preguntas de entrevista

🎤 ¿React Native es una WebView?

No. Los componentes se traducen a vistas nativas (UIView, android.view.View). El JavaScript corre en Hermes y se comunica con lo nativo vía JSI. Una WebView (como Cordova o Ionic con Capacitor) dibuja HTML.

🎤 ¿Qué es la Nueva Arquitectura?

JSI reemplaza al bridge asincrónico: JavaScript llama a C++ directo. Fabric es el renderer nuevo (soporta las features concurrentes de React y layout sincrónico), TurboModules carga los módulos nativos a demanda, y Codegen genera los tipos entre JS y nativo. Es la única arquitectura desde el SDK 55.

🎤 ¿Expo Go o development build? ¿Qué es prebuild?

Expo Go trae compiladas solo las librerías del Expo SDK: sirve para aprender y prototipar. Un development build es tu propia versión con tus librerías nativas, y es lo que se usa en proyectos reales. Prebuild (CNG) genera las carpetas `ios/` y `android/` desde `app.json` y los config plugins; no se editan a mano ni se suben a git.

🎤 ¿Cómo optimizás el rendimiento?

Medir primero en una build de release y en un Android barato. Listas con FlashList o Legend List, filas livianas y memoizadas (el React Compiler lo hace solo), imágenes con expo-image y tamaño fijo, animaciones en el hilo de UI con Reanimated, selectores en Zustand para no re-renderizar de más, no bloquear el hilo de JS, y cargar lo mínimo al arrancar.

🎤 ¿Por qué se traba una animación?

Porque depende del hilo de JavaScript y ese hilo está ocupado. La solución: Reanimated (worklets en el hilo de UI) o `useNativeDriver: true` con Animated, y Gesture Handler para gestos.

🎤 ¿Dónde guardás el token? ¿Y los secretos?

El token en SecureStore (Keychain / Keystore). Los secretos de la app no van en la app: todo lo que está en el bundle se puede leer, incluidas las variables `EXPO_PUBLIC_`. Van en el backend.

🎤 ¿Qué se puede actualizar por OTA?

Todo lo que es JavaScript, estilos e imágenes, con EAS Update, a las builds con la misma runtime version. Lo nativo (librerías nativas, permisos, ícono, SDK) necesita una build nueva y pasar por la tienda. Y siempre respetando que no cambie el propósito de la app.

🎤 ¿Cómo manejás la navegación y la autenticación?

Expo Router: rutas como archivos, Stack y pestañas nativas, parámetros tipados con typed routes. Para el login, `Stack.Protected` con un guard según haya token; el token en SecureStore, hidratado con el splash visible para que no parpadee el login; y el backend valida el token en cada pedido.

🎤 ¿Qué diferencias hay entre iOS y Android que tengas que manejar?

El botón "atrás" de Android (onRequestClose en modales), el gesto de volver de iOS, los permisos (en iOS solo se pregunta una vez), los textos de permisos en iOS, las sombras (hoy `boxShadow` unifica), las fuentes, el teclado (`KeyboardAvoidingView` con `behavior` distinto), las notificaciones (canales en Android), y los requisitos de cada tienda.